

OOP

目录

OOP	1
○ 哇好神秘还有绪论课 2026-02-28	8
壹 编译源程序 2026-03-03	8
○、函数重载深入	8
为什么需要函数重载?	8
返回值不能区分重载	8
重载与自动类型转换	8
缺省参数的规则	8
函数重载与缺省值的二义性（保留原有内容）	9
二、extern 关键字	9
三、宏定义/常量	9
四、Make	10
五、函数重载：一个函数名字，两个以上实现方法。	10
六、auto, decltype	10
auto	11
decltype	11
auto 的其他限制	11
范围 for 循环（C++11）	11
七、NULL vs nullptr	11
NULL 的定义	11
NULL 的重载解析问题	12
八、内存申请和释放	12
贰 对象 2026-03-10	12
一、访问权限：	12
二、this 指针	13
三、内联函数	13
函数调用的开销	13
内联的本质	13
宏的常见陷阱	13
陷阱 1：运算符优先级	13
陷阱 2：副作用的双重求值	13
四、new delete	14
叁 构造函数和析构函数 2026-03-17	14
一、构造函数：诞生	14
委派构造函数：调用其他构造函数。	14
就地初始化：	14
默认/缺省构造函数：	15
对象数组的初始化：	15
二、析构函数：死亡	15
三、局部对象的构造和析构	16
四、全局对象的构造和析构	16
五、引用	16
引用作为返回值：函数调用出现在等号左边	17
为什么引入引用？——运算符重载的需要	17
swap 三种方式对比	17
六、运算符重载	18
前缀后缀重载区分	18
() 重载	18
[] 重载	18
为何 =、[]、()、-> 只能重载为成员函数?	18

前缀与后缀 ++ 的语义	19
全局函数的 ++ 重载	19
流运算符的重载与调用链	19
流运算符重载	19
七、友元 2026-03-24	19
肆 静态变量和静态函数 常量	21
○、C 语言 static 复习	21
静态局部变量	21
静态全局变量与函数（内部链接）	21
多文件链接错误示例	21
一、静态变量，静态函数	21
静态数据成员 / 类变量	21
静态成员函数	22
编译错误示例	22
二、常量	22
常量数据成员	22
常量成员函数	22
常量对象	22
常量静态变量	23
再谈常量对象和静态对象：构造和析构	23
综合示例：各种对象的构造析构顺序	24
伍 参数对象的构造和析构	25
○、拷贝控制与位拷贝的真相	25
参数传递：构造与析构顺序图解	25
浅拷贝问题：最简示例	25
拷贝构造函数的三种调用场景	25
避免拷贝构造的方法：代码示例	26
○○、左值与右值深入	26
编译器错误示例	26
引用的绑定优先级	26
拷贝构造 vs 移动构造：直观图解	26
swap 的两种实现对比	26
○○○、编译器自动合成的函数（重要补充）	27
○○○○、参数传递：常量引用与最小特权原则	27
一、参数中的常量和常量引用	28
二、拷贝构造函数	28
拷贝构造函数：实例 1	29
拷贝构造函数：实例 2	30
问题？	31
三、内联函数、左值引用与右值引用	31
左值与右值	31
右值引用	31
引用绑定规则	31
右值引用示例	32
右值引用的意义	32
四、移动构造函数	32
移动构造函数 vs 拷贝构造函数	32
移动构造函数示例	33
移动构造函数实例	33
情况一：开启返回值优化（默认）	33
情况二：禁止返回值优化 + 有移动构造函数	34
情况三：禁止返回值优化 + 删除移动构造函数	34
什么时候调用移动构造函数？什么时候调用拷贝构造函数？	34
拷贝构造函数的调用时机	34

移动构造函数的调用时机	35
返回值优化 (RVO)	35
std::move 函数	35
构造函数综合实例	35
十、拷贝赋值运算符与移动赋值运算符	38
拷贝赋值运算符	38
拷贝赋值运算符实例	38
移动赋值运算符	38
拷贝/移动赋值运算符的调用时机	39
编译器自动合成的函数/运算符	39
十一、类型转换	39
类型转换概述	39
方法一：类型转换运算符	39
方法二：类型转换构造函数	40
两种方法的使用	40
类型转换运算符：错误示例	41
类型转换实例	41
禁止自动类型转换	42
强制类型转换	42
强制类型转换示例	43
陆 组合与继承 2026-04-07	43
○、本讲概览	43
一、对象(类)之间的关系	43
二、组合	44
对象组合示例	44
子对象构造	44
对象组合示例：构造与析构	44
对象组合运行结果	45
隐式定义的拷贝构造与赋值运算	45
对象组合示例：拷贝与赋值	46
三、继承	46
基本概念	47
什么不能被继承?	47
继承示例	47
派生类对象的构造与析构过程	48
调用基类构造函数：隐式调用默认构造	48
调用基类构造函数：显式调用	48
继承基类构造函数 (using)	49
继承基类构造函数 (多个构造函数)	49
继承基类构造函数 (注意事项)	49
如何选择继承方式?	50
public 继承	50
private 继承	50
成员访问权限	50
公有继承：基类公有成员的访问	50
私有继承：基类公有成员的访问	50
私有继承：用 using 打开基类公有成员的访问权限	51
私有继承：基类私有、保护成员访问	51
基类成员访问权限与三种继承方式	52
组合与继承的对比	52
组合示例：has-a	52
继承示例：is-a	53
四、重写隐藏与重载	53
重载 (overload)	53

重写隐藏 (redefining)	54
函数重写隐藏示例	54
恢复基类成员函数示例	54
using 关键字总结	55
五、多重继承	55
多重继承的问题	55
多重继承示例	55
六、课后练习	56
柒 虚函数 2026-04-14	57
○、OOP 三大核心思想	57
一、向上类型转换	57
二、对象切片	58
对象切片: sizeof 证明	58
对象切片: 方法也会丢失	58
三、指针 (引用) 的向上转换	59
四、函数调用捆绑	59
五、虚函数和虚函数表	59
虚函数表 (VTABLE)	60
VTABLE 内存布局: sizeof 验证	60
六、虚函数和构造函数、析构造函数	61
虚函数与构造函数	61
虚函数与析构造函数	61
七、重写覆盖与重写隐藏	62
三者对比	62
重写覆盖示例	62
八、override 和 final 关键字	62
override	62
final	63
九、const 对重写覆盖的影响 (课后探究)	63
十、虚函数的返回值 (课后探究)	63
协变返回类型: 综合示例	63
十一、课后练习	64
附录: 利用返回值优化提高执行效率	64
捌 多态与模板 2026-04-21	65
○、完整抽象类示例: 谁还是抽象的?	65
一、纯虚函数与抽象类	65
二、纯虚析构造函数	66
三、向下类型转换	66
dynamic_cast	66
static_cast: 不安全示例 (含垃圾值输出)	66
dynamic_cast 与 static_cast 对比	67
向下类型转换示例	67
四、多重继承中的虚函数	67
五、多态 (Polymorphism)	68
Template Method 设计模式	68
六、函数模板和类模板	69
函数模板	69
函数模板示例	69
模板原理	70
为什么模板必须在头文件中实现? (编译链接流程详解)	70
模板参数必须在编译期确定	70
函数模板编译错误实例	70
类模板	71
类模板示例	71

七、模板与多态	72
八、成员函数模板（自学）	72
九、课后练习	72
练习 1	72
练习 2	73
玖 模板与 STL 初步 2026-04-28	73
一、命名空间	73
二、C++ 标准演进与 STL	73
C++ 标准版本概览	73
三、STL 简介	73
三、STL 容器	73
pair	74
tuple (C++11)	74
vector	74
vector 原理	75
迭代器失效：指针追踪示例	75
遍历时修改容器的陷阱	75
四、迭代器	75
迭代器失效	76
五、STL 其他容器	76
list	76
set	76
map	76
关联容器原理	77
容器总结	77
容器选择四大准则	77
六、string 字符串类	77
构造方式	77
常用函数	78
输入方式	78
拼接与比较	78
数值转换	78
七、同义词查询库（自学）	78
八、函数模板与类模板特化（自学）	79
函数模板特化	79
类模板特化	79
模板特化总结	80
函数模板重载解析详解	80
函数模板两种特化语法	81
拾 iostream 与函数对象和智能指针 2026-05-12	81
一、iostream 输入输出流	81
STL 输入输出流层次	81
ostream 和 cout	81
实现自己的 ostream	81
格式化输出	82
流操纵算子	82
endl	82
不能复制的 cout	83
文件输入输出流	83
istream 与 scanf	84
字符串输入输出流	84
类型转换函数	84
二、函数对象	85
引例：一个计算器的三种实现方式	85

方式一： OOP + 模板方法	85
方式二： 函数指针	85
方式三： 函数模板（静多态）	85
方式四： std::function（统一接口）	85
函数指针	86
函数作为变量	86
函数对象	86
如何实现函数对象	87
自己实现 sort	87
自定义类型的排序	87
std::function 类	88
STL 与函数对象	88
三、智能指针与引用计数	89
shared_ptr	89
引用计数	89
实现自己的引用计数	89
weak_ptr： 解决循环引用	90
unique_ptr： 独享所有权	91
智能指针总结	91
四、字符串处理与正则表达式（自学）	91
正则表达式基础	91
引例： 用户名验证	91
字符簇（使用“The car parked in the garage.”演示）	92
重复模式	92
或连接符	92
C++ <regex> 库	92
匹配： regex_match	93
捕获和分组	93
搜索： regex_search	93
替换： regex_replace	93
例题： 学生信息提取	93
更多正则内容（自学）	94
五、拓展自学： function 绑定优先级	94
拾壹 行为型模式 2026-05-19	94
一、设计模式概述	94
二、引例： 负载监视器	95
简单枚举（不好）	95
三、模板方法（Template Method）模式	95
生活类比： 写作文	95
代码实现	96
针对接口编程	96
开放封闭原则	96
模板方法的局限	96
四、策略（Strategy）模式	97
代码实现	97
单一责任原则	98
模板方法 vs 策略模式	98
五、迭代器（Iterator）模式	99
引例： 遍历学生成绩（逐步演进）	99
起点： 原始数组实现	99
提取为函数	99
需求变化： 改用链表存储	99
“变”与“不变”	100
迭代器基类	100

Collection 基类	100
数组实现	100
使用迭代器	101
另一种迭代器接口	101
STL 中的迭代器	101
迭代器模式总结	102
六、总结	102
拾贰 结构型模式 2026-05-26	103
一、引例：栈	103
堆栈基类	103
简单实现	103
二、适配器（Adapter）模式	103
现实类比：手机充电	103
适用场景	104
实现一：组合方式（对象适配器模式）	104
实现二：继承方式（类适配器模式）	105
适配器模式小结	106
三、代理/委托（Proxy）模式	106
从适配器到代理：为什么需要新模式？	106
场景	106
权限检查示例	106
代理 vs 适配器：相似与不同	106
例子：智能指针引用计数	107
“变”与“不变”	107
代理 vs 适配器	107
四、装饰器（Decorator）模式	108
引例：TextView	108
用继承？——类爆炸	108
用策略？——策略个数固定	108
装饰器	109
调用链（逐层展开）	110
装饰器的核心内涵	111
装饰 vs 策略 vs 代理	111
五、总结	111
拾叁 创建型模式 2026-06-02	111
一、单例（Singleton）模式	111
引例：全局计数器	111
方法一：全局变量	111
方法二：静态方法	111
静态方法 + 虚函数 = 编译错误	112
单例模式实现	112
惰性初始化（Lazy Initialization）	113
单例模式的陷阱	113
单例模式 + 虚函数	113
CRTP：奇特的递归模板模式	114
CRTP + 多重继承	114
关于 CRTP	115
关于单例模式	115
二、工厂方法（Factory Method）模式	115
引例：泡茶程序	115
工厂方法	116
单独的工厂类	116
工厂方法的用途	116
构造函数的描述性名称	116

一个现实的例子: Box2D	117
Box2D 中的主要类型	117
三、抽象工厂 (Abstract Factory) 模式	118
工厂方法的局限性	118
编译器前端设计	118
添加一层抽象	118
抽象工厂模式	119
框架设计理念	119
四、设计模式总结	120
三大设计模式回顾	120
行为型模式 (Behavioral Patterns)	120
结构型模式 (Structural Patterns)	120
创建型模式 (Creational Patterns)	120
七大设计原则	120

○ 哇好神秘还有绪论课 2026-02-28

我去，居然是雷霆方法论基础？！结构化？！面向对象？！神秘秘神？！

不能反直觉反常识？！强强！？ 对象？ 我的对象在哪里？！弱弱,,, 属性和操作？！ 静态和动态？！ 抽象？！
过程抽象和数据抽象？！ eg. 类型是什么？！ 数据的存储和表示+数据支持的操作？！

壹 编译源程序 2026-03-03

幻灯片 L3 涵盖函数重载、auto/decltype、nullptr、范围 for、内联函数等，以下为全面补充版本。

○、函数重载深入

为什么需要函数重载？

1. 同一任务，输入类型不同（如 print 可以接受 const char* 也可以接受 int）
2. 同一任务，存储形式不同（如排序 int 数组 vs float 数组）
3. 概念抽象层面相似的任务（如输出到屏幕、打印机、文件）

返回值不能区分重载

```
float f(int s) { return s / 2.0; }
int f(int s) { return s * 2; }
// cout << f(3) << endl; // 二义性！编译器不知道调用哪个
```

重载与自动类型转换

当函数重载时，会优先调用类型匹配的函数实现，否则才会进行类型转换：

```
void print(float x) { cout << "float: " << x << endl; }
void print(int x) { cout << "int: " << x << endl; }
print(1.7); // float: 1.7 (类型匹配)
print(1); // int: 1 (类型匹配)
print('c'); // int: 99 (char→int, 优先匹配int而非float)
```

类型转换方向：float→int 会截断（向零取整，1.7→1，-3.9→-3），int→float 会提升。

缺省参数的规则

```
void print(const char* msg = "hello") { cout << msg << '#'; }
print(); // 输出: hello#
print("Beijing"); // 输出: Beijing#
```

规则：有缺省值的函数参数必须是最后一个参数；多个缺省参数必须连续出现在参数列表末尾。

函数重载与缺省值的二义性（保留原有内容）

关键在于参数类型的不同。但是如果出现了参数的缺省，就会产生二义性。举例：argv argc 等参数的运用
实际意义上的一个完整编译流程如下：

```
g++ -c ex1.cpp
g++ -o ex1 ex1.o
```

第一步是编译，第二步是链接。注意到还有多文件链接。

```
g++ -c ex1.cpp ex1.o
g++ -c func1.cpp func1.o
g++ ex1.o func1.o -o ex1
```

当然我们平时偷懒是可以的。只是想告诉你这个编译和链接的实际过程，第二步把两个文件里的函数链接了？！

链接文件里面声明和定义不相同会类型报错。 #include 的本质是把代码直接复制下来了。**声明可以多声明，定义只能定义一次。** 声明的时候可以修改只说参数类型，不喊名字。**变量也可以声明或者定义。即使不赋值也可以被定义！** 注意下面的例子：

```
// num.cpp
int a=1;
```

```
// ex1.cpp
int a;
int main(){
    a=1;
    return 0
}
```

这时就炸了。重复定义。 **定义=声明+内存分配**

二、extern 关键字

有一点点烧脑。用于全局变量的共享。

```
//num.h
extern int a;
extern int b;
extern int c;
extern int d[10];

//num.cpp
#include<num.h>
a=1;
b=2;
c=3;
```

头文件中只声明，不定义，目的是防止重复定义。

三、宏定义/常量

请使用 const 以保证类型的正确性！对于宏：

```
#define name(value) command_string
#define sqr(x) ((x)*(x))

sqr(3+2) = 25
```

请注意这个时候括号十分重要。因为不带括号的时候只是把串原封不动地搬运过来。宏定义可以用来防止头文件被反复包含。

```
// Method 1
#ifndef _HEAD_
#define _HEAD
...(如声明/定义)
#endif
// Method 2
#pragma once
```

还可以用来神秘的 debug。

```
#define DEBUG
#ifdef DEBUG
debug();
#endif
```

四、Make

Makefile 编写规则：只编译并链接被修改的文件，其他不管。但是如果头文件被改变了，所有引用了这个头文件的都要重新编译。格式举例：

```
all: main test
main: main.cpp stu.cpp
    g++ -o main main.cpp stu.cpp
test: stu.cpp stu_test.cpp
    g++ -o test stu_test.cpp stu.cpp
clean:
    rm main test

//bash 举例
make all
make test
make main
make clean
```

Make 的更多用法还是自己去查吧。氧气和臭氧优化后不能 gdb。

五、函数重载：一个函数名字，两个以上实现方法。

关键在于参数类型的不同。但是如果出现了参数的缺省，就会产生二义性。

举例：

```
int fun(int a, int b=0);
int fun(int a);
```

会产生二义性。因为当调用 fun(1) 的时候，编译器无法判断是调用第一个函数还是第二个函数。

但是！

```
int fun(int a);
int fun(float a);
```

就不会产生二义性。因为当调用 fun(1) 的时候，编译器会选择第一个函数，因为参数类型是 int，而不是 float。

六、auto, decltype

auto 和 decltype 都是 C++11 引入的类型推断机制，可以编译器根据上下文自动推断变量的类型。

auto

```
auto x = 5; // x 的类型被推断为 int
auto y = 3.14; // y 的类型被推断为 double
auto z = "Hello"; // z 的类型被推断为 const char*
```

- auto 必须在定义时初始化。
- auto 不能用于函数参数声明。
- auto 不能用如 sizeof, typeid 等上下文中。
- 在定义模板函数时用于声明依赖模板参数的返回类型：

```
template<typename _Tx, typename _Ty>
void Multiply(_Tx x, _Ty y){
    auto v = x * y; // 返回类型由编译器根据表达式 x * y 推断
    std::cout << v << std::endl;
}
Multiply(3, 4.5); // 输出 13.5, v 的类型被推断为 double
```

decltype

- 重用匿名类型：

```
struct {
    int a;
    double b;
} myStruct;
decltype(myStruct) anotherStruct; // anotherStruct 的类型与 myStruct 相同
```

除此之外还可以用来追踪推导 auto 返回的类型，在 C++ 14 后不必要：

```
auto func(int x, int y) -> decltype(x+y) {
    return x + y; // 返回类型被推断为 int
}
```

auto 的其他限制

1. auto b4 = 10, b5 = 20.0, b6 = 'a';
2. auto 不是真正的类型，不能用于 sizeof(auto) 或 typeid(auto)

范围 for 循环 (C++11)

```
int arr[3] = {1, 3, 9};
for (int e : arr) // auto e : arr 也可以
    cout << e << endl; // 输出 1 3 9
// 等价于遍历容器中的每个元素
```

七、NULL vs nullptr

NULL 的定义

```
#ifdef __cplusplus
#define NULL 0 // C++ 中 NULL 就是整数 0
#else
#define NULL ((void *)0) // C 中是 void* 类型的 0
#endif
```

NULL 的重载解析问题

```
void f(int x, int y)      { cout << "int" << endl; }
void f(int x, double *y) { cout << "pointer" << endl; }

f(2, NULL);              // 调用 f(int, int)! 而非指针版本!
f(2, static_cast<double *>(0)); // 强制调用指针版本 (但写法丑陋)
f(2, nullptr);           // C++11: 正确调用指针版本!
```

教训： NULL 本质是整数 0，在重载解析时会被当作 int 而非指针。应始终使用 nullptr。

八、内存申请和释放

```
int* p = new int(10); // 申请一个 int 类型的内存
int *arr = new int[5]; // 申请一个 int 类型的数组
delete p; // 释放单个 int 类型的内存
delete[] arr; // 释放数组内存
```

nullptr 是 C++11 引入的空指针常量，表示一个空指针。使用 nullptr 可以避免与整数 0 混淆，提高代码的可读性和安全性。

贰 对象 2026-03-10

成员函数必须在类的内部声明，但是可以在类的外部定义（实现）。

举例：

```
class MyClass {
public:
    void myFunction(); // 成员函数声明
};
void MyClass::myFunction() { // 成员函数定义
    // 函数体
}
```

一、访问权限：

- public: 公共成员，类的外部可以访问。
- private: 私有成员，只有类的内部可以访问。也是默认访问权限。
- protected: 受保护成员，只有类的内部和派生类可以访问。

可以通过 object.member 的方式访问对象的成员函数和成员变量。或者 pointer->member 的方式访问指针指向对象的成员函数和成员变量。在类外只能访问 public 成员。

神秘： 在类的内部可以访问传入的同一个类的对象的 private 成员！因为它们属于同一个类。

举例：

```
class MyClass {
private:
    int privateVar;
public:
    void setPrivateVar(int value) {
        privateVar = value; // 访问自己的 private 成员
    }
    void accessOther(MyClass& other) {
        other.privateVar = 10; // 访问同类的其他对象的 private 成员
    }
};
```

二、this 指针

指向当前对象的指针。可以在成员函数内部使用 this 来访问当前对象的成员变量和成员函数。

举例：

```
class MyClass {
private:
    int value;
public:
    MyClass(int v) : value(v) {}
    void printValue() {
        cout << "Value: " << this->value << endl;
    }
};
```

错综复杂的类外类内关系.....阿弥诺斯，小白手套欸呦喂，我这不赖，已取餐。

三、内联函数

函数调用的开销

函数调用要进行一系列准备和后处理工作（压栈、跳转、退栈、返回等），所以函数调用是一个比较慢的过程。

```
cout << max(a, b) << endl;           // 函数调用（慢）
cout << (a > b ? a : b) << endl;      // 等价表达式（快得多！）
```

内联的本质

编译器自动将内联函数替换为等价的表达式。例如 inline int max(int a, int b) 定义后，max(a,b) 会直接被替换为 (a > b ? a : b)。

宏的常见陷阱

由于和宏代码是直接拷贝复制代码到指定位置，会产生莫名其妙的边际效应，你永远无法想象到各种优先级问题，即使加了括号也不一定能完全解决。

陷阱 1：运算符优先级

```
#define MAX(a, b) (a > (b) ? (a) : (b))
cout << MAX(a, b) + 2 << endl;
// 展开为: cout << (a) > (b) ? (a) : (b) + 2 << endl; // 错误！
// 正确写法: #define MAX(a, b) ((a) > (b) ? (a) : (b))
```

陷阱 2：副作用的双重求值

```
#define MAX(a, b) ((a) > (b) ? (a) : (b))
cout << MAX(a++, b) << endl;
// 展开为: cout << ((a++) > (b) ? (a++) : (b)) << endl;
// a 被求值两次！结果不可预测！
```

相比之下，**内联函数**可以执行类型检查，可调试。它的本质是生成和函数等价的表达式。并且**宏定义**无法调用私有成员函数，而内联函数可以。宏使用的最常见场景：字符串定义、字符串拼接、标志粘贴。

注意事项：

- 避免大段代码的内联，因为会增加编译时间和可执行文件的大小。
- **避免对包含循环、递归或复杂逻辑的函数使用内联。**
- 避免将声明和定义分开，因为内联函数必须在调用之前定义。
- 定义在类内部的成员函数默认是内联的，但这只是一个建议，编译器可能会选择不内联。
- **一般构造函数、析构函数**通常会被编译器自动内联。

inline **永远只是建议修饰，不是命令**。编译器有权拒绝不合理的内联请求。

四、new delete

new 生成一个类对象，并且返回它的指针。

```
A *pA = new A(1);
```

delete 释放一个类对象。

叁 构造函数和析构函数 2026-03-17

回顾：要对用户定义类型进行严格检查；要隐藏属性；要自动化对象的初始化和清除，做好对象内存分配等等。

一、构造函数：诞生

- 构造函数没有返回值
- 构造函数名与类名相同
- 构造函数可以重载

举例：

```
class Student {
    int ID;
    public:
        Student(int id) : ID(id) {}
        Student(int year, int order) {
            ID = year * 10000 + order;
        }
};
```

注意！按照声明顺序而不是初始化顺序排列成员！如下面 ID1 不可预测。

```
class Student {
    int ID1;
    int ID2;
    public:
        Student(int id) : ID2(id), ID1(ID2) {}
};
```

委派构造函数：调用其他构造函数。

```
class Info{
    public:
        Info(int a):a(a){}
        Info(int a, int b):Info(a){b=b;}
        Info(int a, int b, int c):Info(a,b){c=c;}
};
```

就地初始化：

```
class A{
    private:
        int a{0};
        double b=2.0;
    public:
        A(){}
};
```

默认/缺省构造函数：

```
class A{
    public:
        A(){};
};

A a;
A b=A();
A c();
// 这个时候 c 是一个函数声明，而不是一个对象声明。它可以生成一个对象。

//显式默认构造函数
class A{
    public:
        A()=default;
};

//显式删除默认构造函数
class A{
    public:
        A()=default;
        A(int i){}
        A(char cls)=delete; // 禁止从 char 构造！
};
A a('c'); // 编译错误！
// 为什么需要？因为 'c'→int→A(int)，隐式转换太危险！
```

一个实例生成的时候，会先调用成员的构造函数（如果成员没有就默认构造），再构造自己的。

如果你已经定义了构造函数，编译器不会帮你默认构造了！！

对象数组的初始化：

重要：无参定义对象数组，必须要有默认构造函数！ 因为 `A a[50]`；需要对每个元素调用默认构造。

```
A a[50];
// 只有一个参数
A a[3]={1,2,3};
//多个参数
A a[3]={A(1,2),A(3,4),A(5,6)};
```

二、析构函数：死亡

- 没有参数
- 唯一
- 没有返回值

举例：

```
class MyClass {
    int num;
    int* ID_list;
public:
    MyClass() { // 构造函数
        // 构造函数体
    }
    ~MyClass() { // 析构函数
        // 析构函数体
        if(ID_list) delete[] ID_list; // 释放内存
    }
};
```

```
    }  
};
```

析构函数会在对象生命周期结束时自动调用。比如说程序结束的时候：

```
//我们此前 Test 里面创建了一个 Member m  
  
Test t;  
int main(){return 0;}  
// 输出: ~Test() ~Member()
```

晚构造，先析构!!!

析构函数也会自动隐式定义。但是不会删除指针成员，因此可能导致内存泄漏。

三、局部对象的构造和析构

其实没啥好说的，就是看括号里面的东西。

四、全局对象的构造和析构

- 在 main() 之前构造
- 在同一编译单元中，按照定义顺序构造
- 在不同编译单元中顺序不确定
- 在 main() 执行完 return 语句后析构

尽量少用全局对象！ 全局对象之间最好不要有依赖关系，否则析构顺序会出问题。

五、引用

引用是对象的别名，可以理解为指针常量。引用必须在声明时初始化，并且一旦初始化就不能改变。

举例：

```
int a = 5;  
int& ref = a; // ref 是 a 的引用  
ref = 10; // 修改 ref 也会修改 a  
cout << a << endl; // 输出 10
```

二者绑定，改了一个另一个也会变。而且不能解绑。

有些时候和指针很像。

举例：交换两个变量的值

```
void swap(int& a, int& b) {  
    int temp = a;  
    a = b;  
    b = temp;  
}
```

看一个做错的题：

```
class Int {  
public:  
    int data;  
    Int() { data = 1; }  
    Int(int i): data(i) {}  
};  
  
void func1(Int& a, Int b) {  
    a.data += b.data;
```

```

}

Int& func2(Int& a, Int b) {
    func1(a, b);
    Int tmp(a.data + b.data);
    return tmp;
}

int main() {
    Int a, b(3);
    Int& f = func2(a, b);
    cout << a.data << "_";
    cout << f.data << endl;
    return 0;
}

```

初始状态：a.data = 1（默认构造），b.data = 3（值构造）。

调用 func2(a, b) 时，Int& a 是 main 中 a 的引用，Int b 是值拷贝。

1. func1(a, b) 执行 a.data += b.data，即 $1 + 3 = 4$ ，main 中的 a.data 变为 4。
2. Int tmp(a.data + b.data) 在栈上构造局部变量，tmp.data = $4 + 3 = 7$ 。
3. return tmp 返回 tmp 的引用。

func2 返回类型为 Int&，而 tmp 是栈帧上的局部变量。函数返回后栈帧销毁，tmp 随之消亡，main 中的 f 成为悬空引用（dangling reference），指向已释放的内存。

注意：tmp 并非“返回前被销毁”，而是函数返回后随栈帧一起消亡。引用被传递出去了，但其所指向的对象已不复存在，后续访问是未定义行为。

引用的用途：

- 作为函数参数，可以避免复制对象，提高效率。
- 作为函数返回值，可以返回引用，而不是复制对象。
- 作为类的成员变量，可以简化代码，提高可读性。
- 相对指针，引用更安全，不会出现空指针的情况。

引用作为返回值：函数调用出现在等号左边

```

int a[3] = {1, 3, 5};
int& get(int i) { return a[i]; } // 返回引用
int main() {
    get(1) += 1; // get(1) 返回 a[1] 的引用，+= 直接修改 a[1]
    cout << get(1); // 输出 4
    // 函数的返回值可以出现在等号左边！
}

```

为什么引入引用？——运算符重载的需要

回到 $a += b$ 这样的写法。如果没有引用，重载的运算符就需要写成 $a += \&b$ （传递指针），语法难看且不一致。引用让运算符重载可以保持自然的语法。

swap 三种方式对比

```

// 值传递：无效！（交换的是副本）
void swap(int a, int b) { int tmp = a; a = b; b = tmp; }
// 指针：可用但语法难看
void swap(int *a, int *b) { int tmp = *a; *a = *b; *b = tmp; }
// 引用：最优雅！（推荐）
void swap(int& a, int& b) { int tmp = a; a = b; b = tmp; }

```

六、运算符重载

举例：

```
A& operator+=(A& a){
    data+=a.data;
    return *this;
}
```

可以重载的运算符：

- 算术运算符：+、-、*、/、%
- 关系运算符：==、!=、<、>、<=、>=
- 逻辑运算符：&&、||、!
- 位运算符：&、|、^、~、<<、>>
- 单目运算符：+、-、*、&、!
- 自增自减运算符：++、--
- 赋值运算符：=、+=、-=、*=、/=、%=、&=、|=、^=、<<=、>>=
- 空间运算符：new、delete、new[]、delete[]
- 其他运算符：()、[]、->、,、->*

前缀后缀重载区分

后缀关键是引入了哑元，哑元可以没有名，所以可能实际不会被调用。

```
A operator++(int){
    A temp(data);
    ++data;
    return temp;
}
```

() 重载

```
int operator()(int a, int b){
    return a+b;
}
```

[] 重载

```
int& operator[](int i){
    return data[i];
}
```

如果是引用，则可以修改原对象。否则只能读取。

为何 =、[]、()、-> 只能重载为成员函数？

原因：当用户没有定义 operator= 时，编译器会自动合成一个。如果允许全局的 operator=，就可能与编译器自动合成的版本产生冲突——编译器无法判断该合成还是使用全局版本。

非整数下标的 operator[]：下标不一定是整数！

```
class AgentScores {
    int score[5];
public:
    int& operator[](const char* name) { // 字符串下标！
        if (strcmp(name, "memory") == 0) return score[0];
        if (strcmp(name, "logic") == 0) return score[1];
        return error_score; // 不匹配时返回默认值
    }
private:
```

```
int error_score = -1;
};
```

前缀与后缀 ++ 的语义

```
int a = ++b; // 前缀：先自增，再赋值
int a = b++; // 后缀：先赋值，再自增（返回旧值，需要哑元int区分）
```

全局函数的 ++ 重载

```
// 全局前缀++
A operator++(A& a) { a.data++; return a; }
// 全局后缀++（哑元int区分）
A operator++(A& a, int) { A tmp(a); a.data++; return tmp; }
```

流运算符的重载与调用链

为什么形参和返回值都是引用？

1. ostream 的拷贝构造函数被 = delete 禁用了——只能引用不能拷贝！
2. 返回 ostream& 是为了支持链式调用：

```
cout << obj1 << obj2 << obj3;
// 等价于：
ostream& out = operator<<(cout, obj1); // cout << obj1
ostream& out1 = operator<<(out, obj2); // out << obj2
ostream& out2 = operator<<(out1, obj3); // out1 << obj3
```

为什么流运算符必须用全局函数？ 在不修改 istream/ostream 类的前提下，只能使用全局函数重载。

注意：=, [], (), -> 这几个运算符只能重载为成员函数。否则可能会对是否自动合成重载符产生影响。

流运算符重载

```
ostream& operator<<(ostream& os, const A& a){
    os<<a.data;
    return os;
}

istream& operator>>(istream& is, A& a){
    is>>a.data;
    return is;
}
```

此时我们不得不谈到：友元。

七、友元 2026-03-24

```
#include <iostream>
using namespace std;

class Test {
    int id;
public:
    Test(int i) : id(i) { cout << "obj_" << id << " created\n"; }

    friend istream& operator>> (istream& in, Test& dst);
    friend ostream& operator<< (ostream& out, const Test& src);
};
```

```

istream& operator>> (istream& in, Test& dst) {
    in >> dst.id;
    return in;
}

ostream& operator<< (ostream& out, const Test& src) {
    out << src.id << endl;
    return out;
}

int main() {
    Test obj(1);
    cout << obj; // operator<<(cout,obj)
    cin >> obj; // operator>>(cin,obj)
    cout << obj;
    return 0;
}

```

友元是这个对象的“朋友”，可以访问这个对象的私有成员 / 保护成员。

举例：

```

class A{
    int data;
    friend B b;
    b.id=data;
}

```

友元函数举例：

```

class A{
    int data;
    friend void func(A a);
    friend void X::foo(A);
    friend X::X(Y),X::~~X();
};
A a;
func(a);

```

友元函数在类外定义，但是可以访问类的私有成员。且在 private,public 声明没有区别。

更详细的例子：

```

class Y; // 前向声明
class X
{
    int data;
    friend void func(X &x, Y &y);
};
class Y
{
    int data;
    friend void func(X &x, Y &y);
};
void func(X &x, Y &y)
{
    cout << x.data << y.data << endl;
}

```

友元类：

```
class Y {};  
class A{  
    int data;  
    friend class Y;  
    friend X;  
}  
class X{};  
// X,Y 都可以访问 A 的私有成员
```

注意：

- 友元关系是**非对称的**，即 A 是 B 的友元，B 不是 A 的友元。
- 友元关系**不传递**，即 A 是 B 的友元，B 是 C 的友元，A 不一定是 C 的友元。
- 友元关系**不具继承性**，即 A 是 B 的友元，C 继承自 B，A 不一定是 C 的友元。
- 友元声明不能直接在里面定义了。

肆 静态变量和静态函数 常量

幻灯片 L5 涵盖 static、const、new/delete 内存布局、全局/静态对象构造析构顺序等，以下为全面补充版本。

〇、C 语言 static 复习

静态局部变量

存储在静态存储区，生命周期持续到程序结束。只在第一次进入函数时初始化。

静态全局变量与函数（内部链接）

内部可链接，只能初始化一次，而且作用域仅限其声明的文件。**比如**在其他文件被 extern 声明后**不可以在**其他文件中使用。

前缀是 static。

多文件链接错误示例

```
// a.cpp  
static int i = 1;           // 内部链接  
int j = 2;                  // 外部链接（默认）  
static int add_j(int k) { return j + k; } // 内部链接  
  
// b.cpp  
extern int i;                // 链接错误！i是static的，不可见  
extern int add_j(int k);     // 链接错误！add_j是static的，不可见  
// g++ a.cpp b.cpp -o test  
// → undefined reference to 'i'  
// → undefined reference to 'add_j(int)'
```

一、静态变量，静态函数

静态数据成员 / 类变量

在整个类内所有对象共享，初始化时需要加 static。应该在 .h 文件中声明，在 .cpp 文件（实现文件）中定义，防止重复定义。

在实例化所有对象之前就已经分配了静态数据成员的内存空间。

举例：

```
// Test.h  
class Test {  
public:
```

```

    static int count;
    Test();
    ~Test();
};

// Test.cpp
#include "Test.h"
int Test::count = 0;
Test::Test() { count++; }
Test::~Test() { count--; }

// main.cpp
#include "Test.h"
int main() {
    Test a;
    Test b;
    cout << Test::count << endl; // 输出 2
    cout << a.count << endl; // 输出 2
    return 0;
}

```

静态成员函数

只能访问静态成员变量，不能访问非静态成员变量。因为非静态成员变量是每个对象独有的，而静态成员函数是所有对象共享的。

深层原因：静态成员函数属于整个类，在类实例化对象之前就已经分配了内存空间；而类的非静态成员必须在类实例化对象后才分配内存空间。相当于没有定义一个变量却要使用它。

编译错误示例

```

class A {
    int data;
public:
    static void output() {
        cout << data << endl; // 编译错误！
    }
};

// [Error] invalid use of member 'A::data' in static member function

```

二、常量

- const 修饰的变量不能被修改。
- 修饰引用/指针时，引用/指针不能被修改，指向的内容不能被修改。
- 修饰函数返回值时，函数返回值不能被修改。

常量数据成员

可以在构造函数初始化列表中初始化、就地初始化，但是不能在构造函数体通过赋值初始化。

常量成员函数

不能修改非静态数据成员，即不能改变对象的状态。

区分 const 的位置！

1. int func() const —— 常量成员函数（不能修改成员变量）
2. const int func() —— 返回常量值的函数（函数本身可以修改成员变量）
3. 两者可以同时出现：const int func() const

常量对象

只能调用常量成员函数，不能调用非常量成员函数。

```

class Student {
    int ID;
public:
    Student(int id) : ID(id) {}
    int MyID() const { return ID; } // 常量成员函数, ✓
    // int Next() const { ID++; return ID; } // 错误! const函数不能修改成员
    int Who() { return ID; } // 非常量成员函数
};
int main() {
    const Student obj(20160301);
    cout << obj.MyID(); // ✓ 常量对象调用常量成员函数
    // cout << obj.Who(); // ✗ 错误!
    // [Error] passing 'const Student' as 'this' argument of
    //      'int Student::Who()' discards qualifiers
}

```

常量静态变量

需要在类外初始化。但是有两个例外：int 和 enum 可以就地初始化。

不存在常量静态函数。原因有四：

1. 静态函数隶属于类，可以不实例化而直接通过类名访问
2. 常量/非常量函数的访问权限需要通过实例化后的对象是否为常量对象来决定
3. 常量修饰函数必须绑定在对象上（通过 this 指针）
4. 静态函数没有 this 指针 → 静态函数和常量函数互相冲突

```

class Foo{
    static const int a=1; // 可以就地初始化
    static const int b; // 可以在类外初始化
    static const char* cs; // 不可以就地初始化
};

```

哦哦哦，给你看哥哥的大表格！

	静态数据成员	常量数据成员	常量静态数据成员 (除 int enum 外)	常量静态数据成员 (int, enum)
初始化				
就地初始化		✓		✓
初始化列表初始化		✓		
构造函数体内初始化				
类外初始化	✓		✓	✓
访问				
普通成员函数	✓	✓	✓	✓
静态成员函数	✓		✓	✓
常量成员函数	✓	✓	✓	✓
修改				
普通成员函数	✓			
静态成员函数	✓			
常量成员函数	✓			

再谈常量对象和静态对象：构造和析构

- 常量对象的构造和析构时机和普通对象一样。
 - 在 main() 函数开始执行前，全局对象被构造。

- 在 main() 函数执行结束后，全局对象被析构。
- 在执行到局部对象的代码前，局部对象被构造。
- 在局部对象作用域执行结束后，局部对象被析构。
- 静态全局对象的构造和析构时机和普通全局对象一样。
- 函数中的静态对象在函数调用时构造，**离开作用域不析构**。第二次调用函数时，静态对象不会被重新构造，直接使用上一次构造的结果。在 main() 函数结束后，静态对象被析构。
- 类静态对象：类 A 的对象 a 作为类 B 的静态变量。
 - 在 main() 函数开始执行前初始化。
 - 在 main() 函数结束后析构。
 - 和 B 的构造和析构时机无关（和 B 是否实例化无关！先于 B 的任何实例存在）。

综合示例：各种对象的构造析构顺序

```
class A {
    const char* s;
public:
    A(const char* str) : s(str) { cout << s << " A constructing" << endl; }
    ~A() { cout << s << " A destructing" << endl; }
};

const A c_a("const c_a");           // ① 全局常量对象
static A s_a("static s_a");         // ② 全局静态对象

class B {
public:
    static A a1;                     // 静态成员声明
    const A a2{"const B::a2"};
    B(const char* s) : a2(s) {}
};
A B::a1("static B::a1");            // ③ 静态成员定义

void fun(int i, int n) {
    if (i >= n)
        static A static_obj("static"); // ④ 函数内静态对象（条件内！）
}

int main() {
    cout << "main starts" << endl;
    static B main_b("static main_b"); // ⑤ main内静态对象
    for (int i = 0; i < 3; i++)
        fun(i, 2);
    cout << "main ends" << endl;
    return 0;
}
```

输出顺序（构造↑/析构↓）：

const c_a A constructing	← ① 全局const最先
static s_a A constructing	← ② 全局static
static B::a1 A constructing	← ③ 类静态成员（在main之前！）
main starts	
const B::a2 A constructing	← B的const成员
static main_b A constructing	← ⑤ main内static
static A constructing	← ④ 第一次fun(2,2)时构造
main ends	
static A destructing	← ④ 最后构造的最先析构

```

const B::a2 A destructing
static main_b A destructing      ← ⑤
static B::a1 A destructing       ← ③
static s_a A destructing         ← ②
const c_a A destructing          ← ① 最先构造的最后析构

```

关键观察：

1. 全局对象在 main() 之前构造，在 main() 结束后析构
2. 函数内静态对象只在第一次执行到时构造，只构造一次
3. 析构顺序严格与构造顺序相反

伍 参数对象的构造和析构

幻灯片 L6 涵盖了从拷贝构造到移动构造、类型转换的完整内容，以下为全面补充版本。

○、拷贝控制与位拷贝的真相

位拷贝原本是 C 中的概念。在 C++ 中，只有基础类型（int、double 等）才会进行位拷贝；对于自定义类，编译器会**递归调用所有数据成员的拷贝构造函数或拷贝赋值运算符**。但一些教材中仍然把这种行为称为“位拷贝”，以区别用户自定义的拷贝方法。

理解要点：编译器隐式定义的拷贝构造不是简单的 memcpy！如果类中有子对象（如 string、vector），会递归调用子对象的拷贝构造。只有基础类型（int、char* 等指针）才是真正的位拷贝——这就是为什么指针成员会出现浅拷贝问题。

参数传递：构造与析构顺序图解

当函数按值传递和返回对象时，完整的执行顺序如下（以 MyClass func(MyClass c) 为例）：

```

MyClass func(MyClass c) { // ① 拷贝构造函数（以类的对象为形参）
    MyClass tmp;          // ② 默认构造函数
    return tmp;           // ③ 拷贝构造函数（返回类对象）
}                          // ④ tmp 的析构函数
                          // ⑤ c 的析构函数

```

浅拷贝问题：最简示例

在讨论 Pointer 类的复杂例子之前，先看一个最简版本：

```

class Test {
    int data;
    char* buffer;
public:
    Test() { buffer = new char[100]; }
    ~Test() { delete[] buffer; }
};
// Test a; Test b = a;
// 位拷贝使得 a.buffer 和 b.buffer 指向同一地址！
// 析构时同一块内存被释放两次 → 程序崩溃！

```

拷贝构造函数的三种调用场景

场景	代码示例	说明
1. 用对象初始化新对象	Test b(a); 或 Test c = a;	a 已存在，b 是新对象
2. 函数传参（值传递）	func(Test a) 调用时	实参拷贝给形参
3. 函数返回对象	return obj;	局部对象拷贝给返回值

关键区分： Test a = b; 是**拷贝构造**（a 是新对象），而 a = b; 是**拷贝赋值**（a 已存在）！

避免拷贝构造的方法：代码示例

```
// 方法1：将拷贝构造函数声明为 private
class MyClass {
    MyClass(const MyClass&) {} // private!
public:
    MyClass() = default;
};

// 方法2：用 =delete 显式删除（C++11，推荐！）
class MyClass {
public:
    MyClass() = default;
    MyClass(const MyClass&) = delete;
};
```

○○、左值与右值深入

判断标准：

- **左值：**可以取地址、有名字的值。如变量 a。
- **右值：**不能取地址、没有名字的值。如常量 1、函数返回值、表达式 a+b。

编译器错误示例

当只定义了左值引用版本的函数，却传入右值时：

```
void ref(int &x) { cout << "left " << x << endl; }
// ref(2); // 编译错误！
// [Error] invalid initialization of non-const reference of type 'int&'
//           from an rvalue of type 'int'
```

引用的绑定优先级

引用的绑定存在优先级！例如常量左值引用和右值引用均能绑定右值，当传入实参类型为右值时，**优先匹配形参类型为右值引用的函数**。这就是为什么移动构造/赋值在传入右值时会被优先调用。

拷贝构造 vs 移动构造：直观图解

```
拷贝构造： 临时对象 → 重新开辟内存 → 新对象（数据拷贝，两倍内存）
移动构造： 临时对象 → 直接“偷走”指针 → 新对象（指针转移，零拷贝）
                                     原对象指针 → nullptr
```

swap 的两种实现对比

```
// 传统实现：3次拷贝（6次内存操作）
template <class T>
void swap(T& a, T& b) {
    T tmp(a); // 拷贝构造：开辟内存 + 拷贝
    a = b;     // 拷贝赋值：拷贝
    b = tmp;   // 拷贝赋值：拷贝
}

// 移动实现：3次移动（仅指针交换！）
template <class T>
void swap(T& a, T& b) {
    T tmp(std::move(a)); // 移动构造：只转移指针
    a = std::move(b);    // 移动赋值：只转移指针
    b = std::move(tmp);  // 移动赋值：只转移指针
}
```

○○○、编译器自动合成的函数（重要补充）

关键规则：如果你定义了什么构造函数，编译器就不会自动生成默认构造函数了！但拷贝/移动构造函数和赋值运算符仍会自动生成（除非显式删除）。

自动合成的函数	条件
默认构造函数	用户未定义任何构造函数时自动合成
拷贝构造函数	用户未显式定义时自动合成（即使定义了其他构造）
移动构造函数	用户未显式定义且未定义拷贝/析构/赋值时自动合成
拷贝赋值运算符	用户未显式定义时自动合成
移动赋值运算符	用户未显式定义且未定义拷贝/析构/移动构造时自动合成
析构函数	用户未显式定义时自动合成

○○○○、参数传递：常量引用与最小特权原则

最小特权原则：给足够完成任务的权限，但是不要多余的修改、删除权限。

```
void add(int& a, int& b)           // 给了写权限（危险！）
void add(const int& a, const int& b) // 只给读权限（安全！推荐！）
```

如果传递的是形参

```
class A{
public:
    const char* s;
    A(const char* str) : s(str) {
        cout << s << "A constructing" << endl;
    }
    ~A() {
        cout << s << "A destructing" << endl;
    }
};
```

如果传递的是引用

```
class A{
public:
    const char* s;
    A(const char* &str) : s(str) {
        cout << s << " A constructing" << endl;
    }
    ~A() {
        cout << s << " A destructing" << endl;
    }
};

void func(A a) {
    cout << a.s << "func" << endl;
}

int main() {
    A a("a");
    func(a);
    return 0;
}
```

输出：（构造一次，析构两次？！）

其实是 b 被拷贝构造函数进行了初始化。函数结束，b 被析构。

```
a A constructing
...
a A destructing
a A destructing
```

如果传递传递指针，你很有可能对同一块空间进行了两次释放，从而报错！

所以尽可能使用引用传递，避免拷贝构造函数和析构函数的调用，还可以节省时间开销。

语句	解析	调用次数
A *p = new A;	堆上创建单个对象	1
A p2[10];	对象数组（10 个元素）	10
A p3;	栈上普通对象	1
A *p4[10];	指针数组（不创建对象）	0
总计		12

- **类内初始化**：C++11 允许声明时直接赋值。
- **静态成员函数**：没有 this 指针，不能访问非静态成员。
- **静态数据成员**：必须类外定义，严禁在 .h 中直接定义（防重复定义）。
- **常量静态整型**：static const int 允许类内初始化。

核心警告：new[] 必须配对 delete[]。

new[] 的内存布局：new A[3] 实际会分配 $3 \times \text{sizeof}(A) + 4$ 字节：

地址：	pA-4	pA	pA+sizeof(A)	pA+2*sizeof(A)
内容：	[count=3]	[A0	][A1	][A2]

前 4 字节存储数组元素个数，pA 指向第一个元素（不是真正的起始地址！）

若误用 delete pA:

1. 仅调用第一个元素的析构函数，导致其余元素内存泄漏。
2. 释放地址错误（应从 pA-4 处释放），直接导致程序崩溃。

delete[] pA 会读取 pA - 4 处的 count，对每个元素调用析构，然后从正确地址释放。

一、参数中的常量和常量引用

最小特权原则：给足够完成任务的权限，但是不要多余修改、删除权限。

这个时候终于出现了我们喜闻乐见众所周知的：**常量引用**！

在这个例子里面，我们通过常量引用，将 const 修饰的变量传递给函数，这样函数内部就不能修改这个变量了。

```
void add(const int& a, const int& b)
```

二、拷贝构造函数

拷贝构造函数是 C++ 的一种特殊的构造函数，它用于创建一个新对象，该对象是已有对象的副本。

拷贝构造函数的格式如下：

```
ClassName(const ClassName& obj){id=obj.id;...}
```

其中，ClassName 是类的名称，obj 是一个已有的对象，用于初始化新对象。**用参数对象初始化当前对象！**

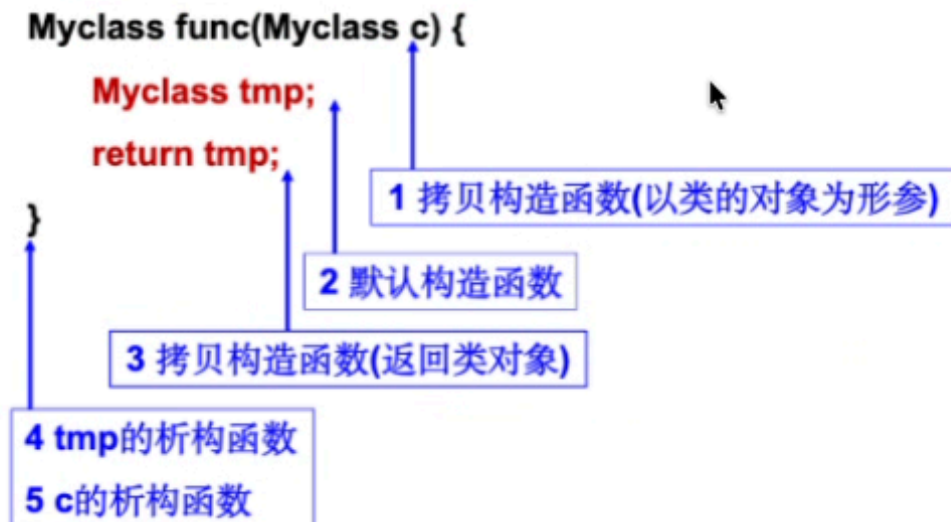
拷贝构造函数被调用的三种常见情况：

1. 用一个类对象定义另一个新的类对象 Test a; Test b(a); Test c=a;
2. 函数调用时以类的对象为形参 Func(Test a)
3. 函数返回类对象 Test Func(void)

编译器会自动调用”拷贝构造函数”，在已有对象基础上生成新对象。

拷贝构造函数：执行顺序

■ 以下述的func函数为例，调用该函数时，函数中各类构造函数和析构函数的执行顺序如下：



11

拷贝构造函数：实例 1

```

#include <iostream>
using namespace std;
class Test {
public:
    Test() { //构造函数
        cout << "Test()" << endl;
    }
    Test(const Test& src) { //拷贝构造
        cout << "Test(const Test&)" << endl;
    }
    ~Test() { //析构函数
        cout << "~Test()" << endl;
    }
};

Test copyObj(Test obj) {
    cout << "func()..." << endl;
    return Test();
}

int main() {
    cout << "main()..." << endl;
    Test t;
    t = copyObj(t);
}

```

```
    return 0;
}
```

输出:

```
main()...
Test() //main函数内初始化 Test
Test(const Test&)//func参数 拷贝构造
func()...
Test() //初始化 Test类的对象
Test(const Test&) //返回时拷贝构造
~Test()
~Test()
~Test()
~Test()
```

注意: 采用编译选项禁止编译器进行返回值优化: `g++ test.cpp --std=c++11 -fno-elide-constructors -o test`

拷贝构造函数: 实例 2

```
#include <iostream>
#include <cstring>
using namespace std;
class Pointer {
    int *m_arr;
    int m_size;
public:
    Pointer(int i):m_size(i) { //构造
        m_arr = new int[m_size];
        memset(m_arr, 0, m_size*sizeof(int));
    }
    ~Pointer(){delete []m_arr;} //析构
    void set(int index, int value) {
        m_arr[index] = value;
    }
    void print();
};
void Pointer::print() {
    cout << "m_arr: ";
    for (int i = 0; i < m_size; ++ i) {
        cout << " " << m_arr[i];
    }
    cout << endl;
}
int main() {
    Pointer a(5);
    Pointer b = a; //调用默认的拷贝构造
    a.print();
    b.print();
    b.set(2, 3);
    b.print();
    a.print();
    return 0;
}
```

输出:

```
m_arr: 0, 0, 0, 0, 0 //a.print()
m_arr: 0, 0, 0, 0, 0 //b.print()
m_arr: 0, 0, 3, 0, 0 //b.print()
m_arr: 0, 0, 3, 0, 0 //a.print()
```

问题分析：位拷贝会使得对象 a, b 的指针成员 m_arr 指向同一个内存地址。当类内含指针类型的成员时，为避免指针被重复删除，不应使用隐式定义的拷贝构造函数。

问题？

造成程序效率显著下降。

尽可能避免使用拷贝构造函数，使用引用传递参数 !!!

解决方法：

- (1) 使用引用/常量引用传参数或返回对象；
- (2) 将拷贝构造函数声明为 private；
- (3) 用 delete 关键字让编译器不生成拷贝构造函数的隐式定义版本。

三、内联函数、左值引用与右值引用

左值与右值

首先区分什么是左值和右值：

- **左值：**可以取地址、有名字的值。例如变量 a。
- **右值：**不能取地址、没有名字的值。常见于常量、函数返回值、表达式结果。例如 1、a+b。

```
int a = 1;
int b = func();
int c = a + b;
// a, b, c 是左值
// 1, func()返回值, a+b的结果 是右值
```

左值可以取地址，并且可以被 & 引用（左值引用）：

```
int *d = &a; // ✓ 左值可以取地址
int &e = a; // ✓ 左值引用绑定左值
int *f = &(a+b); // ✗ 右值不能取地址
int &g = a + b; // ✗ 普通左值引用不能绑定右值
```

右值引用

虽然右值无法取地址，但可以被 && 引用（右值引用）：

```
int &&e = a + b; // ✓ 右值引用绑定右值
int &&f = a; // ✗ 右值引用不能绑定左值
const int &g = 3; // ✓ 常量左值引用可以绑定右值
```

引用绑定规则

	非常量左值	常量左值	右值
非常量左值引用 &	✓		
常量左值引用 const &	✓	✓	✓
右值引用 &&			✓

关键点：

1. 左值引用能绑定左值，右值引用能绑定右值

2. **例外**：常量左值引用也能绑定右值（为了兼容性设计）
3. 所有的引用（包括右值引用）本身都是左值

右值引用示例

```
void ref(int &x) {
    cout << "left " << x << endl;
}
void ref(int &&x) {
    cout << "right " << x << endl;
}
int main() {
    int a = 1;
    ref(a);    // 输出: left 1 (a是左值, 调用左值引用版本)
    ref(2);    // 输出: right 2 (2是右值, 调用右值引用版本)
    return 0;
}
```

一个有趣的例子:

```
void ref(int &x) { cout << "left " << x << endl; }
void ref(int &&x) {
    cout << "right " << x << endl;
    ref(x);    // 这里调用哪个?
}
int main() {
    ref(1);    // 1是右值
    return 0;
}
```

输出:

```
right 1
left 1
```

因为**所有的引用本身都是左值**！x 虽然是右值引用参数，但它本身是个有名字的变量，所以是左值。

右值引用的意义

右值引用的核心用途：**延续即将销毁变量的生命周期**。

当函数返回一个临时对象时，这个对象马上就要被销毁。如果使用拷贝构造函数，需要重新开辟内存并复制数据，效率低下。使用右值引用配合移动构造函数，可以直接“偷走”临时对象的资源。

简单理解：

- 左值 = 有名字、有地址的“常住居民”，可以长期使用
- 右值 = 没名字、即将消亡的“临时工”，资源可以被“偷走”
- 左值引用 & = 给“常住居民”起别名
- 右值引用 && = 在“临时工”消失前，把它的资源转移出来

四、移动构造函数

移动构造函数 vs 拷贝构造函数

```
// 拷贝构造函数
ClassName(const ClassName& VariableName);

// 移动构造函数
ClassName(ClassName&& VariableName);
```

核心区别：

- **拷贝构造**：重新开辟内存，复制数据
- **移动构造**：直接“偷走”临时对象的指针，不复制数据

	拷贝构造函数	移动构造函数
参数类型	const ClassName&	ClassName&&
内存操作	重新开辟并拷贝	直接复用指针
适用场景	对象需要长期保留	临时对象即将销毁

移动构造函数示例

```
class Test {
public:
    int *buf; // only for demo
    Test() {
        buf = new int[10]; // 申请一块内存
        cout << "Test(): this->buf @ " << hex << buf << endl;
    }
    ~Test() {
        cout << "~Test(): this->buf @ " << hex << buf << endl;
        if (buf) delete[] buf;
    }
    // 拷贝构造函数
    Test(const Test& t) : buf(new int[10]) {
        for(int i=0; i<10; i++)
            buf[i] = t.buf[i]; // 拷贝数据
        cout << "Test(const Test&) called. this->buf @ "
            << hex << buf << endl;
    }
    // 移动构造函数
    Test(Test&& t) : buf(t.buf) { // 直接复制地址，避免拷贝
        cout << "Test(Test&&) called. this->buf @ "
            << hex << buf << endl;
        t.buf = nullptr; // 将t.buf改为nullptr，使其不再指向原来内存区域
    }
};
```

移动构造函数实例

```
Test GetTemp() {
    Test tmp;
    cout << "GetTemp(): tmp.buf @ " << hex << tmp.buf << endl;
    return tmp;
}

void fun(Test t) {
    cout << "fun(Test t): t.buf @ " << hex << t.buf << endl;
}

int main() {
    Test a = GetTemp();
    cout << "main() : a.buf @ " << hex << a.buf << endl;
    fun(a);
    return 0;
}
```

情况一：开启返回值优化（默认）

编译指令：g++ test.cpp --std=c++11 -o test

```

Test(): this->buf @ 0x7fa908c04b90
GetTemp(): tmp.buf @ 0x7fa908c04b90
main() : a.buf @ 0x7fa908c04b90
Test(const Test&) called. this->buf @ 0x7fa908c04ba0
fun(Test t): t.buf @ 0x7fa908c04ba0
~Test(): this->buf @ 0x7fa908c04ba0
~Test(): this->buf @ 0x7fa908c04b90

```

注意：编译器进行了返回值优化（RVO），所以没有调用移动构造函数，也少调用了几次拷贝构造函数！

情况二：禁止返回值优化 + 有移动构造函数

编译指令：g++ test.cpp --std=c++11 -fno-elide-constructors -o test

```

Test(): this->buf @ 0x7f8951c04b90
GetTemp(): tmp.buf @ 0x7f8951c04b90
Test(Test&&) called. this->buf @ 0x7f8951c04b90
~Test(): this->buf @ 0x0 (tmp被移动后)
Test(Test&&) called. this->buf @ 0x7f8951c04b90 (a = GetTemp())
~Test(): this->buf @ 0x0 (GetTemp返回的临时对象)
main() : a.buf @ 0x7f8951c04b90
Test(const Test&) called. this->buf @ 0x7f8951c04ba0
fun(Test t): t.buf @ 0x7f8951c04ba0
~Test(): this->buf @ 0x7f8951c04ba0
~Test(): this->buf @ 0x7f8951c04b90

```

注意观察：

1. Test a = GetTemp() 调用了移动构造函数（因为 GetTemp() 返回的是右值）
2. fun(a) 调用了拷贝构造函数（因为 a 是左值）
3. 移动构造后，原对象的 buf 变成了 nullptr

情况三：禁止返回值优化 + 删除移动构造函数

编译指令：g++ test.cpp --std=c++11 -fno-elide-constructors -o test

```

Test(): this->buf @ 0x7fabf8c04b50
GetTemp(): tmp.buf @ 0x7fabf8c04b50
Test(const Test&) called. this->buf @ 0x7fabf8c04b60
~Test(): this->buf @ 0x7fabf8c04b50
Test(const Test&) called. this->buf @ 0x7fabf8c04b50
~Test(): this->buf @ 0x7fabf8c04b60
main() : a.buf @ 0x7fabf8c04b50
Test(const Test&) called. this->buf @ 0x7fabf8c04b60
fun(Test t): t.buf @ 0x7fabf8c04b60
~Test(): this->buf @ 0x7fabf8c04b60
~Test(): this->buf @ 0x7fabf8c04b50

```

没有移动构造函数时，全部使用拷贝构造函数。

什么时候调用移动构造函数？什么时候调用拷贝构造函数？

判断依据：引用的绑定规则

1. 拷贝构造函数的形参类型为 const ClassName&（常量左值引用），可以绑定：常量左值、左值、右值
2. 移动构造函数的形参类型为 ClassName&&（右值引用），可以绑定：右值
3. **优先级**：当传入右值时，优先匹配右值引用参数的函数

拷贝构造函数的调用时机

1. 用一个类对象/引用/常量引用初始化另一个新的类对象
 - Test b = a; (a 是左值)
 - Test b = ref_a; (ref_a 是引用)

2. 以类的对象为函数形参，传入实参为类的对象/引用/常量引用
 - `func(a)`; (`a` 是左值)
3. 函数返回类对象（类中未显式定义移动构造函数，不进行返回值优化）

移动构造函数的调用时机

1. 用一个类对象的右值初始化另一个新的类对象
 - `Test b = func(a)`; (`func` 返回临时对象，是右值)
 - `Test b = std::move(a)`; (`std::move` 把左值转成右值)
2. 以类的对象为函数形参，传入实参为类对象的右值
 - `func(Test())`; (临时对象是右值)
 - `func(std::move(a))`; (`std::move` 把 `a` 转成右值)
3. 函数返回类对象（类中显式定义移动构造函数，不进行返回值优化）
 - `return Test()`; 或 `return tmp`; 均调用移动构造

返回值优化 (RVO)

编译器默认会进行返回值优化，条件：

1. `return` 的值类型与函数签名的返回值类型相同
2. `return` 的是一个局部对象

开启 RVO 时，编译器会直接在调用者的栈帧上构造返回值，完全省去拷贝/移动构造函数的调用。

注意：如果你想观察移动构造函数的效果，需要用 `-fno-elide-constructors` 禁用 RVO。

`std::move` 函数

`std::move` 可以将左值转换为右值，从而触发移动构造函数：

```
Test a;
Test b = std::move(a); // 调用移动构造函数
```

`std::move` 本身不做任何操作，只是类型转换。实际的“移动”操作在移动构造函数中实现。

典型应用：高效 swap

```
template <class T>
void swap(T& a, T& b) {
    T tmp(std::move(a)); // 移动构造
    a = std::move(b);    // 移动赋值
    b = std::move(tmp);  // 移动赋值
}
```

避免了三次不必要的拷贝操作！

构造函数综合实例

写出以下代码的运行结果：

```
#include <iostream>
class Test {
public:
    Test() {
        printf("Test()\n");
    } // 默认构造函数
    ~Test() {
        printf("~Test()\n");
    } // 析构函数
    Test(const Test &con) {
        printf("Test(const Test &con)\n");
    } // 拷贝构造函数
    Test(Test &&con) {
```

```

        printf("Test(Test &&con)\n");
    } //移动构造函数
};
Test func(Test a) {
    return Test();
}
int main() {
    Test a;
    Test b = func(a);
    return 0;
}

```

编译指令: g++ test.cpp --std=c++11 -fno-elide-constructors

答案分析: 用 (1+) 和 (1-) 这样的形式来对应类的构造和析构。

```

Test()                //(1+) 执行 Test a;
Test(const Test &con) //(2+) Test b = func(a);
                      //    func(a)传参调用拷贝构造函数
Test()                //(3+) return Test();
                      //    Test() 对应的构造函数
Test(Test &&con)        //(4+) return Test();
                      //    为了传值调用的移动构造函数
~Test()               //(3-) return Test();
                      //    Test() 对应的析构函数
Test(Test &&con)        //(5+) Test b = func(a);
                      //    给 b 传值时调用的移动构造函数
~Test()               //(4-) Test b = func(a);
                      //    完成赋值后 func(a) 返回值对应的析构函数
~Test()               //(2-) Test b = func(a);
                      //    参数释放对应的析构函数
~Test()               //(5-) 析构 b
~Test()               //(1-) 析构 a

```

详细步骤解析:

步骤	代码	说明
1	Test a;	构造 a, 调用默认构造函数
2	func(a) 传参	拷贝构造函数, 将 a 拷贝给参数对象
3	return Test();	构造临时对象, 调用默认构造函数
4	return Test();	移动构造函数, 将临时对象移动给返回值
5	临时对象析构	步骤 3 创建的临时对象析构
6	Test b = ...	移动构造函数, 将返回值移动给 b
7	返回值析构	步骤 4 创建的返回值对象析构
8	参数对象析构	步骤 2 创建的参数对象析构
9	main 结束	析构 b, 然后析构 a

关键点:

1. 函数参数按值传递时, 会调用拷贝构造函数
2. 函数返回临时对象时, 会调用移动构造函数 (如果有定义)
3. 析构顺序与构造顺序相反 (栈的 LIFO 特性)

多选题

Test 类中显式声明了三类构造函数:

- 普通构造函数: `Test(int val)`
- 移动构造函数: `Test(Test&& t)`
- 拷贝构造函数: `Test(const Test& t)`

现给出一段测试代码，下列描述错误的是：

```
Test F(Test&& a){
    Test b = a;           // (2)
    const Test& c = b;    // (3)
    return c;             // (4)
}
int main(){
    Test A = F(1); // (1)
    return 0;
}
```

选项：

- A: (1)处将 1 传入 F 时会调用普通构造函数 `Test(int val)` 以构建临时对象。
- B: (2)处调用移动构造函数。
- C: (3)处调用拷贝构造函数。
- D: (4)处返回局部变量的引用，可能会为程序带来潜在的风险。

答案：BCD

(1) `Test A = F(1);` — 选项 A（正确描述）

F 的参数类型为 `Test&&`，传入整数字面量 1 时，编译器需要先隐式构造一个 `Test` 类型的临时对象，因此会调用 `Test(int val)` 普通构造函数，再将该临时对象绑定到右值引用参数 `a`。

✅ A 描述正确

(2) `Test b = a;` — 选项 B（错误描述）

关键点：a 的类型虽然是 `Test&&`，但具名的右值引用在函数体内是左值（named rvalue reference is lvalue）。因此 `Test b = a` 匹配 `Test(const Test& t)`，调用的是拷贝构造函数，而非移动构造函数。若想触发移动构造，应写：

```
Test b = std::move(a);
```

❌ B 描述错误：实际调用的是拷贝构造函数

(3) `const Test& c = b;` — 选项 C（错误描述）

此处仅将 c 绑定为 b 的 `const` 引用，不构造任何新对象，不调用任何构造函数。

❌ C 描述错误：没有调用任何构造函数

(4) `return c;` — 选项 D（错误描述）

函数返回类型是 `Test`（按值返回），而非 `Test&`。`return c` 会以 c 为源拷贝构造一个新的返回值对象，返回的是值副本，不是对局部变量的引用，不存在悬空引用风险。

❌ D 描述错误：返回的是值，不是引用，无悬空风险

十、拷贝赋值运算符与移动赋值运算符

拷贝赋值运算符

已定义的对象之间相互赋值，通过调用对象的“拷贝赋值运算符函数”来实现：

```
ClassName& operator= (const ClassName& right) {  
    if (this != &right) { // 避免自己赋值给自己  
        // 将right对象中的内容拷贝到当前对象中...  
    }  
    return *this;  
}
```

注意区分下面两种代码：

```
ClassName a;  
ClassName b;  
a = b;           // 赋值运算符: a 和 b 都已存在  
ClassName a = b; // 拷贝构造函数: a 还不存在
```

拷贝赋值运算符实例

```
class Test {  
public:  
    int *buf;  
    Test() : buf(new int[10]) {}  
    ~Test() { if (buf) delete[] buf; }  
  
    Test& operator= (const Test& right) {  
        if (this == &right)  
            cout << "same obj!\n";  
        else {  
            for(int i=0; i<10; i++)  
                buf[i] = right.buf[i]; // 拷贝数据  
            cout << "operator=(const Test&) called.\n";  
        }  
        return *this;  
    }  
};
```

重要规则：赋值重载函数必须是类的**非静态成员函数**（non-static member function），不能是友元函数。

移动赋值运算符

和移动构造函数原理类似，直接“偷走”资源的指针：

```
Test& operator= (Test&& right) {  
    if (this == &right)  
        cout << "same obj!\n";  
    else {  
        this->buf = right.buf; // 直接赋值地址  
        right.buf = nullptr;  // 置空，防止重复释放  
        cout << "operator=(Test&&) called.\n";  
    }  
    return *this;  
}
```

配合 `std::move` 实现高效 swap：

```
template <class T>
void swap(T& a, T& b) {
    T tmp(std::move(a)); // 第一行调用移动构造函数
    a = std::move(b);    // std::move 的结果为右值引用
    b = std::move(tmp);  // 后两行均调用移动赋值运算符
}
```

拷贝/移动赋值运算符的调用时机

判断依据：引用的绑定规则

1. 拷贝赋值运算符的形参类型为 `const ClassName&`（常量左值引用），可以绑定：常量左值、左值、右值
2. 移动赋值运算符的形参类型为 `ClassName&&`（右值引用），可以绑定：右值
3. **优先级**：当赋值运算符右侧为右值时，优先匹配右值引用参数的函数

场景	调用的运算符	原因
<code>a = b</code> (b 是左值)	拷贝赋值运算符	左值绑定 <code>const &</code>
<code>a = std::move(b)</code>	移动赋值运算符	右值绑定 <code>&&</code>
<code>a = func()</code> (返回临时对象)	移动赋值运算符	右值绑定 <code>&&</code>
<code>a = Test()</code>	移动赋值运算符	临时对象是右值

编译器自动合成的函数/运算符

类中特殊的成员函数/运算符，即使用户不显式定义，编译器也会根据需要自动合成：

类型	说明
默认构造函数	无参构造函数
拷贝构造函数	<code>const ClassName& 参数</code>
移动构造函数	<code>ClassName&& 参数</code> (C++11 起)
拷贝赋值运算符	<code>operator=(const ClassName&)</code>
移动赋值运算符	<code>operator=(ClassName&&)</code> (C++11 起)
析构函数	<code>~ClassName()</code>

注意：如果你定义了任何构造函数，编译器就不会自动生成默认构造函数了！但拷贝/移动构造函数和赋值运算符仍会自动生成（除非显式删除）。

十一、类型转换

类型转换概述

当编译器发现表达式和函数调用所需的数据类型和实际类型不同时，便会进行自动类型转换。

```
void print(int d) { }
int main() {
    print(3.5); // double -> int, 自动类型转换
    print('c'); // char -> int, 自动类型转换
    return 0;
}
```

自动类型转换可通过两种方式实现：

1. 定义**类型转换运算符**（在源类中）
2. 定义**类型转换构造函数**（在目标类中）

方法一：类型转换运算符

在**源类**中定义“目标类型转换运算符”：

```

#include <iostream>
using namespace std;
class Dst { // 目标类 Destination
public:
    Dst() { cout << "Dst::Dst()" << endl; }
};
class Src { // 源类 Source
public:
    Src() { cout << "Src::Src()" << endl; }
    operator Dst() const {
        cout << "Src::operator Dst() called" << endl;
        return Dst();
    }
};

```

语法说明:

1. operator 目标类型() 是特殊的成员函数
2. **不需要指定返回类型**, 因为 operator Dst() 已经指明了返回 Dst
3. 必须是成员函数, 不能是友元

方法二: 类型转换构造函数

在**目标类**中定义“源类对象作参数的构造函数”:

```

#include <iostream>
using namespace std;
class Src; // 前置类型声明, 因为在 Dst 中要用到 Src 类
class Dst {
public:
    Dst() { cout << "Dst::Dst()" << endl; }
    Dst(const Src& s) { // 源类对象作参数的构造函数
        cout << "Dst::Dst(const Src&)" << endl;
    }
};
class Src {
public:
    Src() { cout << "Src::Src()" << endl; }
};

```

两种方法的使用

两种方法任选一种即可:

```

void Transform(Dst d) { }
int main() {
    Src s;
    Dst d1(s); // 显式初始化, 两种方法都支持
    Dst d2 = s; // 隐式转换, 两种方法都支持
    Transform(s); // 隐式转换, 两种方法都支持
    return 0;
}

```

警告: 两种自动类型转换的方法**不能同时使用**, 否则会产生二义性, 编译器不知道选择哪个。使用时任选其中一种!

类型转换运算符：错误示例

```
class SmallInt;
operator int(SmallInt&); // 错误：不是成员函数
class SmallInt {
public:
    int operator int() const; // 错误：不能指定返回类型
    operator int(int = 0) const; // 错误：参数列表不为空
    operator int*() const { return 42; } // 错误：42不是合法指针
};
```

错误分析：

1. 类型转换运算符必须是成员函数，不能是全局函数
2. 不能指定返回类型（operator int() 已经隐含返回 int）
3. 参数列表必须为空（转换运算符不接收额外参数）
4. 必须返回正确类型的值

正确写法：

```
class SmallInt {
public:
    operator int() const { return val; } // 正确
    operator int*() const { return &val; } // 正确
private:
    int val;
};
```

类型转换实例

```
class SmallInt {
public:
    // 构造函数：从 int 转换为 SmallInt
    SmallInt(int i=0): val(i) {
        cout << "SmallInt_Init" << endl;
    }
    // 转换运算符：从 SmallInt 转换为 int
    operator int() const {
        cout << "Int_Transform" << endl;
        return val;
    }
    void print() { cout << val << endl; }
private:
    size_t val;
};

int main() {
    SmallInt si;
    si = 4.10;
    si = si + 3;
    si.print();
    return 0;
}
```

输出：

```
SmallInt_Init // SmallInt si, 调用构造函数
SmallInt_Init // si = 4.10, double->int->SmallInt
Int_Transform // si + 3, SmallInt->int
```

```
SmallInt_Init    // si = si + 3, int->SmallInt
7               // si.print()
```

详细执行流程：

语句	转换过程	输出
SmallInt si;	默认构造函数, val=0	SmallInt_Init
si = 4.10;	4.10→4(double→int)→SmallInt(4)→赋值	SmallInt_Init
si + 3	si→int (调用 operator int()) → 7	Int_Transform
si = 7	7→SmallInt(7)→赋值	SmallInt_Init
si.print()	输出 val	7

思考题解答：为什么 si = si + 3 不是把 3 转换为 SmallInt 再加呢？

因为 operator+ 是内置的整数加法运算符，不是 SmallInt 的成员函数！

执行过程：

1. 编译器看到 si + 3，首先检查 SmallInt 是否有 operator+
2. 没有找到，于是尝试将 si 转换为能和 int 相加的类型
3. 发现 operator int()，将 si 转换为 int，得到 0 + 3 = 3（初始 val=0）
4. 然后赋值 si = 3，调用构造函数将 3 转换为 SmallInt

如果想让 3 转换为 SmallInt 再加，需要重载 operator+：

```
SmallInt operator+(const SmallInt& other) const {
    return SmallInt(val + other.val);
}
```

这样 si + 3 就会：3→SmallInt(3)→si.operator+(SmallInt(3))

禁止自动类型转换

如果用 explicit 修饰类型转换运算符或类型转换构造函数，则相应的类型转换必须显式进行：

```
// 方法一：explicit 修饰转换运算符
explicit operator Dst() const;

// 方法二：explicit 修饰构造函数
explicit Dst(const Src& s);
```

使用 explicit 后：

```
int main() {
    Src s;
    Dst d1(s);           // ✓ 可以执行，被认为是显式初始化
    // Dst d2 = s;       // ✗ 错误，隐式转换被禁止
    // Transform(s);     // ✗ 错误，隐式转换被禁止
    Dst d2 = static_cast<Dst>(s); // ✓ 显式转换
    Transform(static_cast<Dst>(s)); // ✓ 显式转换
    return 0;
}
```

强制类型转换

C++ 提供四种强制类型转换：

关键字	功能	应用场景
const_cast	去除 const 或 volatile 属性	修改常量指针的指向内容
static_cast	静态类型转换，类似 C 风格	基本类型转换、void 转其他指针
dynamic_cast	动态类型转换（运行时检查）	多态类型间的安全下行转换
reinterpret_cast	重新解释类型（不进行二进制转换）	指针与整数互转、不相关指针互转

详细说明：

1. const_cast: 唯一能去除常量性的转换

```
const int* p = &a;
int* q = const_cast<int*>(p); // 去除 const
```

2. static_cast: 最常用的转换，编译时检查

```
double d = 3.14;
int i = static_cast<int>(d); // double -> int
void* p = malloc(100);
int* q = static_cast<int*>(p); // void* -> int*
```

3. dynamic_cast: 安全的下行转换，需要多态支持（有虚函数）

```
Base* b = new Derived;
Derived* d = dynamic_cast<Derived*>(b); // 安全转换
// 如果转换失败，指针返回 nullptr，引用抛出异常
```

4. reinterpret_cast: 最危险的转换，直接重新解释内存

```
int* p = new int(42);
long addr = reinterpret_cast<long>(p); // 指针 -> 整数
int* q = reinterpret_cast<int*>(addr); // 整数 -> 指针
```

强制类型转换示例

```
int main() {
    Src s;
    Dst d1(s); // 显式初始化
    Dst d2 = static_cast<Dst>(s); // 显式转换
    Transform(static_cast<Dst>(s)); // 显式转换
    return 0;
}
```

陆 组合与继承 2026-04-07

幻灯片 L7 涵盖组合、继承、访问权限、重写隐藏、多重继承的完整内容。

○、本讲概览

本讲核心内容：组合 (has-a) → 继承 (is-a) → 访问权限 → 重写隐藏与重载 → 多重继承

一、对象(类)之间的关系

思考：这些是什么关系？

- 智能体：感知模块，思考模块，执行模块
- 智能体：科研智能体，编程智能体，写作智能体

1. has-a: 感知，思考，执行模块是智能体的组成部分（“整体-部分”）
2. is-a: 科研，编程，写作智能体是具有特殊能力的智能体（“一般-特殊”）

二、组合

has-a: 如果对象 a 是对象 b 的一个组成部分，则称 b 为 a 的整体对象，a 为 b 的部分对象。并把 b 和 a 之间的关系，称为“整体- 部分”关系（也可称为“组合”或“has-a”关系）。

程序设计反映对客观世界的认知习惯。

对象组合的两种实现方法：

1. 已有类的对象作为新类的**公有**数据成员，这样通过允许直接访问子对象而“提供”旧类接口
2. 已有类的对象作为新类的**私有**数据成员。新类可以调整旧类的对外接口，可以不使用旧类原有的接口（相当于对接口作了转换）

对象组合示例

```
#include <iostream>
#include <string>
using namespace std;
class Perception{
    string _prompt;
public:
    void set(const string& prompt){_prompt=prompt;}
};
class Action{
    string _prompt;
public:
    void set(const string& prompt){_prompt=prompt;}
};
class Agent{
private:
    Perception p;
public:
    Action a; // 公有成员，直接访问其接口
    void setPerception(const string& prompt){p.set(prompt);} // 提供私有成员的访问接口
};
int main(){
    Agent agent;
    string prompt = "Hello, World";
    agent.a.set(prompt);
    agent.setPerception(prompt);
    return 0;
}
```

子对象构造

子对象构造时若需要参数，则应在当前类的构造函数的初始化列表中进行。若使用默认构造函数来构造子对象，则不用做任何处理。

对象构造与析构函数的次序：

1. 先完成子对象构造，再完成当前对象构造
2. 子对象构造的次序仅由在类中声明的次序所决定
3. 析构函数的次序与构造函数相反

对象组合示例：构造与析构

```
#include <iostream>
using namespace std;
class S1 { //Single1类别
    int ID;
public:
    S1(int id) : ID(id) { cout << "S1(int)" << endl; }
    ~S1() { cout << "~S1()" << endl; }
```

```

};
class S2 { //Single2类别
public:
    S2() { cout << "S2()" << endl; }
    ~S2() { cout << "~S2()" << endl; }
};
class C3 { //Composite3类别
    int num;
    S1 sub_obj1; // 构造函数带参数
    S2 sub_obj2; // 构造函数不带参数
public:
    C3() : num(0), sub_obj1(123) { cout << "C3()" << endl; }
    C3(int n) : num(n), sub_obj1(123) { cout << "C3(int)" << endl; }
    C3(int n, int k) : num(n), sub_obj1(k) { cout << "C3(int, int)" << endl; }
    ~C3() { cout << "~C3()" << endl; }
};
int main(){
    C3 a, b(1), c(2), d(3, 4);
    return 0;
}

```

对象组合运行结果

```

S1(int)
S2()
C3()
S1(int)
S2()
C3(int)
S1(int)
S2()
C3(int)
S1(int)
S2()
C3(int, int)
~C3()
~S2()
~S1()
~C3()
~S2()
~S1()
~C3()
~S2()
~S1()
~C3()
~S2()
~S1()

```

晚构造，先析构！

隐式定义的拷贝构造与赋值运算

回忆：如果调用拷贝构造函数且没有给类显式定义拷贝构造函数，编译器将提供“隐式定义的拷贝构造函数”。该函数的功能为：

1. 递归调用所有子对象的拷贝构造函数
2. 对于基础类型，采用位拷贝
3. 赋值运算的默认操作类似

对象组合示例：拷贝与赋值

```
#include <iostream>
using namespace std;
class C1{
public:
    int i;
    C1(int n):i(n){}
    C1(const C1 &other) // 显式定义拷贝构造函数
    {i=other.i; cout << "C1(const C1 &other)" << endl;}
};
class C2{
public:
    int j;
    C2(int n):j(n){}
    C2& operator= (const C2& right){ // 显式定义赋值运算符
        if(this != &right){
            j = right.j;
            cout << "operator=(const C2&)" << endl;
        }
        return *this;
    }
};
```

```
class C3{
public:
    C1 c1;
    C2 c2;
    C3():c1(0), c2(0){}
    C3(int i, int j):c1(i), c2(j){}
    void print(){cout << "c1.i = " << c1.i << " c2.j = " << c2.j << endl;}
};
int main(){
    C3 a(1, 2);
    C3 b(a); // C1执行显式定义的拷贝构造, C2执行隐式定义的拷贝构造
    cout << "b: ";
    b.print();
    cout << endl;
    C3 c;
    cout << "c: ";
    c.print();
    c = a; // C1执行隐式定义的拷贝赋值, C2执行显式定义的拷贝赋值
    cout << "c: ";
    c.print();
    return 0;
}
```

运行结果:

```
C1(const C1 &other)
b: c1.i = 1 c2.j = 2
c: c1.i = 0 c2.j = 0
operator=(const C2&)
c: c1.i = 1 c2.j = 2
```

三、继承

is-a: “一般- 特殊”结构, 也称“分类结构”, 是由一组具有“一般- 特殊”关系的类所组成的结构。

1. 如果类 A 具有类 B 全部的属性和服务，而且具有自己特有的某些属性或服务，则称 A 为 B 的特殊类，B 为 A 的一般类。
2. 如果类 A 的全部对象都是类 B 的对象，而且类 B 中存在不属于类 A 的对象，则 A 是 B 的特殊类，B 为 A 的一般类。

C++ 使用继承来表达类间的“一般-特殊结构”。例如“编程智能体”继承“智能体”。

基本概念

1. 被继承的已有类，被称为**基类**（base class），也称“父类”。
2. 通过继承得到的新类，被称为**派生类**（derived class），也称“子类”、“扩展类”。

常见的继承方式：

1. `class Derived : public Base { ... };` — 公有继承
2. `class Derived : [private] Base { .. };` — 私有继承（缺省）
3. `class Derived : protected Base { ... };` — 保护继承（很少使用）

什么不能被继承？

1. **构造函数**：创建派生类对象时，必须调用派生类的构造函数，派生类构造函数调用基类的构造函数，以创建派生对象的基类部分。C++11 新增了继承构造函数的机制（使用 `using`），但默认不继承。
2. **析构函数**：释放对象时，先调用派生类析构函数，再调用基类析构函数。
3. **赋值运算符**：编译器不会继承基类的赋值运算符（参数为基类）；但会自动合成隐式定义的赋值运算符（参数为派生类），其功能为调用基类的赋值运算符。
4. **友元函数**：不是类成员，不能被继承。

继承示例

```
#include <iostream>
using namespace std;
class Base{
public:
    int k = 0;
    void f(){cout << "Base::f()" << endl;}
    Base & operator= (const Base &right){
        if(this != &right){
            k = right.k;
            cout << "operator= (const Base &right)" << endl;
        }
        return *this;
    }
};
class Derive: public Base{};
int main(){
    Derive d, d2;
    cout << d.k << endl; //Base数据成员被继承
    d.f(); //Base::f()被继承
    Base e;
    //d = e; //编译错误，Base的赋值运算符不被继承
    d = d2; //调用隐式定义的赋值运算符
    return 0;
}
```

运行结果：

```
0
Base::f()
operator= (const Base &right)
```

派生类对象的构造与析构过程

基类中的数据成员，通过继承成为派生类对象的一部分，需要在构造派生类对象的过程中调用基类构造函数来正确初始化。

1. 若没有显式调用，则编译器会自动调用基类的默认构造函数。
2. 若想要显式调用，则只能在派生类构造函数的**初始化成员列表**中进行，既可以调用基类中不带参数的默认构造函数，也可以调用合适的带参数的其他构造函数。

先执行基类的构造函数来初始化继承来的数据，再执行派生类的构造函数。

对象析构时，**先执行派生类析构函数，再执行由编译器自动调用的基类的析构函数。**

调用基类构造函数：隐式调用默认构造

```
class Base
{
    int data;
public:
    Base() : data(0) { cout << "Base::Base(" << data << ")\n"; }
    Base(int i) : data(i) { cout << "Base::Base(" << data << ")\n"; }
};
class Derive : public Base {
public:
    Derive() { cout << "Derive::Derive()" << endl; }
    // 无显式调用基类构造函数，则调用基类默认构造函数
};
int main() {
    Derive obj;
    return 0;
}
```

运行结果：

```
Base::Base(0)
Derive::Derive()
```

调用基类构造函数：显式调用

若想要显式调用，则只能在派生类构造函数的初始化成员列表中进行。

```
class Base
{
    int data;
public:
    Base() : data(0) { cout << "Base::Base(" << data << ")\n"; }
    Base(int i) : data(i) { cout << "Base::Base(" << data << ")\n"; }
};
class Derive : public Base {
public:
    Derive(int i) : Base(i) { cout << "Derive::Derive()" << endl; }
    // 显式调用基类构造函数
};
int main() {
    Derive obj(356);
    return 0;
}
```

运行结果：

```
Base::Base(356)
Derive::Derive()
```

继承基类构造函数（using）

在派生类中使用 `using Base::Base;` 来继承基类构造函数，相当于给派生类“定义”了相应参数的构造函数。

```
class Base
{
    int data;
public:
    Base(int i) : data(i) { cout << "Base::Base(" << i << ")\n"; }
};
class Derive : public Base {
public:
    using Base::Base; //相当于 Derive(int i):Base(i){};
};
int main() {
    Derive obj(356);
    return 0;
}
```

运行结果：

```
Base::Base(356)
```

继承基类构造函数（多个构造函数）

当基类存在多个构造函数时，使用 `using` 会给派生类自动构造多个相应的构造函数。

```
class Base
{
    int data;
public:
    Base(int i) : data(i) { cout << "Base::Base(" << i << ")\n"; }
    Base(int i, int j)
    { cout << "Base::Base(" << i << ", " << j << ")\n"; }
};
class Derive : public Base {
public:
    using Base::Base; //相当于 Derive(int i):Base(i){};
                        //加上 Derive(int i, int j):Base(i, j){};
};
int main() {
    Derive obj1(356);
    Derive obj2(356, 789);
    return 0;
}
```

运行结果：

```
Base::Base(356)
Base::Base(356,789)
```

继承基类构造函数（注意事项）

1. 如果基类的某个构造函数被声明为私有成员函数，则不能在派生类中声明继承该构造函数。
2. 如果派生类使用了继承构造函数，编译器就不会再为派生类生成隐式定义的默认构造函数。

如何选择继承方式?

public 继承

1. 基类中公有成员仍能在派生类中保持公有。原接口可沿用。最常用。
2. is-a: 基类对象能使用的地方, 派生类对象也能使用。

private 继承

1. is-implementing-in-terms-of (照此实现): 用基类接口实现派生类功能。移除了 is-a 关系。
2. 通常不使用, 用组合替代。可用于隐藏/公开基类的部分接口。公开方法: using 关键字。

成员访问权限

1. 基类中的**私有成员**, 不允许在派生类成员函数中访问, 也不允许派生类的对象访问它们。真正体现“基类私有”, 对派生类也不开放其权限!
2. 基类中的**公有成员**:
 - 允许在派生类成员函数中被访问
 - 若是使用 public 继承方式, 则成为派生类公有成员, 可以被派生类的对象访问
 - 若是使用 private/protected 继承方式, 则成为派生类私有/保护成员, 不能被派生类的对象访问。若想让某成员能被派生类的对象访问, 可在派生类 public 部分用关键字 using 声明它的名字
3. 基类中的**保护成员**:
 - 保护成员允许在派生类成员函数中被访问, 但不能被外部函数访问

公有继承: 基类公有成员的访问

```
#include <iostream>
using namespace std;
class Base {
public:
    void baseFunc() { cout << "in Base::baseFunc()..." << endl; }
};
class Derive1: public Base {}; // public继承
int main() {
    Derive1 obj1;
    cout << "calling obj1.baseFunc()..." << endl;
    obj1.baseFunc(); // 基类接口成为派生类接口的一部分, 派生类对象可调用
    return 0;
}
```

私有继承: 基类公有成员的访问

```
#include <iostream>
using namespace std;
class Base {
public:
    void baseFunc() { cout << "in Base::baseFunc()..." << endl; }
};
class Derive2: private Base {
// 私有继承, is-implementing-in-terms-of: 用基类接口实现派生类功能
public:
    void deriveFunc() {
        cout << "in Derive2::deriveFunc(), calling Base::baseFunc()..." << endl;
        baseFunc(); // 私有继承时, 基类接口在派生类成员函数中可以使用
    }
};
int main() {
    Derive2 obj2;
    cout << "calling obj2.deriveFunc()..." << endl;
    obj2.deriveFunc();
    //obj2.baseFunc(); ERROR: 基类接口不允许从派生类对象调用
}
```

```

    return 0;
}

```

私有继承：用 using 打开基类公有成员的访问权限

```

#include <iostream>
using namespace std;
class Base {
public:
    void baseFunc() { cout << "in Base::baseFunc()..." << endl; }
};
class Derive3: private Base {
public:
    using Base::baseFunc; // 私有继承时，在派生类public部分声明基类成员名字
};
int main() {
    Derive3 obj3;
    cout << "calling obj3.baseFunc()..." << endl;
    obj3.baseFunc(); // 基类接口在派生类public部分声明，则派生类对象可调用
    return 0;
}

```

私有继承：基类私有、保护成员访问

```

#include <iostream>
using namespace std;
class Base{
private:
    int a{0};
protected:
    int b{0};
};
class Derive : private Base{
public:
    void getA(){cout<<a<<endl;} // 编译错误，不可访问基类中私有成员
    void getB(){cout<<b<<endl;} // 可以访问基类中保护成员
};
int main()
{
    Derive d;
    d.getB();
    //cout<<d.b; // 编译错误，派生类对象不可访问基类中保护成员
    return 0;
}

```

```

#include <iostream>
using namespace std;
class Base {
private:
    int data{0};
public:
    int getData(){ return data;}
    void setData(int i){ data=i;}
};
class Derive1 : private Base {
public:
    using Base::getData;
}

```

```
};
int main() {
    Derivel d1;
    cout<<d1.getData();
    //d1.setData(10); // 隐藏了基类的setData函数, 不可访问
    //Base& b = d1; // 不允许私有继承的向上转换
    //b.setData(10); // 否则可以绕过D1, 调用基类的setData函数
    return 0;
}
```

基类成员访问权限与三种继承方式

1. **public 继承**: 基类的公有成员、保护成员、私有成员作为派生类的成员时, 都保持原有的状态。
2. **private 继承**: 基类的公有成员、保护成员、私有成员作为派生类的成员时, 都作为私有成员。
3. **protected 继承**: 基类的公有成员、保护成员作为派生类的成员时, 都成为保护成员, 基类的私有成员仍然是私有的。

继承权限表:

派生类成员函数能否访问基类成员			
	public 继承	private 继承	protected 继承
基类 public 成员	✓	✓	✓
基类 private 成员	×	×	×
基类 protected 成员	✓	✓	✓

基类成员在派生类中的成员类型 / 派生类对象能否访问			
	public 继承	private 继承	protected 继承
基类 public 成员	public / ✓	private / ×	protected / ×
基类 private 成员	private / ×	private / ×	private / ×
基类 protected 成员	protected / ×	private / ×	protected / ×

类似集合交运算: 成员类型与继承类型之间取交。Order: public > protected > private。

组合与继承的对比

优点: 支持增量开发。允许引入新代码而不影响已有代码正确性。

相似:

1. 实现代码重用
2. 将子对象引入新类
3. 使用构造函数的初始化成员列表初始化

不同:

1. 组合:
 - 嵌入一个对象以实现新类的功能
 - has-a 关系
2. 继承:
 - 沿用已存在的类提供的接口
 - public 继承: is-a
 - private 继承: is-implementing-in-terms-of

组合示例: has-a

```
#include <iostream>
using namespace std;
class Perception{
public:
```

```

    void init(){ cout<<"Perception::init"<<endl; }
};
class Action{
public:
    void init(){ cout<<"Action::init"<<endl; }
    void stop(){}
};
class Agent{
public:
    Perception perception;
    Action action;
};
int main()
{
    Agent agent;
    agent.perception.init();
    agent.action.init();
    return 0;
}

```

运行结果:

```

Perception::init
Action::init

```

继承示例: is-a

```

#include <iostream>
using namespace std;
class Agent{
public:
    void init(){cout<<"Agent init"<<endl;}
    void act(){cout<<"Agent act"<<endl; }
};
class Coder: public Agent{
public:
    void act(){cout<<"Coder act"<<endl;}
};
int main()
{
    Coder coder;
    coder.init();
    coder.act();
    return 0;
}

```

运行结果:

```

Agent init
Coder act

```

四、重写隐藏与重载

重载 (overload)

1. 目的: 提供同名函数的不同实现, 属于静态多态。
2. 函数名必须相同, 函数参数必须不同, 作用域相同 (如位于同一个类中; 或同名全局函数)。

重写隐藏 (redefining)

1. 目的：在派生类中重新定义基类函数，实现派生类的特殊功能。
2. **屏蔽了基类的所有其它同名函数。**
3. 函数名必须相同，函数参数可以不同。

重写隐藏发生时，基类中该成员函数的其他重载函数都将被屏蔽掉，不能提供给派生类对象使用。

可以在派生类中通过 `using 类名::成员函数名`；在派生类中“恢复”指定的基类成员函数（即去掉屏蔽），使之重新可用。

函数重写隐藏示例

```
#include <iostream>
using namespace std;
class T {};
class Base {
public:
    void f() { cout << "B::f()\n"; }
    void f(int i) { cout << "Base::f(" << i << ")\n"; } // 重载
    void f(double d) { cout << "Base::f(" << d << ")\n"; } // 重载
    void f(T) { cout << "Base::f(T)\n"; } // 重载
};
class Derive : public Base {
public:
    void f(int i) { cout << "Derive::f(" << i << ")\n"; } // 重写隐藏
};
int main() {
    Derive d;
    d.f(10);
    d.f(4.9); // 编译警告。执行自动类型转换。
    //d.f(); // 被屏蔽，编译错误
    //d.f(T()); // 被屏蔽，编译错误
    return 0;
}
```

运行结果：

```
Derive::f(10)
Derive::f(4)
```

4.9 → 4：自动类型转换，因为基类的 `f(double)` 被屏蔽了，只能匹配派生类的 `f(int)`。

恢复基类成员函数示例

使用 `using 基类名::函数名`；恢复基类函数：

```
#include <iostream>
using namespace std;
class T {};
class Base {
public:
    void f() { cout << "Base::f()\n"; }
    void f(int i) { cout << "Base::f(" << i << ")\n"; }
    void f(double d) { cout << "Base::f(" << d << ")\n"; }
    void f(T) { cout << "Base::f(T)\n"; }
};
class Derive : public Base {
public:
    using Base::f;
    void f(int i) { cout << "Derive::f(" << i << ")\n"; }
}
```

```
};
int main() {
    Derive d;
    d.f(10);
    d.f(4.9);
    d.f();
    d.f(T());
    return 0;
}
```

运行结果:

```
Derive::f(10)
Base::f(4.9)
Base::f()
Base::f(T)
```

using 关键字总结

using 关键字可用于:

1. 继承基类构造函数: using Base::Base;
2. 恢复被屏蔽的基类成员函数: using Base::f;
3. 指示命名空间: using namespace std;
4. 将另一个命名空间的成员引入当前命名空间: using std::cout;
5. 定义类型别名: using a = int;

五、多重继承

派生类同时继承多个基类。

应用场景:

```
class File{};
class InputFile: public File{};
class OutputFile: public File{};
class IOFile: public InputFile, public OutputFile{};
```

多重继承的问题

1. **数据存儲**: 如果派生类 D 继承的两个基类 A, B 是同一基类 Base 的不同继承, 则 A, B 中继承自 Base 的数据成员会在 D 有**两份独立的副本**, 可能带来数据冗余。
2. **二义性**: 如果派生类 D 继承的两个基类 A, B 有同名成员 a, 则访问 D 中 a 时, 编译器无法判断要访问的哪一个基类成员。

多重继承示例

```
#include <iostream>
using namespace std;
class Base {
public:
    int a{0};
};
class MiddleA : public Base {
public:
    void addA() { cout << "a=" << ++a << endl; };
    void bar() { cout << "A::bar" << endl; };
};
class MiddleB : public Base {
public:
    void addB() { cout << "a=" << ++a << endl; };
```

```

    void bar() { cout << "B::bar" << endl; };
};
class Derive : public MiddleA, public MiddleB{
};

```

MiddleA 和 MiddleB 各有一份独立的 Base::a，Derive 继承了两份。

```

int main() {
    Derive d;
    d.addA(); // 输出 a=1
    d.addB(); // 仍然输出 a=1 (另一份副本)
    d.addB(); // 输出 a=2
    //cout << d.a; // 编译错误, MiddleA和MiddleB都有成员a
    cout << d.MiddleA::a << endl; // 输出A中的成员a的值 1
    //d.bar(); // 编译错误, MiddleA和MiddleB都有成员函数bar
    cout << d.MiddleB::a << endl; // 输出B中的成员a的值 2
    return 0;
}

```

解决二义性：用 类名::成员名 显式指定访问哪个基类的成员。

六、课后练习

一家工厂生产飞机、汽车和摩托车。一架飞机需要三个轮子和两个机翼；一辆汽车需要四个轮子；一辆摩托车需要两个轮子。

这些交通工具都具有一个 run 函数，其中汽车和摩托车调用时输出“I am running”，但是飞机调用时输出“I am running and flying”。

编写以下几个类：Plane, Motor, Car, Wing, Wheel, Vehicle（交通工具），设计合理的继承、组合关系以及使用函数的继承与重写实现 add_wing, add_wheel, finished 以及 run 函数。

测试代码：

```

#include <iostream>
#include "Car.h"
#include "Plane.h"
#include "Motor.h"
#include "Wing.h"
#include "Wheel.h"
int main() {
    int m;
    std::cin >> m;
    Plane planes = new Plane[100];
    Car cars = new Car[100];
    Motor motors = new Motor[100];
    int i_p = 0, i_c = 0, i_m = 0;
    for (int i = 0; i < m; ++i) {
        int op;
        std::cin >> op;
        if (op == 0) { // plane
            int part;
            std::cin >> part;
            if (part == 0) planes[i_p].add_wing(new Wing());
            else planes[i_p].add_wheel(new Wheel());
            if (planes[i_p].finished()) planes[i_p++].run();
        }
        else if (op == 1) { // car
            cars[i_c].add_wheel(new Wheel());
        }
    }
}

```

```

        if (cars[i_c].finished()) cars[i_c++].run();
    }
    else { // motor
        motors[i_m].add_wheel(new Wheel());
        if (motors[i_m].finished())motors[i_m++].run();
    }
}
return 0;
}

```

输入：

```

0 0
0 0
0 1
0 1
0 1
1
1
2
2
1
1

```

输出：

```

I am running and flying
I am running
I am running

```

柒 虚函数 2026-04-14

幻灯片 L8 涵盖向上类型转换、对象切片、虚函数表、重写覆盖、协变返回类型、OOP 核心思想等完整内容。

〇、OOP 三大核心思想

在深入虚函数之前，先回顾 OOP 的三大支柱：

1. **数据抽象**：通过纯虚函数、模板等技术，隐藏实现细节，只暴露接口
2. **继承**：is-a（公有继承）和 is-implementing-in-terms-of（私有继承）
3. **动态绑定**：通过虚函数和类型转换，在运行时根据实际类型决定调用哪个函数

一、向上类型转换

派生类对象/引用/指针转换成基类对象/引用/指针，称为**向上类型转换**。只对 public 继承有效，对 private、protected 继承无效。

向上类型转换可以由编译器自动完成，是一种隐式类型转换。凡是接受基类对象/引用/指针的地方（如函数参数），都可以使用派生类对象/引用/指针。

```

class Base {
public:
    void print() { cout << "Base::print()" << endl; }
};
class Derive : public Base {
public:
    void print() { cout << "Derive::print()" << endl; }
};

```

```

void fun(Base obj) { obj.print(); }
int main() {
    Derive d;
    d.print();    // Derive::print()
    fun(d);       // Base::print() — 早绑定！
    return 0;
}

```

二、对象切片

当派生类的**对象**（不是指针或引用）被转换为基类的对象时，派生类的对象被**切片**为对应基类的子对象。派生类新定义的数据和方法会丢失。

```

class Pet {
public: int att_i;
    Pet(int x=0): att_i(x) {};
};
class Dog: public Pet {
public: int att_j;
    Dog(int x=0, int y=0): Pet(x), att_j(y) {}
};
int main() {
    Pet p(1);
    Dog g(2,3);
    cout << p.att_i << endl;           // 1
    cout << g.att_i << " " << g.att_j << endl; // 2 3
    p = g; // 对象切片，只赋值基类数据
    cout << p.att_i << endl;           // 2
    //cout << p.att_j << endl;        // 编译错误，没有该参数
    return 0;
}

```

对象切片两种情况：函数传参（值传递）和赋值操作。都会丢失派生类新增的数据和方法。

对象切片：sizeof 证明

使用 #pragma pack(4) 和 sizeof 可以直观证明对象切片：

```

#pragma pack(4)
class Pet { public: int att_i;
    Pet(int x=0): att_i(x) {} };
class Dog: public Pet { public: int att_j;
    Dog(int x=0, int y=0): Pet(x), att_j(y) {} };
void getSize(Pet p) {
    cout << "Pet size:" << sizeof(p) << endl; // 4
}
int main() {
    Dog g;
    cout << "sizeof(Pet)=" << sizeof(Pet) << endl; // 4
    cout << "sizeof(Dog)=" << sizeof(Dog) << endl; // 8
    getSize(g); // 对象切片！p丢失了att_j → sizeof(p)=4
}

```

对象切片：方法也会丢失

不仅数据成员会丢失，重写的方法也会因对象切片而丢失！

```

class Pet { public:
    void name() { cout << "Pet::name()" << endl; }
}

```

```
};
class Dog: public Pet { public:
    void name() { cout << "Dog::name()" << endl; }
};
void getName(Pet p) { p.name(); } // 值传递 → 对象切片!
int main() {
    Dog g;
    g.name();           // Dog::name() 直接调用, 没问题
    getName(g);         // Pet::name() 对象切片, 调用基类版本!
    Pet p = g;
    p.name();           // Pet::name() 切片后丢失派生类方法
}
```

三、指针（引用）的向上转换

当派生类的指针（引用）被转换为基类指针（引用）时，**不会创建新的对象**，但只保留基类的接口。引用向上转换后修改基类存在的数据，会影响派生类。

```
Dog g(2,3);
Pet& p = g;           // 引用向上转换
cout << p.att_i << endl; // 2
p.att_i = 1;          // 修改基类存在的数据
cout << g.att_i << " " << g.att_j << endl; // 1 3, 影响派生类
```

但引用向上转换后，函数调用仍是**早绑定**：

```
class Instrument {
public:
    void play() { cout << "Instrument::play" << endl; }
};
class Wind : public Instrument {
public:
    void play() { cout << "Wind::play" << endl; }
};
void tune(Instrument& i) { i.play(); }
int main() {
    Wind flute;
    tune(flute); // Instrument::play — 早绑定!
    Instrument &inst = flute;
    inst.play(); // Instrument::play — 早绑定!
    return 0;
}
```

四、函数调用捆绑

把函数体与函数调用相联系称为**捆绑**(binding)。

1. **早捆绑**(early binding)：捆绑在程序运行之前（由编译器和连接器）完成。上面的程序中 tune 里的 i.play() 与 Instrument::play() 绑定就是早捆绑。
2. **晚捆绑**(late binding)：捆绑根据对象的实际类型，发生在程序运行时，又称动态捆绑或运行时捆绑。**晚捆绑只对类中的虚函数起作用，使用 virtual 关键字声明。**

五、虚函数和虚函数表

对于被派生类重新定义的成员函数，若它在基类中被声明为**虚函数**，则通过基类指针或引用调用该成员函数时，编译器将根据所指（或引用）对象的实际类型决定调用哪个函数。

```
class Instrument {
public:
```

```

    virtual void play() { cout << "Instrument::play" << endl; }
};
class Wind : public Instrument {
public:
    void play() { cout << "Wind::play" << endl; } // 重写覆盖
};
void tune(Instrument& ins) { ins.play(); }
int main() {
    Wind flute;
    tune(flute); // Wind::play — 晚绑定！
    return 0;
}

```

晚绑定只对指针和引用有效！ 值传递会产生对象切片，仍是早绑定：

```

void tune(Instrument ins) { ins.play(); } // 值传递，对象切片，早绑定

```

虚函数表 (VTABLE)

1. **虚函数表(VTABLE)**: 每个包含虚函数的类用于存储虚函数地址的表（唯一性，即使没有重写虚函数也有自己的表）。
2. **虚函数指针(VPTR)**: 每个包含虚函数的类对象中，编译器秘密放一个指针，指向这个类的 VTABLE。
3. 编译期间：建立 VTABLE，记录每个类或其基类中所有已声明的虚函数入口地址。
4. 运行期间：建立 VPTR，在构造函数中发生，指向相应的 VTABLE。

编译器在构造函数的开头秘密地插入能初始化 VPTR 的代码。 构造函数的初始化顺序为：

1. 基类初始化
2. 对象成员初始化（按声明顺序，与初始化列表顺序无关！）
3. 构造函数体

当通过基类指针做虚函数调用时，编译器静态地插入能取得这个 VPTR 并在 VTABLE 表中查找函数地址的代码，这样就能调用正确的函数并引起晚捆绑的发生。

```

class B{
    int i; float j;
public:
    virtual void fun1() { cout << "B::fun1()" << endl; }
    virtual void fun2() { cout << "B::fun2()" << endl; }
};
class D: public B{
public:
    double k;
    virtual void fun1() { cout << "D::fun1()" << endl; }
};
int main() {
    B b; D d;
    B *pB = &d;
    pB->fun1(); // D::fun1() — 通过VPTR查找VTABLE，晚绑定
    return 0;
}

```

VTABLE 内存布局：sizeof 验证

使用 #pragma pack(4) 保证对齐一致，通过 sizeof 验证 VPTR 的存在：

```

#pragma pack(4)
class NoVirtual { int a; public: void f1() const {} int f2() const {return 1;} };
class OneVirtual { int a; public: virtual void f1() const {} int f2() const {return 1;} };

```

```

class TwoVirtual { int a; public: virtual void f1() const {} virtual int f2() const {return 1;}};

int main() {
    cout << "int: " << sizeof(int) << endl;           // 4
    cout << "NoVirtual: " << sizeof(NoVirtual) << endl; // 4 (只有int)
    cout << "void* : " << sizeof(void*) << endl;       // 8 (64位)
    cout << "OneVirtual: " << sizeof(OneVirtual) << endl; // 12 = int(4) + VPTR(8)
    cout << "TwoVirtual: " << sizeof(TwoVirtual) << endl; // 12 = int(4) + VPTR(8)
    // 注意：多个虚函数共享同一个VTABLE，所以只有一个VPTR！
}

```

六、虚函数和构造函数、析构函数

虚函数与构造函数

1. 构造函数**不能也不必**是虚函数。不能：如果构造函数是虚函数，创建对象时需要先知道 VPTR，但 VPTR 在构造函数调用前未初始化。不必：构造函数调用时明确指定要创建对象的类型。
2. 在构造函数中调用虚函数，被调用的只是这个函数的**本地版本**（当前类的版本），即虚机制在构造函数中不工作。
3. 原因：基类的构造函数比派生类先执行，调用基类构造函数时派生类中的数据成员还没有初始化。

```

class Base {
public:
    virtual void foo(){cout<<"Base::foo"<<endl;}
    Base(){foo();} // 构造函数中调用虚函数
    void bar(){foo();}
};

class Derived : public Base {
public:
    int _num;
    void foo(){cout<<"Derived::foo"<<_num<<endl;}
    Derived(int j):Base(),_num(j){}
};

int main() {
    Derived d(0); // 输出 Base::foo (构造函数中虚机制不工作)
    Base &b = d;
    b.bar();      // 输出 Derived::foo0
    b.foo();      // 输出 Derived::foo0
    return 0;
}

```

虚函数与析构函数

1. 析构函数**能是虚的，且常常是虚的**。虚析构函数仍需定义函数体。
2. 若基类析构不是虚函数，则删除基类指针所指派生类对象时，编译器仅自动调用基类的析构函数，可能导致**内存泄漏**。
3. 在析构函数中调用虚函数，虚机制同样不工作。

重要原则：总是将基类的析构函数设置为虚析构函数！

```

class Base1 { public: ~Base1() { cout << "~Base1()\n"; } };
class Derived1 : public Base1 { public: ~Derived1() { cout << "~Derived1()\n"; } };
class Base2 { public: virtual ~Base2() { cout << "~Base2()\n"; } };
class Derived2 : public Base2 { public: ~Derived2() { cout << "~Derived2()\n"; } };

int main() {
    Base1* bp = new Derived1;
    delete bp; // ~Base1() — 只调用了基类析构！内存泄漏！
    Base2* b2p = new Derived2;
}

```

```

delete b2p; // ~Derived2() ~Base2() — 虚析构, 正确调用
return 0;
}

```

七、重写覆盖与重写隐藏

三者对比

重载、重写隐藏与重写覆盖			
	重载(overload)	重写隐藏(redefining)	重写覆盖(override)
作用域	相同（同一个类中，或均为全局函数）	不同（派生类和基类）	不同（派生类和基类）
函数名	相同	相同	相同
函数参数	不同	相同/不同	相同
返回值	不能仅返回值不同	无要求	相同或协变的
其他要求	—	若参数相同，则基类函数不能为虚函数	基类函数为虚函数

关键区别：重写覆盖要求基类的函数是虚函数且参数相同；重写隐藏是参数不同或基类函数不是虚函数。重写覆盖会使派生类虚函数表中基类的虚函数指针被派生类的虚函数指针覆盖；重写隐藏不会。

重写覆盖示例

```

class Base{
public:
    virtual void foo(){cout<<"Base::foo()"<<endl;}
    virtual void foo(int){cout<<"Base::foo(int)"<<endl;} // 重载
};
class Derived1 : public Base {
public:
    void foo(int){cout<<"Derived1::foo(int)"<<endl;} // 重写覆盖
};
class Derived2 : public Base {
public:
    void foo(float){cout<<"Derived2::foo(float)"<<endl;} // 参数写错, 是重写隐藏!
};
int main() {
    Derived1 d1; Derived2 d2;
    Base* p1 = &d1; Base* p2 = &d2;
    p1->foo(3); // Derived1::foo(int) — 重写覆盖
    p2->foo(3.0); // Base::foo(int) — 重写隐藏, 虚函数表中是基类的
    return 0;
}

```

八、override 和 final 关键字

override

override 关键字明确告诉编译器一个函数是对基类中虚函数的重写覆盖，编译器将对各项条件进行检查。如果没有 override 但满足条件，也能实现重写覆盖——它只是编译器的一个检查。

```

class Derived3 : public Base {
public:
    void foo(int) override {cout<<"Derived3::foo(int)"<<endl;}; // 正确
    //void foo(float) override {}; // 参数不同, 编译错误
    //void bar() override {}; // bar非虚函数, 编译错误
};

```

final

final 关键字：在虚函数声明中使用，确保函数不可被派生类重写；在类定义中使用，指定此类不可被继承。

```
class Base{ virtual void foo(){}; };
class A: public Base {
    void foo() final {}; // 重写覆盖，且是最终覆盖
    //void bar() final {}; // bar非虚函数，编译错误
};
class B final : public A{
    //void foo() override {}; // A::foo已是最终覆盖，编译错误
};
//class C : public B{}; // B不能被继承，编译错误
```

九、const 对重写覆盖的影响（课后探究）

使用 const 修饰成员函数，可能导致重写覆盖失效（变成重写隐藏）：

```
class Base1{
public:
    virtual void f() {cout << "Base1::f" << endl;}
};
class Derive1: public Base1{
public:
    void f() const {cout << "Derive1::f" << endl;} // 重写覆盖失效，其实是重写隐藏
    using Base1::f; // 恢复被隐藏的基类函数
};
int main(){
    Derive1 a; const Derive1 b;
    a.f(); // Base1::f（非常量对象优先匹配Base1::f）
    b.f(); // Derive1::f（常量对象调用Derive1::f）
    return 0;
}
```

十、虚函数的返回值（课后探究）

一般来说，派生类虚函数的返回类型应该和基类相同；或者，是**协变**(Covariant)的——基类和派生类的指针/引用是协变的。

```
class Instrument {
public:
    virtual Instrument& getObj() { return *this; }
};
class Wind : public Instrument {
public:
    virtual Wind& getObj() { return *this; } // Wind&和Instrument&协变
};
```

协变条件（严格）：

1. 都是指针（不能是多级指针）、都是左值引用或都是右值引用
2. 基类返回类型中被引用/指向的类是派生类返回类型中被引用/指向的类的**祖先类**
3. 基类返回类型相比派生类的返回类型同等或更加 cv-qualified（即基类 const 更多是可以的）

协变返回类型：综合示例

```
class A{};
class B : public A{};
class C{};
```

```

class D : public B, public C{}; // 菱形继承

class Base {
public:
    virtual Base* f1() { return nullptr; }
    virtual Base** f2() { return nullptr; }
    virtual Base& f3() { static Base b; return b; }
    virtual A& f4() { static A a; return a; }
};

class Derive : public Base {
public:
    Derive* f1() override { return nullptr; } // ✓ 指针协变
    // Derive** f2() override {} // ✗ 多级指针, 编译错误!
    Base** f2() override { return nullptr; } // 同类型
    Derive& f3() override { static Derive d; return d; } // ✓ 引用协变
    // Derive* f3() override {} // ✗ 指针vs引用, 不是协变
    D& f4() override { static D d; return d; } // ✓ A是D的祖先
};

```

思考： 去掉引用（改为值返回）是否还能协变？不能！值返回会产生对象切片，不满足协变条件。

十一、课后练习

根据以下代码实现 Animal、Bird、Fish 类：

```

void action(Animal* pAnimal) {
    pAnimal -> sing();
    pAnimal -> swim();
}

int main(){
    Animal *myBird = new Bird();
    Animal *myFish = new Fish();
    action(myBird);
    action(myFish);
    delete myBird;
    delete myFish;
    return 0;
}

```

参考输出：

```

bird is singing.
bird can't swim.
fish can't sing.
fish is swimming.
bird has gone.
fish has gone.

```

附录：利用返回值优化提高执行效率

返回值优化（RVO）条件：

1. 返回的值类型与函数签名的返回值类型相同
2. 返回的是一个局部对象的左值

```

Test fn1(){ Test tmp; return tmp; } // 满足RVO条件
Test&& fn2(){ Test tmp; return move(tmp); } // 不建议：d指向被析构的tmp
int main(){
    const Test& a = fn1(); // (4) 常量左值引用接收
}

```

```

Test&& b = fn1();           // (5) 右值引用接收
Test c = fn1();             // (6) 构造新对象接收
// Test&& d = fn2();        // (7) 运行错误
return 0;
}

```

捌 多态与模板 2026-04-21

幻灯片 L9 涵盖纯虚函数、向下类型转换、多态、函数/类模板、静多态 vs 动多态等完整内容。

〇、完整抽象类示例：谁还是抽象的？

```

class Base {
public:
    virtual void func() = 0; // 纯虚函数
};
class Derive1 : public Base {
    // 没有重写 func() → Derive1 仍是抽象类！
};
class Derive2 : public Base {
public:
    void func() override { cout << "Derive2::func" << endl; } // 重写了
};
int main() {
    // Base b;           // x 抽象类不能实例化
    // Derive1 d1;       // x 仍是抽象类！
    Derive2 d2;          // ✓ 可以实例化
    d2.func();           // Derive2::func
}

```

一、纯虚函数与抽象类

虚函数还可以进一步声明为**纯虚函数**：

```
virtual 返回类型 函数名(形式参数) = 0;
```

包含纯虚函数的类被称为**抽象类**。抽象类**不允许定义对象**，主要用途是为派生类规定共性“接口”。

```

class A {
public:
    virtual void f() = 0; // 可在类外定义函数体提供默认实现
};
A obj; // 编译错误！不准抽象类定义对象！

```

抽象类特点：

1. 不允许定义对象
2. 只能为派生类提供接口
3. 能避免对象切片：保证只有指针和引用能被向上类型转换

基类纯虚函数被派生类重写覆盖之前仍是纯虚函数。因此当继承一个抽象类时，除纯虚析构函数外，必须实现所有纯虚函数，否则继承出的类也是抽象类。

```

class Pet {
public:
    virtual void motion()=0;
};
void Pet::motion(){ cout << "Pet motion: " << endl; }

```

```

class Dog: public Pet {
public:
    void motion() override {Pet::motion(); cout << "dog run" << endl; }
};
class Bird: public Pet {
public:
    void motion() override {Pet::motion(); cout << "bird fly" << endl; }
};
int main() {
    Pet* p = new Dog;
    p->motion(); // Pet motion: dog run
    p = new Bird;
    p->motion(); // Pet motion: bird fly
    return 0;
}

```

二、纯虚析构函数

析构函数也可以是纯虚函数：

1. 纯虚析构函数**仍然需要函数体**
2. 目的：使基类成为抽象类，不能创建基类的对象

```

class Base { public: virtual ~Base()=0; };
Base::~~Base() {} // 必须有函数体
class Derive : public Base {};
int main() {
    //Base b; // 编译错误，基类是抽象类
    Derive d1; // 派生类不必实现纯虚析构函数
    return 0;
}

```

与一般纯虚函数的区别：对于纯虚析构函数，即便派生类中不显式实现，编译器也会自动合成默认析构函数。因此只要派生类覆盖了其他纯虚函数，该派生类就不是抽象类。

三、向下类型转换

基类指针/引用转换成派生类指针/引用，称为**向下类型转换**。

为什么要向下类型转换？当我们用基类指针表示各种派生类时，保留了共性但丢失了特性。使用向下类型转换可以表现特性。

dynamic_cast

安全的向下类型转换，使用虚函数表中的信息判断实际类型：

```

T2* pObj = dynamic_cast<T2*>(obj_p); // 运行时失败返回nullptr
T2& refObj = dynamic_cast<T2&>(obj_r); // 运行时失败抛出bad_cast异常

```

T1 必须是多态类型（声明或继承了至少一个虚函数的类）。

static_cast：不安全示例（含垃圾值输出）

```

class B { public: int i; };
class D : public B { public: int j; };
int main() {
    B b;
    D* pd1 = static_cast<D*>(&b); // 编译通过，但不安全！
    cout << "D::i=" << pd1->i << endl; // 正常（继承来的）
    cout << "D::j=" << pd1->j << endl; // 垃圾值！如 D::j=124455624
    D* pd2 = dynamic_cast<D*>(&b); // 运行时检查，返回nullptr
}

```

```

    if (pd2 == nullptr) cout << "FAILED" << endl; // FAILED
}
// 当基类指针实际指向派生类对象时，两种转换都安全：
D d;
B* pb = &d;
D* pd3 = static_cast<D*>(pb);    // 安全
D* pd4 = dynamic_cast<D*>(pb);   // 安全

```

注意：dynamic_cast 也能用于向上类型转换（但很少使用，因为向上转换支持隐式转换）。static_cast 也能对不同对象类型进行转换。

dynamic_cast 与 static_cast 对比

	dynamic_cast	static_cast
检查时机	运行时	编译时
安全性	安全（失败返回 nullptr/异常）	不安全（不保证指向正确类型）
性能	较慢（需要 RTTI）	较快
要求	类必须有虚函数	只需有继承关系

重要原则：

1. 指针或引用的向上转换总是安全的
2. 向下转换时用 dynamic_cast，安全检查
3. 避免对象之间的转换

向下类型转换示例

```

class Pet { public: virtual ~Pet() {} };
class Dog : public Pet { public: void run() { cout << "dog run" << endl; } };
class Bird : public Pet { public: void fly() { cout << "bird fly" << endl; } };
void action(Pet* p) {
    auto d = dynamic_cast<Dog*>(p);
    auto b = dynamic_cast<Bird*>(p);
    if (d) d->run();
    else if(b) b->fly();
}
int main() {
    Pet* p[2];
    p[0] = new Dog;
    p[1] = new Bird;
    for (int i = 0; i < 2; ++i) action(p[i]);
    return 0;
}

```

四、多重继承中的虚函数

Best Practice:

1. 最多继承一个非抽象类（is-a）
2. 可以继承多个抽象类（接口）

这样避免了多重继承的二义性，且一个对象可以实现多个接口。

```

class WhatCanSpeak {
public:
    virtual ~WhatCanSpeak() {}
    virtual void speak() = 0;
};
class WhatCanMotion {
public:

```

```

    virtual ~WhatCanMotion() {}
    virtual void motion() = 0;
};
class Human : public WhatCanSpeak, public WhatCanMotion {
    void speak() { cout << "say" << endl; }
    void motion() { cout << "walk" << endl; }
};
void doSpeak(WhatCanSpeak* obj) { obj->speak(); }
void doMotion(WhatCanMotion* obj) { obj->motion(); }
int main() {
    Human human;
    doSpeak(&human); // say
    doMotion(&human); // walk
    return 0;
}

```

五、多态 (Polymorphism)

按照基类的接口定义，调用指针或引用所指对象的接口函数，函数执行过程因对象实际所属派生类的不同而呈现不同的效果——这就是“多态”。

1. 利用基类指针/引用调用函数时：虚函数在**运行时**确定执行哪个版本；非虚函数在**编译时**绑定
2. 利用类的对象直接调用函数时：无论什么函数，均在**编译时**绑定

产生多态效果的条件：继承 && 虚函数 && (引用 或 指针)

多态使得 C++ 可以用一段相同的代码，在运行时完成不同的任务。好处：

1. 通过基类定好接口后，不必对每一个派生类特殊处理，大大提高程序的可复用性
2. 不同派生类对同一接口的实现不同，提高了程序可拓展性和可维护性

```

class Animal{
public:
    void action() { speak(); motion(); }
    virtual void speak() { cout << "Animal speak" << endl; }
    virtual void motion() { cout << "Animal motion" << endl; }
};
class Bird : public Animal {
public:
    void speak() { cout << "Bird singing" << endl; }
    void motion() { cout << "Bird flying" << endl; }
};
class Fish : public Animal {
public:
    void speak() { cout << "Fish cannot speak ..." << endl; }
    void motion() { cout << "Fish swimming" << endl; }
};
int main() {
    Animal *pBase1 = new Fish;
    Animal *pBase2 = new Bird;
    pBase1->action(); // Fish cannot speak ... / Fish swimming
    pBase2->action(); // Bird singing / Bird flying
    return 0;
}

```

Template Method 设计模式

在接口的一个方法中定义算法的骨架，将一些步骤的实现延迟到子类中，使得子类可以在不改变算法结构的情况下重新定义算法中的某些步骤。

```

class Base{
public:
    void action() { step1(); step2(); step3(); } // 算法骨架
    virtual void step1() { cout << "Base::step1" << endl; }
    virtual void step2() { cout << "Base::step2" << endl; }
    virtual void step3() { cout << "Base::step3" << endl; }
};
class Derived1 : public Base{
    void step1() { cout << "Derived1::step1" << endl; }
};
class Derived2 : public Base{
    void step2() { cout << "Derived2::step2" << endl; }
};
int main(){
    Base* ba[] = {new Base, new Derived1, new Derived2};
    for (int i = 0; i < 3; ++i) ba[i]->action();
    return 0;
}

```

六、函数模板和类模板

继承与组合提供了重用**对象代码**的方法，而 C++ 的模板特征提供了重用**源代码**的方法。

函数模板

将函数的参数类型也定义为一种特殊的“参数”：

```

template <typename T>
T sum(T a, T b) { return a + b; }
// typename 也可换为 class

```

编译器能自动推导出实际参数的类型（实例化）：

```

cout << sum(9, 3); // int
cout << sum(2.1, 5.7); // double
// cout << sum(9, 2.1); // 编译错误，类型不一致
cout << sum<int>(9, 2.1); // 手工指定类型

```

函数模板示例

```

template<class T>
void sort(T* data, int len) {
    for(int i = 0; i < len; i++){
        for(int j = i + 1; j < len; j++) {
            if(data[i] > data[j])
                std::swap(data[i], data[j]);
        }
    }
}

template<class T>
void output(T* data, int len) {
    for(int i = 0; i < len; i++)
        std::cout << data[i] << " ";
    std::cout << std::endl;
}

int main() {
    int arr_a[] = {3,2,4,1,5};
    sort(arr_a, 5); output(arr_a, 5);
    float arr_b[] = {3.2, 2.1, 4.3, 1.5, 5.7};
}

```

```

    sort(arr_b, 5); output(arr_b, 5);
    return 0;
}

```

模板也可以支持自定义类型，只要类型满足函数的要求（如定义了 `operator>`）：

```

class MyInt {
public:
    int data;
    MyInt(int val): data(val) {};
    bool operator>(const MyInt& b){ return data > b.data; }
    friend std::ostream& operator<<(std::ostream& out, const MyInt& obj){
        out << obj.data; return out;
    }
};

```

模板原理

对模板的处理是在**编译期**进行的。每当编译器发现对模板的一种参数的使用，就生成对应参数的一份代码。

为什么模板必须在头文件中实现？（编译链接流程详解）

```

// x.h:      template<typename T> T sum(T a, T b);
// x.cpp:    template<typename T> T sum(T a, T b) { return a + b; }
// main.cpp: #include "x.h" → sum(3, 5);

```

编译流程：

- ① `g++ -c main.cpp → main.o`
编译器看到 `sum(3,5)`，在 `x.h` 中找到了声明，但找不到定义！
→ 编译器假设其他编译单元提供了定义，不报错
- ② `g++ -c x.cpp → x.o`
编译器看到模板定义，但没有被使用 → 不生成任何代码！
- ③ `g++ main.o x.o -o main`
链接器在 `x.o` 中找不到 `sum<int>(int,int)` → `undefined reference!`

解决方案： 将模板的定义和声明都放在头文件中（或使用 `.hpp` 文件 `include` 到头文件末尾）。

模板参数必须在编译期确定

```

int n = 5;
// array<char, n> array0;      // x 编译错误！不能使用变量
const int m = 5;
array<char, m> array1;        // ✓ 可以使用常量
array<char, 5> array2;        // ✓ 直接使用字面量

```

函数模板编译错误实例

模板编译错误通常产生大量报错信息（几百行），需要学会阅读。例如缺少 `operator>`：

```

class MyInt {
    int data;
public:
    MyInt(int val): data(val) {}
    // 缺少 operator> 的定义！
};
// 当用 sort(arr, 5) 排序 MyInt 数组时：
// error: no match for 'operator>' (operand types are 'MyInt' and 'MyInt')
// ...几百行的模板展开错误...

```

经验： 遇到模板错误时，从错误信息最底部开始读，找到第一个提到用户代码的行，通常就是问题所在。

类模板

将类中的类型信息抽取出来用模板参数替换：

```
template <typename T> class A {
    T data;
public:
    A(T _data): data(_data) {}
    void print();
};
template<typename T>
void A<T>::print() { cout << data << endl; } // 类外定义
int main() {
    A<int> a(1);
    a.print();
    return 0;
}
```

类模板的模板参数：

1. **类型参数**：使用 typename 或 class 标记
2. **非类型参数**：整数、枚举、指针、引用。无符号整数比较常用

```
template<typename T, unsigned size>
class array { T elems[size]; };
array<char, 10> array0;
```

所有模板参数必须在编译期确定，不可以使用变量（可以使用常量或具体数值）。

类模板示例

```
template<class T, unsigned size>
class MyArr {
    T data[size];
public:
    void sort(){
        for(int i = 0; i < size; i++)
            for(int j = i + 1; j < size; j++)
                if(data[i] > data[j])
                    std::swap(data[i], data[j]);
    }
    void output(){
        for(int i = 0; i < size; i++)
            std::cout << data[i] << " ";
        std::cout << std::endl;
    }
    void input(){
        for(int i = 0; i < size; i++)
            std::cin >> data[i];
    }
};
int main() {
    MyArr<int, 5> arr_a;
    arr_a.input(); arr_a.sort(); arr_a.output();
    MyArr<float, 5> arr_b;
    arr_b.input(); arr_b.sort(); arr_b.output();
    return 0;
}
```

七、模板与多态

模板也是多态的一种体现，但模板的关联是在**编译期**处理，称为**静多态**。

	静多态（模板）	动多态（虚函数）
处理时机	编译期	运行时
特点	高效，省去函数调用；编译后代码增多	灵活方便；存在函数调用；侵入式，必须继承
关联方式	泛型标记+函数重载	继承+虚函数

八、成员函数模板（自学）

普通类的成员函数也可以定义为模板函数：

```
class normal_class {
public:
    int value;
    template<typename T> void set(T const& v) { value = int(v); }
    template<typename T> T get();
};
template<typename T>
T normal_class::get() { return T(value); }
```

模板类的成员函数也可以有额外的模板参数：

```
template<typename T0> class A {
    T0 value;
public:
    template<typename T1> void set(T1 const& v) { value = T0(v); }
    template<typename T1> T1 get();
};
template<typename T0> template<typename T1> // 注意：不能写成 template<typename T0, typename T1>
T1 A<T0>::get(){ return T1(value); }
int main() {
    A<int> a;
    a.set(5); // 自动推导
    double t = a.get<double>(); // 手动指定返回值类型
    return 0;
}
```

九、课后练习

练习 1

重载、重写、虚函数重写的情况下，编译器如何处理返回值不同的函数？

```
class Base {
public:
    virtual void f() {std::cout << "Call Base void f() " << std::endl;}
    virtual void g() {std::cout << "Call Base void g() " << std::endl;}
    void h() {std::cout << "Call Base void h()" << std::endl;}
};
class Derived : public Base {
public:
    void f() { std::cout << "Call Derive void f() " << std::endl; } // 重写覆盖
    int g() { std::cout << "Call Derive int g()" << std::endl; return 0; } // 编译错误！
    int h() { std::cout << "Call Base int h()" << std::endl; return 0; } // 重写隐藏
};
```

练习 2

仿照 C++ 的 vector 实现一个 Vector 类，要求使用模板以支持任意类型的元素，并且至少具有以下成员函数：

1. void push_back();
2. void pop_back();
3. int size();
4. operator[]();

玖 模板与 STL 初步 2026-04-28

一、命名空间

为了避免在大规模程序中使用各种 C++ 库时标识符命名冲突，C++ 引入了 namespace（命名空间）。

标准 C++ 库中所有内容（常量、变量、结构、类和函数等）都被定义在命名空间 std 中。

定义命名空间：

```
namespace A {  
    int x, y;  
}
```

使用命名空间：

```
A::x = 3;  
A::y = 6;
```

使用 using 声明简化：

```
using namespace A;    // 使用整个命名空间，所有成员直接可用  
x = 3; y = 6;  
  
using A::x;            // 使用部分成员  
x = 3; A::y = 6;
```

任何情况下，都不应出现命名冲突。

二、C++ 标准演进与 STL

C++ 标准版本概览

标准	主要新增特性
C++98	STL、模板、异常、命名空间等基础特性
C++11	移动语义、lambda、智能指针、auto/decltype、范围 for、tuple、array、unordered_map
C++14	make_unique、泛型 lambda、decltype(auto)
C++17	string_view、optional、variant、any、结构化绑定、if constexpr

推荐文档：<http://www.cplusplus.com/>（多写多查多用！）

三、STL 简介

标准模板库（Standard Template Library，STL）是高效的 C++ 软件库，包含 4 个组件：**算法、容器、函数、迭代器**。

关键理念：将“在数据上执行的操作”与“要执行操作的数据”分离。

1. STL 的命名空间是 std
2. 一般使用 std::name 来使用 STL 的函数或对象
3. 也可以使用 using namespace std（不推荐在大型工程中使用，容易污染命名空间）

三、STL 容器

容器是包含、放置数据的工具，通常为数据结构。包括：简单容器、序列容器、关系容器。

pair

最简单的容器，由两个单独数据组成：

```
template<class T1, class T2> struct pair {
    T1 first;
    T2 second;
};
```

使用：

```
std::pair<int, int> t;
t.first = 4; t.second = 5;
// 创建：使用 make_pair, 自动推导类型
auto t = std::make_pair("abc", 7.8);
```

pair 支持小于、等于等比较运算符：先比较 first，后比较 second。

```
std::make_pair(1, 4) < std::make_pair(2, 3); // true
std::make_pair(1, 4) > std::make_pair(1, 2); // true
```

tuple (C++11)

pair 的扩展，由若干成员组成的元组类型：

```
template<class ...Types> class tuple;
```

创建：make_tuple 和 tie 函数：

```
auto t = std::make_tuple("abc", 7.8, 123, '3');
// tie 返回左值引用的元组
std::string x; double y; int z;
std::tie(x, y, z) = std::make_tuple("abc", 7.8, 123);
```

通过 std::get 获取数据（下标需在编译时确定）：

```
auto v0 = std::get<0>(t);
int i = 0;
// v = std::get<i>(t); // 编译错误！下标必须编译时常量
```

tuple 常用于函数多返回值的传递：

```
std::tuple<int, double> f(int x) {
    return std::make_tuple(x, double(x) / 2);
}
int main() {
    int xval; double half_x;
    std::tie(xval, half_x) = f(7);
    return 0;
}
```

vector

会自动扩展容量的数组，STL 中最基本的序列容器：

```
template<class T, class Allocator = std::allocator<T>>
class vector;
```

常用操作：

```
std::vector<int> x;           // 创建
x.size();                   // 当前长度
x.clear();                  // 清空
x.push_back(1);             // 末尾添加（高速）
x.pop_back();               // 末尾删除（高速）
x.insert(x.begin()+1, 5);   // 中间添加（低速）
x.erase(x.begin()+1);       // 中间删除（低速）
```

vector 原理

除了 size，另保存 capacity（最大容量限制）。如果 size 达到了 capacity，则另申请一片 capacity*2 的空间，并整体迁移内容。

容量扩展图解：

```
原始: [1][2][3][1]   size=4, capacity=4
           begin()→      ←end()
push_back(2):
① 申请新空间: [ ][ ][ ][ ][ ][ ][ ][ ]   capacity=8
② 拷贝迁移:   [1][2][3][1][2][ ][ ][ ]
③ 释放旧空间
→ 所有迭代器失效！（因为指向的空间已被释放）
```

1. push_back 等修改 vector 大小的方法可能会使**所有迭代器失效**（因为整体迁移）
2. insert/erase 后，所修改位置之后的所有迭代器失效
3. 时间复杂度为均摊 $O(1)$

迭代器失效：指针追踪示例

```
vector<int> vec = {1, 2, 3, 4, 5};
auto first = vec.begin();           // 指向 1
auto second = vec.begin() + 1;     // 指向 2
auto third = vec.begin() + 2;      // 指向 3
auto ret = vec.erase(second);       // 删除 2
// first 仍指向 1（删除位置之前的迭代器仍有效）
// second 和 third 失效！（删除位置及之后）
// ret 指向 3（erase返回被删元素的下一个位置）
```

遍历时修改容器的陷阱

```
vector<int> vec = {1, 2, 3, 4, 5};
for (auto it = vec.begin(); it != vec.end(); ++it)
    vec.push_back(*it); // 危险！push_back可能导致所有迭代器失效
```

安全准则： 修改容器后，不使用之前的任何迭代器！迭代器是否会失效，与实现容器的数据结构有关，查阅文档中的“Iterator validity”部分。

四、迭代器

一种检查容器内元素并遍历元素的数据类型，为遍历不同的聚合结构提供统一的接口。使用上类似指针。

以 vector 为例：

```
vector<int>::iterator iter; // 定义迭代器
x.begin(); // 第一个元素的迭代器
x.end();   // 最后一个元素之后的位置的迭代器
// begin 和 end 构成左闭右开区间
```

迭代器操作：

1. ++iter/--iter: 下一个/上一个元素
2. iter += n/iter -= n: 移动 n 个元素
3. *iter: 解引用, 返回左值引用
4. iter1 - iter2: 元素位置差

遍历 vector:

```
// 迭代器遍历
for (auto it = vec.begin(); it != vec.end(); ++it)
    *it *= 2;
// C++11 按范围遍历 (等价)
for (auto& x : vec)
    x *= 2;
```

迭代器失效

当迭代器不再指向本应指向的元素时, 称此迭代器**失效**。

1. insert/erase 后, 所修改位置之后的所有迭代器失效
2. push_back 等修改 vector 大小的方法可能会使所有迭代器失效 (整体迁移)
3. 修改容器后, **不使用之前的迭代器** (绝对安全的准则)

五、STL 其他容器

list

链表容器 (底层实现是双向链表):

```
std::list<int> l;
l.push_front(1);    // 插入前端
l.push_back(2);     // 插入末端
std::find(l.begin(), l.end(), 2); // 查询, 返回迭代器
l.insert(it, 4);    // 在迭代器位置插入
```

1. 不支持下标等随机访问
2. 支持在任意位置高速插入/删除
3. 插入和删除操作不会导致迭代器失效 (除指向被删除元素的迭代器外)

set

不重复元素构成的集合 (内部按大小顺序排列):

```
std::set<int> s;
s.insert(val);           // 插入 (不允许重复)
s.find(val);             // 查询, 返回迭代器
s.erase(s.find(val));    // 删除
s.count(val);            // val 的个数, 总是0或1
```

注意: set 的“无序”是指不保持插入顺序, 内部按元素大小排列。

map

关联数组, 将一个数据项映射到另一个数据项。值类型为 pair<Key, T>, key 必须互不相同。

```
std::map<std::string, int> s;
s["Monday"] = 1;        // 下标访问 (key不存在则创建)
s.insert(std::make_pair(std::string("Tuesday"), 2));
s.find(key);            // 查询, 返回迭代器
s.count(key);           // 返回0或1
s.erase(s.find(key));   // 删除
```

map 常用作稀疏数组或以字符串为下标的数组:

```
std::map<std::string, std::string> M;
M["fp"] = "c";
M["oop"] = M["fp"] + "++"; // M["oop"] = "c++"
```

关联容器原理

set 和 map 底层数据结构都是红黑树（一种二叉平衡树），几乎所有操作复杂度均为 $O(\log n)$ 。

容器总结

	vector	list
底层结构	动态数组	双向链表
随机访问	支持（高速）	不支持
中间插入/删除	低速	高速
迭代器失效	操作位置之后全部失效	仅被删除元素失效

	set	map
底层结构	红黑树	红黑树
特点	不重复有序集合	键值映射
操作复杂度	$O(\log n)$	$O(\log n)$

选择容器的参考：

1. 频繁在中间插入/删除 → list，否则 → vector
2. 需要按值快速查找 → set、map
3. 希望迭代器尽量不失效 → list、关联容器

容器选择四大准则

1. 算法复杂度

：不同容器的插入、删除、查找复杂度不同

2. 元素排序要求

：是否需要保持顺序（序列容器）还是按大小访问（关联容器）

3. 查找速度

：是否需要常数值查找（unordered_map/unordered_set）

4. 迭代器/指针/引用失效保证

：如 vector 扩容时所有迭代器失效，list 几乎不失效

序列容器中的元素有顺序，可以按顺序访问。关联容器中的元素无顺序，可以按数值（大小）访问。vector 中插入删除操作会使操作位置之后全部的迭代器失效；其他容器中只有被删除元素的迭代器失效。

推荐阅读：《Effective STL》（容器选择章节）、《STL 源码剖析》、《C++编程思想》第 16 章。

六、string 字符串类

STL 提供的变长字符串类型，比 C 风格的 char 数组更方便：

```
string fullname = firstname + " " + lastname; // 简洁拼接
cout << fullname << endl;
```

构造方式

```
string s0("Initial string"); // 从C风格字符串构造
string s1; // 默认空字符串
string s2(s0, 8, 3); // 截取："str" (从index 8开始, 长度3)
string s3("Another character sequence", 12); // 截取："Another char"
```

```
string s4(10, 'x');           // 复制字符: "xxxxxxxxxx"
string s5(s0.begin(), s0.begin()+7); // 迭代器范围复制: "Initial"
```

转为 C 风格字符串: `str.c_str()` (返回 `const char*`)。

常用函数

与 `vector` 类似: `str[i]`、`str.size()`、`str.clear()`、`str.empty()`、`str.push_back('a')`、`str.append(s2)`。

不同之处:

1. 查询长度也可以用 `str.length()`, 与 `size()` 返回值相同
2. 向尾部增加也可以用 `str += 'a'` 或 `str += s2`

输入方式

```
cin >> firstname;           // 读到空格为止
getline(cin, fullname);      // 读一行
getline(cin, fullnames, '#'); // 读到指定分隔符为止 (可读入换行符)
```

拼接与比较

```
// 拼接
string fullname = firstname + " " + lastname;
// 注意: 拼接时间复杂度为生成字符串长度
// 多次拼接应使用 operator+= 或 stringstream
for (int i = 0; i < n; i++)
    allname = allname + name[i] + "\n"; // O(n^2 * L), 很慢!
allname += name[i] + "\n";             // 改用 += 更高效

// 比较: 按字典序
string a = "alice", b = "bob";
a == b; // false
a < b;  // true
```

数值转换

```
// 数值 → 字符串
to_string(1);           // "1"
to_string(3.14);        // "3.14"

// 字符串 → 数值
int a = stoi("2001");    // a = 2001
std::string::size_type sz;
int b = stoi("50 cats", &sz); // b = 50, sz = 2
int c = stoi("40c3", nullptr, 16); // c = 0x40c3 (十六进制)
int d = stoi("0x7f", nullptr, 0); // d = 0x7f (自动检查进制)
double e = stod("34.5"); // e = 34.5
```

七、同义词查询库 (自学)

使用 `map<string, vector<string>>` 构建词语与同义词的映射:

```
class SynonymBase {
    map<string, vector<string>> synonyms;
public:
    void add(string word, string synonym) {
        synonyms[word].push_back(synonym);
    }
    void query(string word) {
```

```

        if (synonyms.find(word) == synonyms.end())
            cout << "no synonyms found" << endl;
        else
            for (auto& x : synonyms[word]) cout << x << endl;
    }
};

```

增加判定两个词是否为同义词的需求时，将 vector 改为 set 以提高查找效率：

```

class SynonymBase {
    map<string, set<string>> synonyms;
public:
    bool _has_synonym(string word1, string word2) {
        return synonyms.find(word1) != synonyms.end() &&
            synonyms[word1].find(word2) != synonyms[word1].end();
    }
    bool isSynonyms(string word1, string word2) {
        if (word1 == word2) return true;
        return _has_synonym(word1, word2) || _has_synonym(word2, word1);
    }
};

```

八、函数模板与类模板特化（自学）

函数模板特化

对模板在某种具体类型下进行特殊处理：

```

template<class T>
T div2(const T& val) {
    cout << "using template" << endl;
    return val / 2;
}
template<>
int div2(const int& val) { // 函数模板特化
    cout << "better solution!" << endl;
    return val >> 1;       // 右移取代除以2
}

```

注意：函数模板有多个模板参数时，特化必须提供**所有**参数的特例类型，不能部分特化。但可以用重载来替代。

函数模板重载解析顺序：

1. 类型匹配的普通函数 → 基础函数模板 → 全特化函数模板
2. 先从基础模板中选择最匹配的，再查看有无对应的全特化版本

重要：函数模板的全特化版本匹配优先级可能低于重载的非特化基础模板，因此最好直接使用重载函数而非全特化。

类模板特化

类模板可以进行全部特化和部分特化：

```

// 通用模板
template<typename T1, typename T2> class A { ... };
// 全部特化：指定所有类型
template<> class A<int, int> { ... };
// 部分特化：只限制部分类型
template<typename T1> class A<T1, int> { ... };

```

全部特化示例：

```
template<typename T1, typename T2>
class Sum {
public:
    Sum(T1 a, T2 b) { cout << "Sum general: " << a+b << endl; }
};
template<>
class Sum<int, int> {
public:
    Sum(int a, int b) { cout << "Sum specific: " << a+b << endl; }
};
Sum<int, int> s1(1, 2);           // Sum specific: 3
Sum<int, double> s2(1, 2.5);     // Sum general: 3.5
```

部分特化示例：

```
template<typename T1>
class Sum<T1, int> {
public:
    Sum(T1 a, int b) { cout << "Sum specific: " << a+b << endl; }
};
Sum<double, int> s1(1.5, 2);      // Sum specific: 3.5
Sum<double, double> s2(1.5, 2.5); // Sum general: 4
```

模板特化总结

1. 类模板可以部分特化或全部特化，编译器根据类型参数自动选择
2. 函数模板只能全部特化，但可通过重载代替部分特化
3. 函数模板全特化匹配优先级可能低于重载的非特化基础模板，最好直接用重载

函数模板重载解析详解

解析顺序（优先级从高到低）：

1. 类型匹配的普通函数
2. 基础函数模板（选择最匹配的）
3. 该基础模板的全特化版本

关键陷阱： 全特化版本优先匹配的前提是——被特化的基础函数模板被匹配到！看下面的例子：

```
// 基础模板 1
template<class T> void f(T) { cout << "func1" << endl; }
// 基础模板 2（重载，更特化）
template<class T> void f(T*) { cout << "func2" << endl; }
// 特化基础模板 2
template<> void f(char*) { cout << "func3" << endl; }

int main() {
    char* p;
    f(p); // 输出：func3
    // 解析：f(T) vs f(T*), f(T*) 更特化 → 选基础模板2 → 查全特化 → f(char*)!
}
```

```
// 但如果特化的是基础模板1：
template<class T> void f(T) { cout << "func1" << endl; }
template<class T> void f(T*) { cout << "func2" << endl; }
template<> void f<char*>(char*) { cout << "func4" << endl; } // 特化f(T)!

int main() {
    char* p;
```

```
f(p); // 输出: func2 (不是 func4!)
// 解析: f(T) vs f(T*), f(T*) 更特化 → 选基础模板2 → f(T*)无全特化 → func2
}
```

教训： 函数模板的全特化不参与重载解析！先选基础模板，再找特化。因此最好直接使用重载函数而非全特化。

函数模板两种特化语法

```
// 方式1：显式指定类型
template<> char* sum<char*>(char* a, char* b);

// 方式2：编译器推导
template<> char* sum(char* a, char* b);
// 两种等价
```

拾 iostream 与函数对象和智能指针 2026-05-12

一、iostream 输入输出流

STL 输入输出流层次

流的继承关系：

	输入流	输出流	文件流	字符串流
头文件	<istream>	<ostream>	<fstream>	<sstream>
基类	istream	ostream	fstream	stringstream
标准对象	cin	cout		

文件流和字符串流都是继承自相应的基类。注意 iostream 是多重继承自 istream 和 ostream 的？！ ifstream 是 istream 的子类，ofstream 是 ostream 的子类。

ostream 和 cout

ostream 即 output stream，是 STL 库中所有输出流的基类。它重载了针对基础类型的输出流运算符 (<<)，统一了输出接口，改善了 C 中 printf 方式混乱的状况。

cout 是 STL 中内建的一个 ostream 对象，它会将数据送到标准输出流（一般是屏幕）。

<< 运算符为左结合！ 原理如下：

```
cout << "hello" << ' ' << "world";
// 先执行 cout << "hello" → 返回 cout 的引用 c1
// 再执行 c1 << ' ' → 返回 cout 的引用 c2
// 再执行 c2 << "world" → 完成
```

实现自己的 ostream

可以自己实现一个简易的 ostream 来理解原理：

```
class ostream {
public:
    ostream& operator<<(char c) {
        printf("%c", c);
        return *this;
    }
    ostream& operator<<(const char* str) {
        printf("%s", str);
        return *this;
    }
};
```

```

    }
} cout;

```

关键：返回 `*this` 的引用，使得链式调用得以实现！

格式化输出

使用 `#include <iomanip>` 进行格式化：

```

cout << fixed << 2018.0 << " " << 0.0001 << endl;
// 浮点数 → 2018.000000 0.000100
cout << scientific << 2018.0 << " " << 0.0001 << endl;
// 科学计数法 → 2.018000e+03 1.000000e-04
cout << defaultfloat; // 还原默认输出格式
cout << setprecision(2) << 3.1415926 << endl;
// 输出精度设置为2 → 3.2
cout << oct << 12 << " " << hex << 12 << endl;
// 八进制 → 14 十六进制 → c
cout << dec; // 还原十进制
cout << setw(3) << setfill('*') << 5 << endl;
// 设置对齐长度为3，对齐字符为* → **5

```

流操纵算子

`setprecision(2)` 看起来像函数调用，实际上它是创建了一个类的对象！

```

class setprecision {
private:
    int precision;
public:
    setprecision(int p) : precision(p) {}
    friend class ostream;
};
// setprecision(2) 是一个类的对象

```

`ostream` 中有对应的重载：

```

class ostream {
private:
    int precision; // 记录流的状态
public:
    ostream& operator<<(const setprecision &m) {
        precision = m.precision;
        return *this;
    }
} cout;

```

借助辅助类设置成员变量，这种类叫流操纵算子（stream manipulator）。

endl

`endl` 也是一个函数：

```

ostream& endl(ostream& os) {
    os.put('\n');
    os.flush();
    return os;
}
// 等同于输出 '\n'，再清空缓冲区 os.flush()

```

endl 同时也是流操纵算子，如何实现 `cout << endl`?

```
ostream& operator<<(ostream& (*fn)(ostream&)) {  
    // 流运算符重载，函数指针作为参数  
    return (*fn)(*this);  
}
```

缓冲区的目的是减少外部读写次数。写文件时，只有清空缓冲区或关闭文件才能保证内容正确写入。

不能复制的 cout

ostream 的复制构造函数被禁用了：

```
ostream(const ostream&) = delete; // 禁止复制  
ostream(ostream&& x);             // 只允许移动
```

为什么重载流运算符要返回引用？ 避免复制！ostream 不能被复制。为什么只能使用一个对象？

1. 减少复制开销
2. 一个对象对应一个标准输出，符合 OOP 思想
3. 多个对象之间无法同步输出状态

注意： 全局对象往往引入初始化顺序问题。更好的方式是单件模式（Singleton Pattern），在设计模式中会介绍。

文件输入输出流

ifstream 是 istream 的子类，功能是从文件中读入数据。

```
// 打开文件  
ifstream ifs("input.txt");  
ifstream ifs("binary.bin", ifstream::binary); // 二进制形式  
ifstream ifs;  
ifs.open("file");  
// do something  
ifs.close();
```

读入示例：

```
ifstream ifs("input.txt");  
while(ifs) { // 利用了重载的bool运算符判断文件是否到末尾  
    ifs >> ws; // 除去前导空格，ws也是流操纵算子  
    int c = ifs.peek(); // 检查下一个字符，但不读取  
    if (c == EOF) break;  
    if (isdigit(c)) {  
        int n;  
        ifs >> n;  
        cout << "Read a number: " << n << endl;  
    } else {  
        string str;  
        ifs >> str;  
        cout << "Read a word: " << str << endl;  
    }  
}
```

其他操作：

1. `getline(cin, str)` 读一行（ifstream 是 istream 的子类，所以 `getline(ifs, str)` 也有效）
2. `get()` 读取一个字符
3. `ignore(int n=1, int delim=EOF)` 丢弃 n 个字符，或直至遇到 delim

4. peek() 查看下一个字符
5. putback(char c) / unget() 返还一个字符

istream 与 scanf

为什么 C++ 使用流输入取代了 scanf?

1. **友好性**: scanf 不同类型要使用不同的标识符

```
scanf("%d %hd %f %lf %s", &i, &s, &f, &d, name);
cin >> i >> s >> f >> d >> name;
```

2. **安全性**: scanf 可能写入非法内存
3. **可拓展性**: 可以重载 >> 支持自定义类型 cin >> obj
4. **性能**: scanf 在运行期间解析格式字符串, istream 在编译期间已解析完毕

字符串输入输出流

stringstream 是 istream 的子类, 而 iostream 多重继承自 istream 和 ostream。

stringstream 在对象内部维护了一个 buffer:

1. 用流输出函数可以将数据写入 buffer
2. 用流输入函数可以从 buffer 中读出数据

```
// 构造方式
stringstream ss;           // 空字符串流
stringstream ss(str);      // 以字符串初始化流
```

使用示例:

```
stringstream ss;
ss << "10";
ss << "0 200";
int a, b;
ss >> a >> b; // a=100, b=200
```

获取 buffer: ss.str() 返回一个 string 对象, 内容为 stringstream 的 buffer。

注意 buffer 内容并不是未读取的内容:

```
stringstream ss;
ss << "100 200";
cout << ss.str() << endl; // "100 200"
int a, b;
ss >> a; // a = 100
cout << ss.str() << endl; // 仍然是 "100 200"!
ss >> b; // b = 200
```

类型转换函数

利用 stringstream 可以实现通用的类型转换:

```
template<class outtype, class intype>
outtype convert(intype val) {
    static stringstream ss; // 静态变量避免重复初始化
    ss.str(""); // 清空缓冲区
    ss.clear(); // 清空状态位 (不是清空内容!)
    ss << val;
    outtype res;
    ss >> res;
    return res;
}
```

ss.str("") 和 ss.clear() 的区别：前者清空内容，后者清空状态位（记录流的状态，例如是否读入了非法字符）。两个都要做！

二、函数对象

引例：一个计算器的三种实现方式

问题场景：一个计算器包含三个步骤——读取数据、处理数据、输出数据。每个步骤都有多种实现方式（从屏幕读取 vs 从文件读取，加法 vs 乘法，输出到屏幕 vs 输出到文件）。

方式一：OOP + 模板方法

```
class CalculatorBase {
public:
    virtual string read() = 0;
    virtual string calculate(string) = 0;
    virtual void write(string) = 0;
    void process() { // 算法骨架
        string data = read();
        string output = calculate(data);
        write(output);
    }
};
// 问题: n种read × m种calculate × k种write → n*m*k个子类！
```

方式二：函数指针

```
void process(
    string (*read)(),
    string (*calculate)(string),
    void (*write)(string))
{
    string data = read();
    string output = calculate(data);
    write(output);
}
process(readFromScreen, calculateAdd, writeToFile);
// 问题: 函数指针和函数对象是不同类型，不能混用！
```

方式三：函数模板（静多态）

```
template<class ReadFunc, class CalFunc, class WriteFunc>
void process(ReadFunc read, CalFunc calculate, WriteFunc write) {
    string data = read();
    string output = calculate(data);
    write(output);
}
// 函数指针和函数对象都能传入！
process(readFromScreen, calculateAdd, writeToScreen);
process(ReadFromFile(), calculateAdd, writeToScreen);
```

方式四：std::function（统一接口）

```
void process(
    function<string()> read,
    function<string(string)> calculate,
    function<void(string)> write)
{
    string data = read();
    string output = calculate(data);
}
```

```

    write(output);
}
// 最灵活！函数指针和函数对象都能赋值给function

```

函数指针

如果 flag 为 1 则对每个元素调用 increase，否则调用 decrease。如何减少重复逻辑？

可以用函数指针：

```

void (*func)(int&); // 函数指针的声明
// 返回值  指针符号  声明的变量名  参数列表

```

```

void (*func)(int&);
if (flag == 1) func = increase;
else func = decrease;
for (int &x : arr) { func(x); }

```

也可以用 auto 自动推断：

```

auto func = flag==1 ? increase : decrease;
for (int &x : arr) { func(x); }
// auto 自动推断出 func 的类型为 void (*)(int&)

```

和数组类似：数组名 = 指向数组第一个元素的指针，函数名 = 指向函数的指针。

函数作为变量

std::sort 来自 <algorithm>：

```

int arr[5] = { 5, 2, 3, 1, 7 };
std::sort(arr, arr + 5); // 默认从小到大

```

如果想倒序排序？sort 重载了另一套参数：

```

template <class Iterator, class Compare>
void sort(Iterator first, Iterator last, Compare comp);

```

比较函数 comp：传入两个值，若 a 在 b 前则返回 true。

```

bool comp(int a, int b) { return a > b; }
std::sort(arr, arr + 5, comp); // 7 5 3 2 1

```

STL 还提供了预定义的比较函数（#include <functional>）：

```

sort(arr, arr+5, less<int>()); // 从小到大
sort(arr, arr+5, greater<int>()); // 从大到小

```

疑问：greater<int>() 为什么带括号？是什么？

函数对象

greater<int>() 实际上是一个对象！

1. greater 是一个模板类
2. greater<int> 是用 int 实例化的类
3. greater<int>() 是该类的一个对象

同时它表现得像一个函数：

```
auto func = greater<int>();
cout << func(2, 1) << endl; // True
cout << func(1, 1) << endl; // False
cout << func(1, 2) << endl; // False
```

因此，这种对象被称为**函数对象**（Functor）。

如何实现函数对象

```
template<class T>
class Greater {
public:
    bool operator()(const T &a, const T &b) const {
        return a > b;
    }
};
```

注意三个 const! 排序中 comp 不能修改数据，一般情况下 comp 也不应该修改自身。

函数对象的要求：

1. 需要重载 operator() 运算符
2. 该函数需要是 public 访问权限

小知识：Duck Typing 鸭子类型 —— 如果一个物体，叫声像鸭子、走路像鸭子，那么它就是鸭子。如果一个对象用起来像函数，那么它就是函数对象！C++没有严格定义什么是函数对象，但实践上按 Duck Typing 来处理。

自己实现 sort

sort 的第三个参数 Compare 是模板类型，可以接受函数指针或函数对象：

```
template<class Iterator, class Compare>
void mysort(Iterator first, Iterator last, Compare comp) {
    for (auto i = first; i != last; i++)
        for (auto j = i; j != last; j++)
            if (!comp(*i, *j)) swap(*i, *j);
}
mysort(arr, arr + 5, comp); // 函数指针
mysort(arr, arr + 5, greater<int>()); // 函数对象
```

自定义类型的排序

假设有 class People { public: int age, weight; };

方法一：重载小于运算符 —— 但只能定义一种排序规则，体重怎么办？

```
class People {
public:
    int age, weight;
    bool operator<(const People &b) const { return age < b.age; }
};
sort(vec.begin(), vec.end());
```

方法二：定义比较函数

```
bool compByAge(const People &a, const People &b) { return a.age < b.age; }
sort(vec.begin(), vec.end(), compByAge);
```

方法三：定义比较函数对象

```
class AgeComp {
public:
    bool operator()(const People &a, const People &b) const { return a.age < b.age; }
};
sort(vec.begin(), vec.end(), AgeComp());
```

std::function 类

来自 <functional> 头文件。问题：想用数组储存函数指针和函数对象的选项？auto 无法推导！因为函数指针和函数对象不是同一种类型。

std::function 为两者提供了统一的接口：

```
function<string()> readArr[] =
    {readFromScreen, ReadFromFile()};
function<string(string)> calculateArr[] =
    {calculateAdd, CalculateMul()};
function<void(string)> writeArr[] =
    {writeToScreen, WriteToFile()};
```

function 也允许赋值和改写：

```
function<string()> readFunc;
readFunc = readFromScreen;    // 允许函数的赋值
readFunc = ReadFromFile();    // 也允许函数对象的赋值

string (*readFunc2)();
readFunc2 = readFromScreen;
// readFunc2 = ReadFromFile(); // 错误！类型不一致
```

对比几种实现方式：

	虚函数	模板	std::function
支持函数指针	否	✓	✓
支持函数对象	✓ (子类)	✓	✓
确定时机	运行时	编译期	运行时
需要基类	✓	否	否

std::function 的意义：

1. 函数对象化，万物皆对象，符合 OOP 设计理念
2. 函数可以作为参数传递、作为变量储存
3. 不再需要模板来调用不同的函数，所有的函数都可以看做 std::function

STL 与函数对象

STL 有大量函数用到了函数对象 (#include <algorithm>):

1. for_each 对序列进行指定操作
2. find_if 找到满足条件的对象
3. count_if 对满足条件的对象计数
4. binary_search 二分查找满足条件的对象

预置的函数对象 (#include <functional>):

1. less 比较 $a < b$
2. equal_to 比较 $a == b$
3. greater 比较 $a > b$
4. plus 返回 $a + b$

三、智能指针与引用计数

shared_ptr

问题：A、B 对象共享一个 C 对象，C 对象不想交由外部销毁。A、B 中的谁负责销毁 C？应该在 A、B 都销毁时 C 才能销毁！

shared_ptr 来自 <memory> 库：

```
// 构造方法
shared_ptr<int> p1(new int(1));
shared_ptr<MyClass> p2 = make_shared<MyClass>(2);
shared_ptr<MyClass> p3 = p2;          // 共享所有权
shared_ptr<int> p4;                  // 空指针

// 访问对象
int x = *p1;                        // 从指针访问对象
int y = p2->val;                     // 访问成员变量

// 销毁对象
// p2和p3指向同一对象，当两者均出作用域才会被销毁
```

引用计数

为什么智能指针能够知道何时销毁对象？**引用计数**！当引用计数归 0 时，销毁对象。

```
shared_ptr<int> p1(new int(4));
cout << p1.use_count() << ' '; // 1
{
    shared_ptr<int> p2 = p1;
    cout << p1.use_count() << ' '; // 2
    cout << p2.use_count() << ' '; // 2
} // p2 出作用域
cout << p1.use_count() << ' '; // 1
} // p1 出作用域，引用计数归0，销毁 int* 4
```

实现自己的引用计数

使用辅助指针类 U_Ptr 来存放实际数据和计数：

```
template <typename T>
class U_Ptr { // 辅助指针
    friend class SmartPtr<T>;
    U_Ptr(T *ptr) : p(ptr), count(1) {}
    ~U_Ptr() { delete p; }
    int count;
    T *p; // 实际数据存放
};
```

智能指针类：

```
template <typename T>
class SmartPtr {
    U_Ptr<T> *rp;
public:
    SmartPtr(T *ptr) : rp(new U_Ptr<T>(ptr)) {}
    SmartPtr(const SmartPtr<T> &sp) : rp(sp.rp) {
        ++rp->count;
    }
    SmartPtr& operator=(const SmartPtr<T> & rhs) {
```

```

        ++rhs.rp->count;
        if (--rp->count == 0) // 减少自身所指rp的引用计数
            delete rp;
        rp = rhs.rp;
        return *this;
    }
    ~SmartPtr() {
        if (--rp->count == 0)
            delete rp;
    }
    T& operator*() { return *(rp->p); }
    T* operator->() { return rp->p; }
};

```

注意： 不能使用同一裸指针初始化多个智能指针！

```

int* p = new int();
shared_ptr<int> p1(p);
shared_ptr<int> p2(p); // 会产生多个辅助指针，双重释放！

```

shared_ptr 其他用法：

1. p.get() 获取裸指针
2. p.reset() 清除指针并减少引用计数
3. static_pointer_cast<int>(p) 类似 static_cast，无类型检查
4. dynamic_pointer_cast<Base>(p) 类似 dynamic_cast，动态类型检查

weak_ptr：解决循环引用

智能指针不总是智能！ Parent 和 Child 互相持有 shared_ptr 会怎样？

```

class Parent {
    shared_ptr<Child> child;
    ...
};
class Child {
    shared_ptr<Parent> parent; // 互相引用！
    ...
};
// test() 结束后没有析构！内存泄漏！

```

原因：两个对象的引用计数都是 1，谁都不会先销毁。

解决：使用 weak_ptr，指向对象但不增加引用计数：

```

class Child {
    weak_ptr<Parent> parent; // 修改为弱引用
    ...
};

```

weak_ptr 的用法：

1. wp.use_count() 获取引用计数
2. wp.reset() 清除指针
3. wp.expired() 检查对象是否无效
4. sp = wp.lock() 从弱引用获得一个 shared_ptr

```
std::weak_ptr<int> wp;
{
    auto sp1 = std::make_shared<int>(20);
    wp = sp1;
    cout << wp.use_count() << endl; // 1
    auto sp2 = wp.lock();
    cout << wp.use_count() << endl; // 2
    sp1.reset();
    cout << wp.use_count() << endl; // 1
} // sp2 销毁
cout << wp.use_count() << endl; // 0
cout << wp.expired() << endl; // true
```

unique_ptr: 独享所有权

shared_ptr 涉及引用计数，性能较差。如果保证一个对象只被一个指针引用，使用 unique_ptr:

```
auto up1 = std::make_unique<int>(20);
// unique_ptr<int> up2 = up1; // 错误！不能复制
unique_ptr<int> up2 = std::move(up1); // 可以移动
int* p = up2.release(); // 放弃指针控制权，返回裸指针
delete p;
```

智能指针总结

	shared_ptr	weak_ptr	unique_ptr
所有权	共享	不拥有	独占
引用计数	✓	可查询但不增加	无
可复制	✓	✓	否（可移动）
适用场景	一般共享	解决循环引用	独占所有权

优点:

1. 帮助管理内存，避免内存泄露
2. 区分 unique_ptr 和 shared_ptr 能够明确语义
3. 在手动维护指针不可行、复制对象开销太大时是唯一选择

缺点:

1. 引用计数会影响性能
2. 智能指针不总是智能，需要了解内部原理
3. 需要小心环状结构和数组指针

四、字符串处理与正则表达式（自学）

正则表达式基础

正则表达式：由字母和符号组成的特殊文本，搜索文本时定义的一种规则。

三种模式:

1. **匹配**: 判断整个字符串是否满足条件
2. **搜索**: 找出符合正则表达式的子串
3. **替换**: 按规则替换字符串的子串

引例：用户名验证

先看一个不使用正则表达式的实现（繁琐！）:

```
bool valid_username(string s) {
    if (s.size() < 3 || s.size() > 15) return false;
    for (char c : s)
```

```

    if (!islower(c) && !isdigit(c) && c != '_')
        return false;
    return true;
}
// 等价的正则表达式: ^[a-z0-9_]{3,15}$

```

在线测试工具: <http://tool.chinaz.com/regex/>

字符簇 (使用 “The car parked in the garage.” 演示)

以例句 “The car parked in the garage.” 演示各模式匹配结果:

1. [a-z] 匹配所有单个小写字母
2. [0-9] 匹配所有单个数字
3. \d 等价 [0-9]
4. \w 匹配字母、数字、下划线, 等价 [a-zA-Z0-9_]
5. \s 匹配所有非空白字符
6. . 匹配除换行以外任意字符
7. [^a-z] 匹配所有非小写字母的单个字符

具体匹配演示:

1. [Tt]he → 匹配 “The” (句子开头)
2. \. → 匹配 “.” (句号)
3. .ar → 匹配 “car”、“par” (“car parked” 中匹配 “car”、“par”)
4. [^c]ar → 匹配 “par” (“parked” 中匹配 “par”, 排除 “car”)
5. (c|g|p)ar → 匹配 “car”、“gar”、“par”
6. (T|t)he|car → 匹配 “The” 或 “car”

重复模式

1. x{n,m} 前面内容出现 n 到 m 次
2. + 至少出现 1 次 (等价 {1,})
3. * 至少出现 0 次 (等价 {0,})
4. ? 出现 0 次或 1 次 (等价 {0,1})
5. ^ 代表字符串开头, \$ 代表字符串结尾

以 “The car parked in the garage.” 演示:

1. a\w+ → 匹配 “ar” (“car”后面)、“arked” (“parked”后面)、“age” (“garage”后面)
2. a\w* → 匹配更多 (包括单独的 “a”)
3. c.+e → 匹配 “car parked in the garage” (贪婪! 从 “c” 到最后的 “e”)
4. [a-z]* → 匹配 “he”、“car”、“parked”、“in”、“the”、“garage”
5. .{5} → 匹配任意 5 个字符的子串

或连接符

1. (Chapter|Section) [1-9][0-9]? 匹配 Chapter 1、Section 10 等
2. m|food 匹配 m 或 food
3. (m|f)ood 匹配 mood 或 food

C++ <regex> 库

创建正则表达式对象:

```

regex re("[1-9][0-9]{10}$"); // 11位数
// 注意: C++字符串中 "\" 也是转义字符
regex re("\\d+"); // 如果需要 \d+, 应该写成 "\\d+"

```

原生字符串: R"(str)" 表示 str 的字面值, 取消转义。 "\\d+" = R"(\d+)" = \d+

匹配: `regex_match`

```
string s("subject");
regex e("sub.*");
if(regex_match(s, e))
    cout << "matched" << endl; // matched
```

捕获和分组

使用 `()` 进行标识，每个标识的内容被称作分组。匹配后每个分组的内容将被**捕获**。

```
string s("version10");
regex e(R"(version(\d+))");
smatch sm;
if(regex_match(s, sm, e)) {
    cout << sm.size() << " matches\n"; // 2 matches
    for (unsigned i = 0; i < sm.size(); ++i)
        cout << sm[i] << endl; // version10 → 10
}
```

分组编号：0 号永远是匹配的字符串本身，之后按顺序编号。

搜索: `regex_search`

搜索字符串中能匹配正则表达式的**第一个**子串：

```
string s("this subject has a submarine");
regex e(R"((sub)([S]*))");
smatch sm;
while(regex_search(s, sm, e)) {
    for (unsigned i = 0; i < sm.size(); ++i)
        cout << "[" << sm[i] << " ";
    cout << endl;
    s = sm.suffix().str(); // 继续搜索剩余部分
}
// [subject] [sub] [ject]
// [submarine] [sub] [marine]
```

替换: `regex_replace`

```
string s("this subject has a submarine");
regex e(R"((sub)([S]*))");
cout << regex_replace(s, e, "SUBJECT") << endl;
// this SUBJECT has a SUBJECT
cout << regex_replace(s, e, "[$&]") << endl;
// this [subject] has a [submarine] ($& = 匹配的子串)
cout << regex_replace(s, e, "$1") << endl;
// this sub has a sub ($1 = 第1个分组)
cout << regex_replace(s, e, "$2") << endl;
// this ject has a marine ($2 = 第2个分组)
```

例题：学生信息提取

从自我介绍中提取名字、出生年月、电话号码、邮箱：

```
void extract(string input) {
    smatch sm;
    regex get_name(R"((My name is |I am )(\w+)\.))");
    regex get_date(R"((\d{4})[\. -](\d{1,2})[\. -](\d{1,2}))");
    regex get_mobile(R"([1-9]\d{10})");
```

```

regex get_email(R"([\w]+@(\w+\.)+\w+)");
if (regex_search(input, sm, get_name))
    cout << sm[2] << endl;
if (regex_search(input, sm, get_date))
    cout << sm[1] << "." << sm[2] << "." << sm[3] << endl;
if (regex_search(input, sm, get_mobile))
    cout << sm[0] << endl;
if (regex_search(input, sm, get_email))
    cout << sm[0] << endl;
}

```

更多正则内容（自学）

1. (?:pattern) 不捕获的分组
2. 正向预查 (?=pattern) / (!pattern)
3. 反向预查 (?<=pattern) / (?<!pattern)
4. 后向引用: \b(\w+)\b\s+\1\b 匹配重复两遍的单词（如 go go）
5. 贪婪与懒惰：默认贪婪匹配，在重复模式后加 ? 变为懒惰匹配

五、拓展自学：function 绑定优先级

对于 `function<____> pf = func`，有如下准则：

绑定准则：

1. function 的参数要比实际参数**更严格**（所有能被 function 接受的参数都应被实际函数接受）
2. function 的返回值要比实际返回值**更宽松**（所有可能的实际函数返回值都可能被 function 返回）

绑定优先级表：

	lvalue	const lvalue	rvalue	const rvalue
int	✓	✓	✓	✓
int&	✓			
const int&	✓	✓	✓	✓
int&&			✓	
const int&&			✓	✓

例外：当实际参数为 T&& 时，function 参数可以为 T。pf2 在内部拷贝了一份参数，然后将该拷贝的右值传给 Func2()。

拾壹 行为型模式 2026-05-19

幻灯片 L12 涵盖设计模式概述、模板方法、策略模式、迭代器模式的完整内容。课后参考：<https://www.liaoxuefeng.com/wiki/1252599548343744/1281319453589538>

一、设计模式概述

设计模式（Design Pattern）是在长时间实践中开发人员总结出的优秀架构与解决方案。

1. 首见于《Design Patterns - Elements of Reusable Object-Oriented Software》
2. 遵循面向对象设计原则：对接口编程而不是对实现编程；优先使用对象组合而不是继承

三大类设计模式：

行为型模式	关注对象行为功能上的抽象，提升可拓展性，以最少代码变动完成功能增减
结构型模式	关注对象之间结构关系上的抽象，提升可维护性和健壮性，在结构层面解耦合
创建型模式	将对象的创建与使用划分，规避复杂对象创建的资源消耗，高效创建对象

本讲内容：12.1 模板方法、12.2 策略模式、12.3 迭代器模式

二、引例：负载监视器

监视计算节点的负载状态（如 CPU 占用率），不同 OS 获得 CPU 占用率的方法不同。

简单枚举（不好）

用 switch-case 枚举所有系统类型：

```
enum MonitorType {Win32, Win64, Ganglia};

class Monitor {
public:
    void getLoad();
    void getTotalMemory();
    void getUsedMemory();
    void getNetworkLatency();
    void show();
private:
    float load, latency;
    long totalMemory, usedMemory;
    Display* m_display;
};

void Monitor::getLoad() {
    switch (type) {
        case Win32: load = ...; break;
        case Win64: load = ...; break;
        case Ganglia: load = ...; break;
    }
}
```

问题：每新增一种系统就要修改 Monitor 类内部的 switch，违反开放封闭原则。

三、模板方法（Template Method）模式

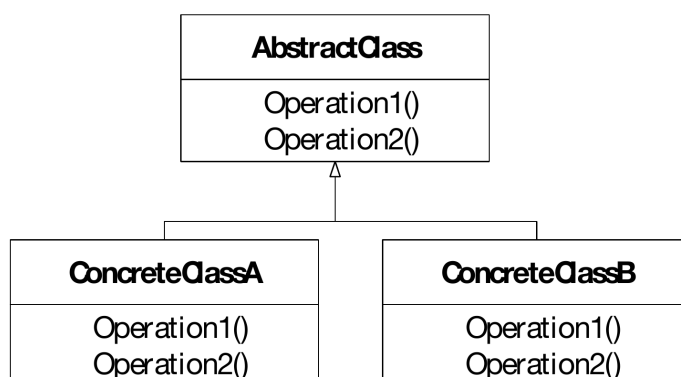
生活类比：写作文

小学学写作文时，老师先教你经典结构——“起承转合”（开头→承接→转折→结尾）。这个**结构（算法骨架）**是固定的。但面对不同题目（“我的妈妈”、“难忘的一天”），你填入的**具体内容（步骤实现）**可以完全不同。

模板方法模式就是：**骨架在基类中定义，具体内容在子类中实现。**

模板方法

- 在接口的一个方法中定义算法的骨架
- 将一些步骤的实现延迟到子类中
- 使得子类可以在不改变算法结构的情况下，重新定义算法中的某些步骤。



1. 在接口的一个方法中定义**算法的骨架**，将一些步骤的实现延迟到子类中
2. 子类可以在不改变算法结构的情况下，重新定义算法中的某些步骤
3. 抽象类定义骨架，子类实现细节；扩展时只需新增子类，无需修改已有类

代码实现

```
class Monitor {
public:
    virtual void getLoad() = 0;
    virtual void getTotalMemory() = 0;
    virtual void getUsedMemory() = 0;
    virtual void getNetworkLatency() = 0;
    void show();
protected:
    float load, latency;
    long totalMemory, usedMemory;
    Display* m_display;
};

// 子类只需实现纯虚函数
class MonitorWin32 : public Monitor {
public:
    void getLoad() { ... load = ...; }
    void getTotalMemory() { ... totalMemory = ...; }
    void getUsedMemory() { ... usedMemory = ...; }
    void getNetworkLatency() { ... latency = ...; }
};

// 使用基类指针调用
int main() {
    Monitor* monitor = new MonitorWin32(&display);
    while (running()) {
        monitor->getLoad();
        monitor->getTotalMemory();
        monitor->getUsedMemory();
        monitor->getNetworkLatency();
        monitor->show();
        sleep(1000);
    }
    delete monitor;
}
```

针对接口编程

1. 抽象出“接口类”，有一系列（纯）虚函数描述“接口”
2. 继承并实现这些虚函数，形成“实现类”
3. 使用接口类引用概念，不直接使用实现类 → 避免实现变化造成大规模代码变动

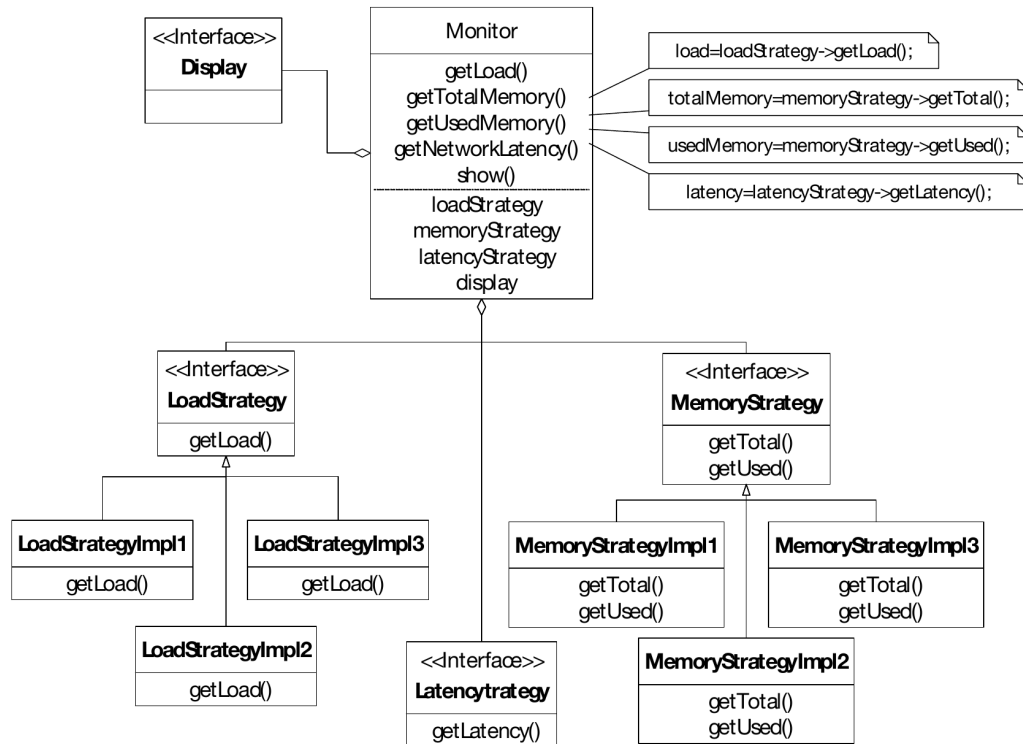
开放封闭原则

1. 对**扩展开放**：有新需求时可以方便地扩展，无需整体变动
2. 对**修改封闭**：新的扩展类一旦设计完成，可以独立工作
3. 核心：对**抽象**编程，而不对**具体**编程。抽象结构简单稳定，具体实现复杂多变

模板方法的局限

若 `getLoad()` 有 n 种实现、`getNetworkLatency()` 有 m 种实现、`getTotalMemory()` 与 `getUsedMemory()` 有 k 种实现，则需要 $n \times m \times k$ 个子类。大量冗余！

具体化到我们的问题



23

1. 定义一系列算法并加以封装，使得这些算法可以互相替换
2. 将每个功能抽象为独立的策略接口，Monitor 类通过**组合**策略对象来工作
3. 类数量：1 个 Monitor + 3 个抽象策略类 + $(n + m + k)$ 个策略实现类 = $n + m + k + 4$

代码实现

```
// 负载策略基类
class LoadStrategy {
public:
    virtual float getLoad() = 0;
};

class LoadStrategyImpl1 : public LoadStrategy {
public:
    float getLoad() { ... return load; }
};

// 内存策略基类
class MemoryStrategy {
public:
    virtual long getTotal() = 0;
    virtual long getUsed() = 0;
};

class MemoryStrategyImpl1 : public MemoryStrategy {
public:
    long getTotal() { ... return total; }
    long getUsed() { ... return used; }
};

// Monitor 组合各个策略
class Monitor {
```

```

public:
    Monitor(LoadStrategy *ls, MemoryStrategy *ms,
            LatencyStrategy *lts, Display *display);
    void getLoad() { load = m_loadStrategy->getLoad(); }
    void getTotalMemory() { totalMemory = m_memStrategy->getTotal(); }
    void getUsedMemory() { usedMemory = m_memStrategy->getUsed(); }
    void getNetworkLatency() { latency = m_latencyStrategy->getLatency(); }
    void show();
private:
    LoadStrategy *m_loadStrategy;
    MemoryStrategy *m_memStrategy;
    LatencyStrategy *m_latencyStrategy;
    Display *m_display;
    float load, latency;
    long totalMemory, usedMemory;
};

// 使用：为每个策略选择具体实现
int main() {
    GangliaLoadStrategy loadStrategy;
    WinMemoryStrategy memoryStrategy;
    PingLatencyStrategy latencyStrategy;
    WindowsDisplay display;
    Monitor monitor(&loadStrategy, &memoryStrategy, &latencyStrategy, &display);
    while (running()) {
        monitor.getLoad();
        monitor.getTotalMemory();
        monitor.getUsedMemory();
        monitor.getNetworkLatency();
        monitor.show();
        sleep(1000);
    }
}

```

单一责任原则

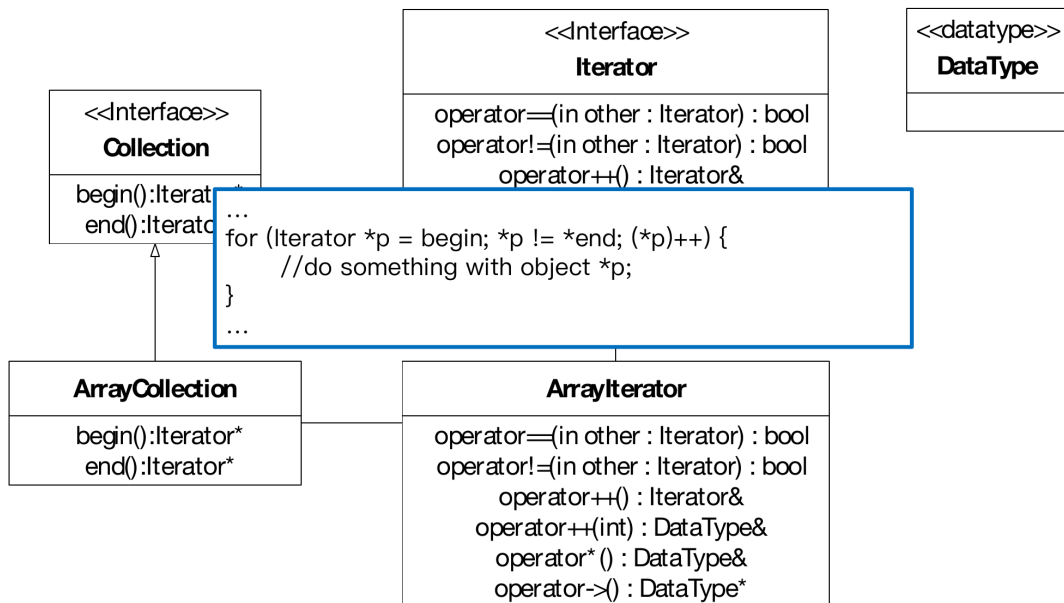
1. 一个类（接口）只负责一项职责，不要存在多于一个导致类变更的原因
2. 职责过多 → 耦合度大 → 职责变化可能削弱其他职责的能力
3. 核心：在**功能**层面上解耦

模板方法 vs 策略模式

	模板方法	策略模式
核心思路	定义算法骨架，延迟到子类实现；优先继承	定义一系列算法封装替换；优先组合
新增 getTotalMemory()+getUsedMemory()	需新增 $n \times m$ 个子类	只需新增 1 个 MemoryStrategy 实现类
优点	基类高度抽象统一，逻辑简洁；封装性好	每个策略只负责一个功能，易于拓展；算法修改不影响整体
弊端	接口同时负责所有功能，任何算法修改导致整个实现类变化	功能较多时结构复杂；策略组合对外暴露，封装性较差

1. 模板方法：适合**逻辑复杂但结构稳定**的场景，部分步骤变化剧烈且无相互关联
2. 策略模式：适合**算法本身灵活多变**的场景，且多种算法需协同工作
3. 业务简单时二者几乎等效

迭代器模式



48

1. 提供一种方法顺序访问聚合对象中各个元素，又不暴露该对象的内部表示
2. 与对象内部数据结构形式无关（数组还是链表）
3. 分离“变”（存储方式）与“不变”（访问方式）

引例：遍历学生成绩（逐步演进）

起点：原始数组实现

```

int main() {
    float scores[] = {90, 20, 40, 40, 30, 60, 70, 30, 90, 100};
    int passed = 0;
    for (int i = 0; i < 10; i++)
        if (scores[i] >= 60) passed++;
    cout << "passing rate = " << (float)passed / 10 << endl;
}

```

提取为函数

```

void analyze(float *scores, int student_count) {
    int passed = 0;
    for (int i = 0; i < student_count; i++)
        if (scores[i] >= 60) passed++;
    cout << "passing rate = " << (float)passed / student_count << endl;
}

```

需求变化：改用链表存储

```

struct Student {
    float score;
    Student* next;
};
void analyze(Student *head) {
    int passed = 0, count = 0;
}

```

```

for (Student *p = head; p != NULL; p = p->next) {
    if (p->score >= 60) passed++;
    count++;
}
cout << "passing rate = " << (float)passed / count << endl;
}

```

核心问题： 算法逻辑相同，但数据结构一变就要重写函数！ m 种算法 $\times$ n 种存储 = $m \times n$ 份代码。

“变”与“不变”

1. **不变：** 算法（遍历元素、计数、计算通过率）
2. **变：** 数据存储方式（数组、链表、树、图...）

关键在于：将“访问数据”这个操作抽象为统一接口——这就是**迭代器模式**的动机！

```

// 数组遍历
for (int i = 0; i != STUDENT_COUNT; i++) {
    if (scores[i] >= 60) passed++;
}
// 链表遍历
for (Student *p = head; p != NULL; p = p->next) {
    if (p->score >= 60) passed++;
}

```

如何实现与底层数据结构无关的统一算法接口？→ 迭代器！

迭代器基类

```

class Iterator {
public:
    virtual ~Iterator() {}
    virtual Iterator& operator++() = 0;    // 前缀++
    virtual float& operator++(int) = 0;    // 后缀++
    virtual float& operator*() = 0;        // 解引用
    virtual float* operator->() = 0;       // 箭头
    virtual bool operator!=(const Iterator &other) const = 0;
    bool operator==(const Iterator &other) const {
        return !(*this != other);
    }
};

```

Collection 基类

```

class Collection {
public:
    virtual ~Collection() {}
    virtual Iterator* begin() const = 0;    // 头迭代器 [begin, end)
    virtual Iterator* end() const = 0;      // 尾迭代器
    virtual int size() = 0;
};

```

数组实现

```

class ArrayCollection : public Collection {
    friend class ArrayIterator;
    float* _data;
    int _size;
public:

```

```

ArrayCollection(int size, float* data) : _size(size) {
    _data = new float[_size];
    for (int i = 0; i < size; i++) _data[i] = data[i];
}
~ArrayCollection() { delete[] _data; }
int size() { return _size; }
Iterator* begin() const { return new ArrayIterator(_data, 0); }
Iterator* end() const { return new ArrayIterator(_data, _size); }
};

class ArrayIterator : public Iterator {
    float* _data;
    int _index;
public:
    ArrayIterator(float* data, int index) : _data(data), _index(index) {}
    Iterator& operator++() { _index++; return *this; }
    float& operator++(int) { _index++; return _data[_index - 1]; }
    float& operator*() { return *(_data + _index); }
    float* operator->() { return (_data + _index); }
    bool operator!=(const Iterator &other) const {
        return (_data != ((ArrayIterator*)&other)->_data ||
                _index != ((ArrayIterator*)&other)->_index);
    }
};

```

使用迭代器

```

// 算法不依赖具体数据结构！
void analyze(Iterator* begin, Iterator* end) {
    int passed = 0, count = 0;
    for (Iterator* p = begin; *p != *end; (*p)++) {
        if (**p >= 60) passed++;
        count++;
    }
    cout << "passing rate = " << (float)passed / count << endl;
}

int main() {
    float scores[] = {90, 20, 40, 40, 30, 60, 70, 30, 90, 100};
    Collection *collection = new ArrayCollection(10, scores);
    analyze(collection->begin(), collection->end());
}

```

前缀 ++ 返回 Iterator&, 后缀 ++ 返回 float& (而非 Iterator) —— 因为 Iterator 是抽象类, 无法实例化。

另一种迭代器接口

```

// hasNext/next/getValue 风格
Iterator* it = collection->iterator();
while (it->hasNext()) {
    it->next();
    Object object = it->getValue();
}

```

STL 中的迭代器

STL 用模板而非继承实现迭代器模式, 更简洁:

```
// 基于继承：需要基类指针，间接调用
void analyze(Iterator* begin, Iterator* end);

// 基于模板：直接使用迭代器对象，但对每种类型都生成一份代码
template<class Iterator>
void analyze(Iterator begin, Iterator end) {
    int passed = 0, count = 0;
    for (Iterator p = begin; p != end; p++) {
        if (*p >= 60) passed++;
        count++;
    }
    cout << "passing rate = " << (float)passed / count << endl;
}

int main() {
    std::vector<float> scores{90, 20, 40, 40, 30, 60, 70, 30, 90, 100};
    analyze(scores.begin(), scores.end());
}
```

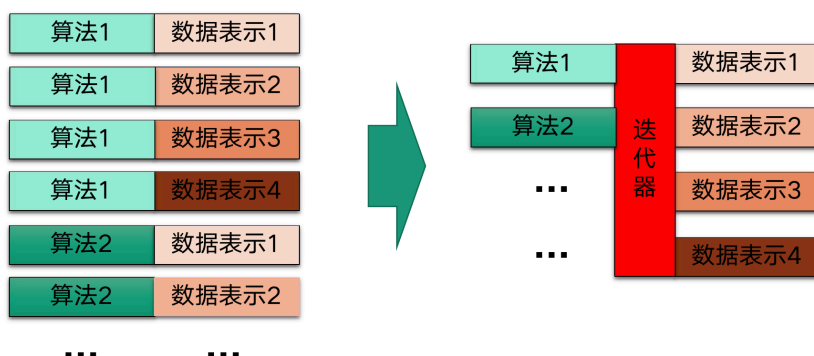
	继承	模板
调用方式	基类指针间接调用	直接使用迭代器对象
代码量	只生成一份代码	对每种类型生成一份，编译慢、可执行文件大
返回类型	Iterator*	vector::iterator<T>

for (auto i : container) 语法糖本质上也是基于迭代器。

迭代器模式总结

总结

- 迭代器模式实现了**算法**和**数据存储**的隔离
- 规避了为每一个**算法**和**数据存储**的组合均进行代码实现的巨大工作量



67

迭代器模式实现了**算法**和**数据存储**的隔离： m 个算法 $\times$ n 种存储 $\rightarrow$ 只需 $m + n$ 份代码（而非 $m \times n$ 份）。

六、总结

行为型设计模式核心：抽象行为功能中**不变**的成分，具体实现**变**的成分。

1. **模板方法**：归纳通用功能，基类固定接口，子类实现细节 $\rightarrow$ 体现开放封闭原则
2. **策略模式**：抽象功能的选择与组合，隔离不同功能互不影响 $\rightarrow$ 体现单一责任原则
3. **迭代器模式**：抽象数据访问方法，不暴露底层实现，隔离具体算法与数据结构

拾贰 结构型模式 2026-05-26

一、引例：栈

功能类似数组，但元素访问规则是“后进先出”（LIFO）。简单起见，只支持 int 类型。

堆栈基类

```
#include <climits>
#include <vector>
#include <iostream>
using namespace std;

class Stack {
public:
    virtual ~Stack() { }
    virtual bool full() = 0;
    virtual bool empty() = 0;
    virtual void push(int i) = 0;
    virtual void pop() = 0;
    virtual int size() = 0;
    virtual int top() = 0;
};
```

简单实现

```
class MyStack : public Stack {
private:
    int *m_data; const int m_size; int m_top;
public:
    MyStack(int size) : m_size(size), m_top(-1), m_data(NULL) {
        if (m_size > 0) m_data = new int[m_size];
    }
    virtual ~MyStack() {
        if (m_data) delete [] m_data;
    }
    bool full() { return m_size <= 0 || (m_top+1) == m_size; }
    bool empty() { return m_top < 0; }
    void push(int i) { if (m_top+1 < m_size) m_data[++ m_top] = i; }
    void pop() { if (!empty()) --m_top; }
    int size() { return m_top+1; }
    int top() {
        if (!empty()) return m_data[m_top];
        else return INT_MIN;
    }
};
```

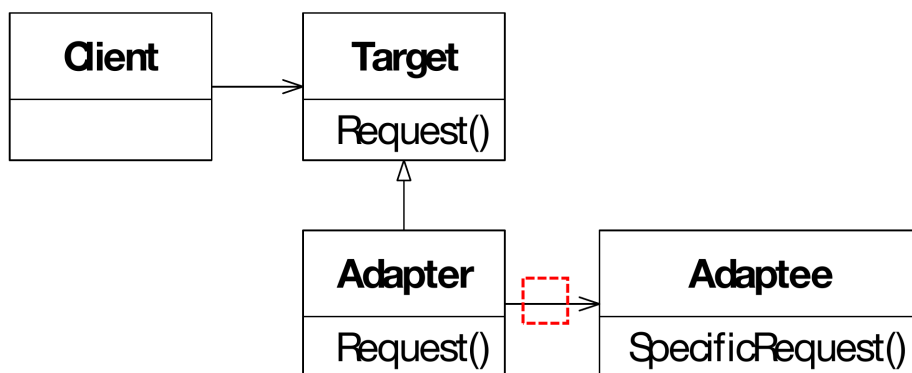
问题：工作量太大，需要自行管理内存。STL 中 vector 已有 push_back()、size()、back()、pop_back() 等方法，**功能上满足要求但接口不一致**，需要进行接口的“转换”。

二、适配器（Adapter）模式

现实类比：手机充电

手机需要 5V 直流电充电，但插座提供的是 220V 交流电。充电头（适配器）将 220V 交流电**转换**为 5V 直流电。手机不需要知道插座的具体电压——它只和充电头的接口打交道。

适配器——实现一



使用组合实现适配，称作对象适配器模式

14

将一个类的接口转换成客户希望的另一个接口，使得原本由于接口不兼容而不能一起工作的类可以在统一接口环境下工作。

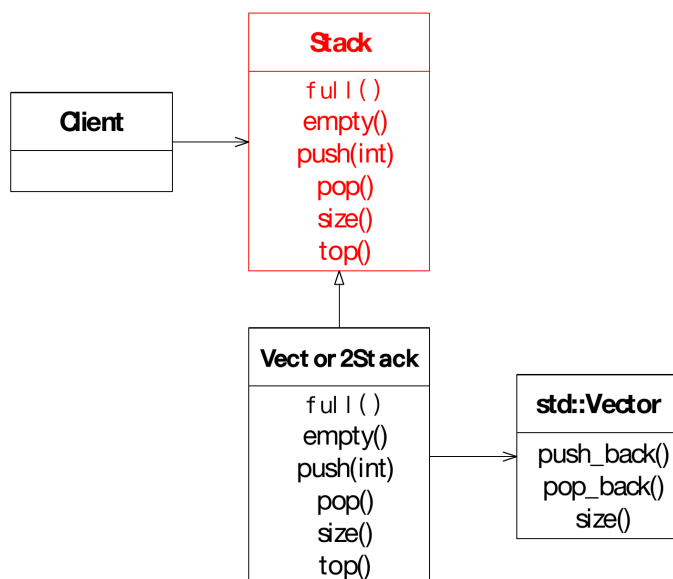
1. **目标 (Target)**：客户所期待的接口
2. **被适配类 (Adaptee)**：需要适配的类
3. **适配器 (Adapter)**：包装被适配类，把原接口转换成目标接口

适用场景

1. 系统需要复用已有的类，但这些类的接口不符合系统的接口
2. 接入第三方组件，但组件接口定义与自身定义不同
3. 旧系统开发的类已经实现了一些功能，但是客户端只能以新接口的形式访问，且不希望手动更改原有类

实现一：组合方式（对象适配器模式）

适配器基类定义



15

Vector2Stack 内部组合一个 `std::vector<int>`，将 Stack 接口翻译为 vector 调用：

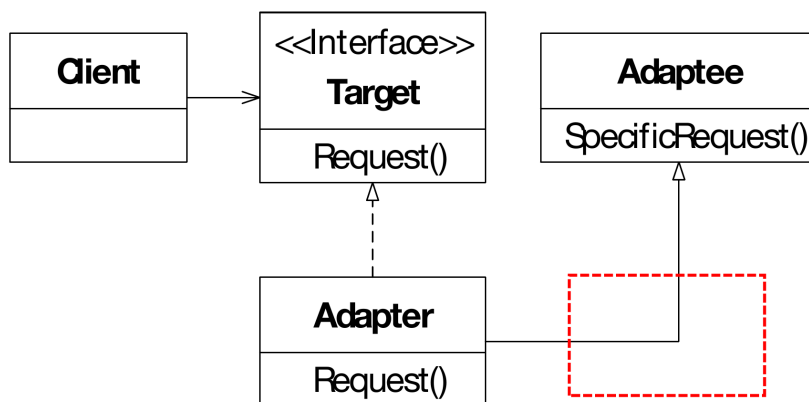
```

class Vector2Stack : public Stack {
private:
    std::vector<int> m_data;
    const int m_size;
public:
    Vector2Stack(int size) : m_size(size) { }
    bool full() { return (int)m_data.size() >= m_size; }
    bool empty() { return (int)m_data.size() == 0; }
    void push(int i) { m_data.push_back(i); }
    void pop() { if (!empty()) m_data.pop_back(); }
    int size() { return m_data.size(); }
    int top() {
        if (!empty()) return m_data[m_data.size()-1];
        else return INT_MIN;
    }
};

```

实现二：继承方式（类适配器模式）

适配器——实现二



使用继承实现适配，称作类适配器模式

20

Vector2Stack 同时私有继承 std::vector<int> 和公开继承 Stack:

```

class Vector2Stack : private std::vector<int>, public Stack {
private:
    int m_size;
public:
    Vector2Stack(int size) : m_size(size) { reserve(size); }
    bool full() { return false; }
    bool empty() { return vector<int>::empty(); }
    void push(int i) { push_back(i); }
    void pop() { pop_back(); }
    int size() { return vector<int>::size(); }
    int top() { return back(); }
};

```

私有继承使得外界只能接触到 Vector2Stack 中的接口，无法直接访问 vector 的方法。

适配器模式小结

1. 通过适配器，客户端可以用统一接口调用各种底层类，**提高代码复用率**
2. 将目标类和适配者类解耦，无需修改原有代码
3. 适用场景：复用已有类但接口不一致、接入第三方组件、旧系统迁移

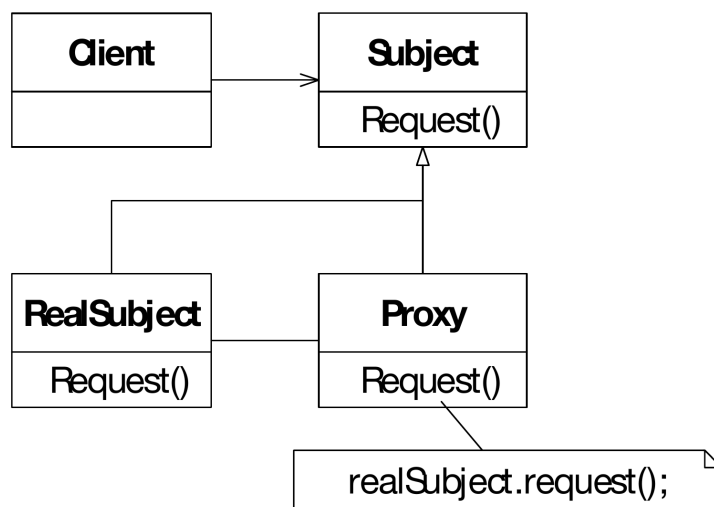
三、代理/委托（Proxy）模式

从适配器到代理：为什么需要新模式？

C++ 中指针使用需要格外小心，尤其是类中有指针成员时，析构与释放是一个棘手的问题。我们想实现智能指针——能够包裹原始指针，具有指针的各项功能，并能在引用计数归零时自动释放。

使用**适配器模式**可以进行指针的封装，对外提供指针各项功能的接口。但是适配器模式**仅仅是接口的转换**，无法在提供接口的同时进行**引用计数控制**这样的额外功能。我们如何在提供功能的同时进行计数控制呢？→ 代理/委托模式！

代理/委托



32

在一些应用中直接访问对象会带来问题（远程访问、创建开销大、安全控制），可以在被访问对象上加上一个**访问层**，将复杂操作包裹在内部，仅对外暴露功能接口——这就是代理/委托模式。

场景

1. **远程代理**：要访问的对象在远程机器上
2. **资源安全**：多进程编程中检查访问权限
3. **创建开销大**：被访问对象创建开销很大，需要延迟初始化
4. **复杂处理**：被访问对象需要根据访问者的行为做出诸多复杂处理

权限检查示例

```
void Proxy::request() {
    if (checkAuthority(nowProcess, nowResource)) {
        // do something...
    }
}
```

代理 vs 适配器：相似与不同

相似之处：

1. 均是在被访问对象之上进行封装
2. 均提供被封装对象的功能接口供外部使用

不同之处：代理的核心要素是**分割访问对象与被访问对象以减少耦合**，接口不变；适配器的核心要素是**变换接口**。代理可能会增加控制功能（引用计数、权限检查等），适配器不会。

例子：智能指针引用计数

```
// 辅助指针，存储指针计数及封装实际指针地址
template <typename T>
class U_Ptr {
private:
    friend class SmartPtr<T>;
    U_Ptr(T *ptr) : p(ptr), count(1) { }
    ~U_Ptr() { delete p; }
    int count;
    T *p;
};
```

```
template <typename T>
class SmartPtr {
private:
    U_Ptr<T> *rp;
public:
    SmartPtr(T *ptr) : rp(new U_Ptr<T>(ptr)) { }
    SmartPtr(const SmartPtr<T> &sp) : rp(sp.rp) { ++rp->count; }
    SmartPtr& operator=(const SmartPtr<T>& rhs) {
        ++rhs.rp->count;
        if (--rp->count == 0) delete rp;
        rp = rhs.rp;
        return *this;
    }
    ~SmartPtr() {
        if (--rp->count == 0) delete rp;
    }
    T& operator*() { return *(rp->p); }
    T* operator->() { return rp->p; }
};
```

使用代理包裹指针，之后操作均通过代理进行：

```
int *i = new int(2);
SmartPtr<int> ptr1(i);
SmartPtr<int> ptr2(ptr1);
SmartPtr<int> ptr3 = ptr2;
cout << *ptr1 << endl;
*ptr1 = 20;
cout << *ptr2 << endl;
```

“变”与“不变”

SmartPtr<int> 与 int* 有相同的接口（*、->、赋值、析构），但增加了引用计数的控制操作：

1. 拷贝构造时引用计数加一
2. 析构时引用计数减一，为 0 时释放
3. 赋值时分别处理当前和参数的引用计数

代理类好比被代理类的“经纪人”：一方面提供被代理类所有接口的功能，另一方面可进行额外的控制操作（引用计数、权限控制、远程代理、延迟初始化等）。

代理 vs 适配器

	适配器	代理
--	-----	----

核心目的	变换接口	分割访问对象与被访问对象，减少耦合
接口变化	可能会改变接口	不会改变接口
额外控制	不会增加控制	可能增加控制功能

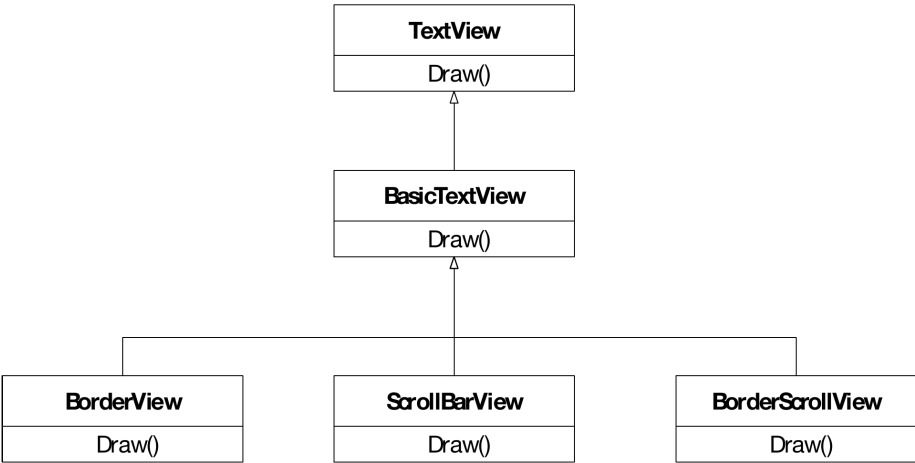
四、装饰器（Decorator）模式

引例：TextView

有一个对象 TextView 在窗口中显示文本，希望**接口不变**，增加滚动条、边框等功能。

用继承？——类爆炸

继承

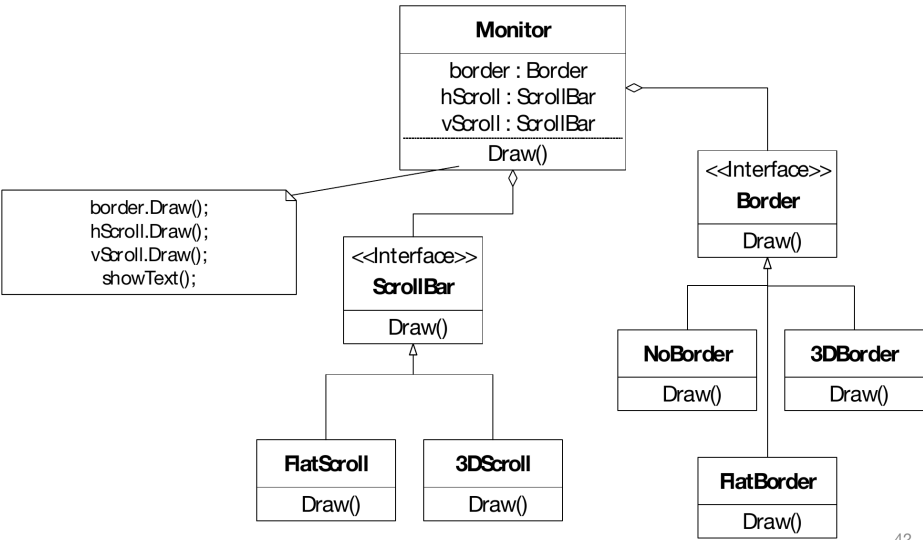


40

随着功能增多，继承类的数量急剧膨胀——最大派生类数目可以是所有功能的组合数。且基类新增接口时所有派生类都需修改。

用策略？——策略个数固定

策略



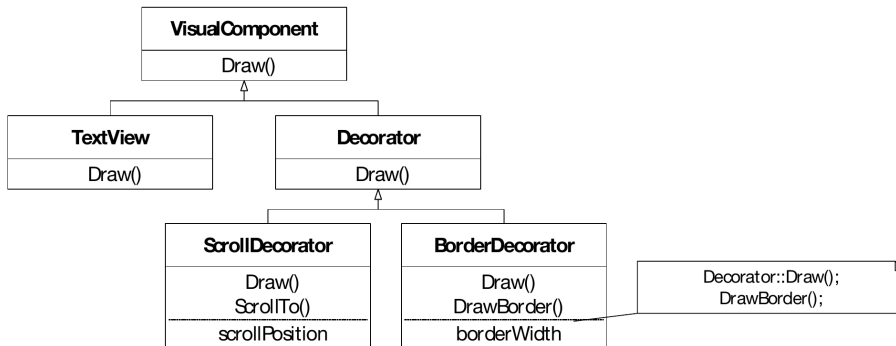
42

策略的个数是基类中预先定义好的，若要增加新功能（如滚动条和边框之外的），就要修改基类。

装饰器

装饰器

- 创建了一个装饰类，用来包装原有的类，并在保持接口完整性的前提下，提供了额外的功能。
- 且装饰类与被包装的类继承于同一基类，这样装饰之后的类可以被再次包装并赋予更多功能。



44

创建装饰类包装原有类，保持接口完整的前提下提供额外功能。装饰类与被包装类继承同一基类，因此装饰后的类可以**再次被包装**，递归增加功能。

```
class Component {
public:
    virtual ~Component() { }
    virtual void draw() = 0;
};

class TextView : public Component {
public:
    void draw() { cout << "TextView." << endl; }
};
```

```
class Decorator : public Component {
    Component* _component;
public:
    Decorator(Component* component) : _component(component) { }
    virtual void addon() = 0;
    void draw() {
        addon();
        _component->draw();
    }
};

class Border : public Decorator {
public:
    Border(Component* c) : Decorator(c) { }
    void addon() { cout << "Bordered "; }
};

class HScroll : public Decorator {
public:
```

```

HScroll(Component* c) : Decorator(c) { }
void addon() { cout << "HScrolled "; }
};

class VScroll : public Decorator {
public:
    VScroll(Component* c) : Decorator(c) { }
    void addon() { cout << "VScrolled "; }
};

```

使用——逐层包装：

```

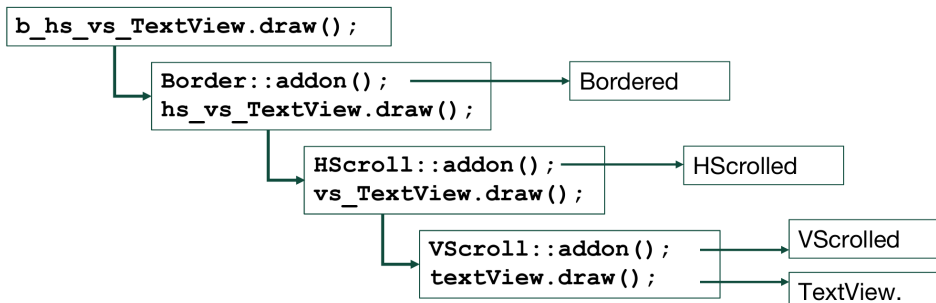
TextView textView;
VScroll vs_TextView(&textView);
HScroll hs_vs_TextView(&vs_TextView);
Border b_hs_vs_TextView(&hs_vs_TextView);
b_hs_vs_TextView.draw();
// 输出: Bordered HScrolled VScrolled TextView.

```

调用链（逐层展开）

运行过程与结果

Bordered HScrolled VScrolled TextView.



49

每个对象无需了解整个链的全貌。每一次都是将之前的版本完全包裹住再增加新功能——有多少个新功能就包裹几次。

调用 b_hs_vs_TextView.draw() 时的执行过程：

```

b_hs_vs_TextView.draw()
→ Border::addon()          输出: "Bordered "
→ hs_vs_TextView.draw()
  → HScroll::addon()        输出: "HScrolled "
  → vs_TextView.draw()
    → VScroll::addon()      输出: "VScrolled "
    → textView.draw()       输出: "TextView."
// 最终输出: Bordered HScrolled VScrolled TextView.

```

装饰器的核心内涵

用装饰器类整体包裹改动之前的类，以保留原来的全部接口。在原来接口保留的基础上进行新功能扩充。装饰器类和被包装类继承同一基类，因此装饰后的类可以**再次被包装**——这就是递归嵌套的基础。

装饰 vs 策略 vs 代理

装饰 vs 策略：装饰侧重于**功能组装**（以组合替代继承），策略侧重于**功能选择**（算法可替换）。策略修改功能的内核（行为），装饰修改功能的外壳（结构）。

装饰 vs 代理：都把“改变对象行为”作为目标，可以粗略地把“装饰”看成是一连串的“代理”。区别在于：装饰不创建被装饰对象，只是将新功能添加到已有对象上；代理被代理对象不存在时常创建被代理对象。装饰经常多重嵌套，代理少见多重嵌套。

	策略	装饰	代理
核心	修改功能内核（行为）	修改功能外壳（结构）	对被代理对象精细控制
组件认知	组件须了解有哪些策略	组件无需了解有哪些装饰	被代理对象不存在时常创建
嵌套	不常见	经常多重嵌套	少见多重嵌套

五、总结

结构型设计模式关心对象组成结构上的抽象，包括接口、层次、对象组合等。核心在于**抽象结构层次上的不变量**，尽可能减少类与类之间的联系与耦合，以最小代价支持新功能的增加。

1. **适配器模式**：在类与类之间进行转接，提高类的复用度与灵活性
2. **代理/委托模式**：减少类与类层次间的耦合，使各类职责清晰
3. **装饰器模式**：动态扩展被装饰类的功能，并留有接口持续扩展

拾叁 创建型模式 2026-06-02

一、单例（Singleton）模式

引例：全局计数器

一个在多个地方被调用的计数器：

```
void func() {  
    addCount();  
}  
int main() {  
    addCount();  
    func();  
    cout << getCount() << endl; // 2  
}
```

如何实现 addCount 和 getCount 函数？

方法一：全局变量

最直接的方法：

```
int count = 0;  
void addCount() {  
    count += 1;  
}  
int getCount() {  
    return count;  
}
```

好的设计应当避免全局变量！ 用户可能访问到 count 并修改数据，不安全。

方法二：静态方法

定义一个类，将函数实现为静态方法：

```

class Counter {
private:
    static int count;
public:
    static void addCount() { count += 1; }
    static int getCount() { return count; }
};
int Counter::count = 0;

Counter::addCount();
cout << Counter::getCount() << endl;

```

静态方法 + 虚函数 = 编译错误

如果我们有多种不同的 Counter:

```

class BaseCounter {
public:
    static virtual void addCount() = 0; // 编译错误！
    // ...
};

```

静态方法不可以是虚的！ 静态方法只与类相关而不与实例相关，调用时必须知道类名。

我们想用静态方法实现的是：无论在何处调用，都会访问到相同的函数和变量。也即，**类存在全局唯一的实例**。

单例模式实现

所谓单例，就是**只能构造一份实例**的类：

```

class Counter {
    // 显式删除拷贝构造函数与赋值操作符
    Counter(const Counter &) = delete;
    void operator=(const Counter &) = delete;
    int count;
    Counter() { count = 0; } // 构造函数为private
    ~Counter() {}           // 析构函数也为private
    static Counter _instance; // 全局唯一的实例
public:
    static Counter &instance() {
        return _instance;
    }
    // 成员函数而非静态方法
    void addCount() { count += 1; }
    int getCount() { return count; }
};
// 定义类中的静态成员，单例在此被初始化
Counter Counter::_instance;

```

调用单例：

```

int main() {
    // 由于删去了拷贝构造函数，必须存为引用
    Counter &c = Counter::instance();
    c.addCount();
    cout << c.getCount() << endl;
}

```

惰性初始化 (Lazy Initialization)

能否在使用时再构造单例实例?

```
class Counter {
    // ...
public:
    static Counter &instance() {
        static Counter _instance;
        return _instance;
    }
    // ...
};
```

在第一次调用 `instance` 方法时才会构造单例。

单例模式的陷阱

需要避免的情况:

1. **实例被重复构造**: 由于构造函数为 `private`, 且拷贝构造函数、赋值操作符被显式删除, 故无法重复构造。
2. **实例被意外删除**: 考虑下面的代码:

```
Counter &c = Counter::instance();
delete &c; // 可以成功执行!
c.addCount(); // 运行时错误
```

应当把析构函数也设为 `private`!

```
class Counter {
private:
    ~Counter() {}
    // ...
};
// error: calling a private destructor of class 'Counter'
```

单例模式 + 虚函数

回到一开始的问题——多种 `Counter` 的多态:

```
class BaseCounter {
public:
    virtual void addCount() = 0;
    virtual int getCount() = 0;
};

class SimpleCounter : public BaseCounter {
    // ... 单例相关的一大堆逻辑 ...
    int count;
    SimpleCounter() { count = 0; }
public:
    virtual void addCount() { count += 1; }
    virtual int getCount() { return count; }
};

class NotSimpleCounter : public BaseCounter { ... };
```

使用:

```

void doStuff(BaseCounter *counter) {
    counter->addCount();
    counter->addCount();
    cout << counter->getCount() << endl;
}
int main() {
    doStuff(&SimpleCounter::instance()); // 2
    doStuff(&NotSimpleCounter::instance()); // ...
    doStuff(&SimpleCounter::instance()); // 4
}

```

唯一的不便：单例相关的逻辑较多，有很多重复代码，却无法在基类中实现——因为基类不知道派生类的类别。

CRTP：奇特的递归模板模式

Curiously Recurring Template Pattern (CRTP):

```

template <class Derived> // 模板参数为派生类类型
class Singleton {
    Singleton(const Singleton &) = delete;
    void operator=(const Singleton &) = delete;
protected:
    Singleton() {}
    virtual ~Singleton() {}
public:
    static Derived &instance() { // 魔法在此发生
        static Derived _instance;
        return _instance;
    }
};

```

“奇特的递归模板模式”

■ “奇特的递归模板模式”：Curiously Recurring Template Pattern (CRTP)

- `template <class Derived>` // 模板参数为派生类类型


```

class Singleton {
    Singleton(const Singleton &) = delete;
    void operator=(const Singleton &) = delete;
protected:
    Singleton() {}
    virtual ~Singleton() {}
public:
    static Derived &instance() { // 魔法在此发生
        static Derived _instance;
        return _instance;
    }
};

```

17

CRTP + 多重继承

基于 Singleton 类实现计数器派生类：

```

class SimpleCounter : public BaseCounter,
                    public Singleton<SimpleCounter> {

```

```
// 友元声明是必要的，因为Singleton类需要访问派生类的
// 构造函数，而为了实现单例，构造函数是私有的
friend class Singleton<SimpleCounter>;
// ... 只需实现计数器逻辑即可
};
```

注意：不能直接将 Singleton 类的逻辑实现在 BaseCounter 类中，否则不存在“基类指针”。

关于 CRTP

CRTP 是实现多态的另一种方式。与虚函数不同，本质上实现的还是**编译期多态**：

1. 使用虚函数实现：运行时通过虚函数表寻找调用的方法
2. 使用 CRTP 实现：函数需要被实现为模板函数，编译时由编译器为每种被调用的派生类进行模板实例化

关于单例模式

单例模式是存在争议的一种设计模式：

1. **优点：**实现似乎比较简单；以相对安全的形式提供可供全局访问的数据
2. **缺点：**难以完全正确地实现；违反单一职责原则；过度使用会使得实际的依赖关系变得隐蔽

二、工厂方法（Factory Method）模式

引例：泡茶程序

根据客户需求提供绿茶或者红茶：

```
GreenTea *orderGreenTea() {
    GreenTea *greenTea = new GreenTea();
    greenTea->addIngredients(); // 加料
    greenTea->brew();           // 泡茶
    greenTea->pour();           // 装杯
    return greenTea;
}

BlackTea *orderBlackTea() {
    BlackTea *blackTea = new BlackTea();
    blackTea->addIngredients(); // 加料
    blackTea->brew();           // 泡茶
    blackTea->pour();           // 装杯
    return blackTea;
}
```

代码存在大量冗余！

抽出公共逻辑并简化：

```
Tea *orderTea(string type) {
    Tea *tea;
    if (type == "GreenTea")
        tea = new GreenTea;
    else if (type == "BlackTea")
        tea = new BlackTea;
    tea->addIngredients(); // 加料
    tea->brew();           // 泡茶
    tea->pour();           // 装杯
    return tea;
}
```

类似的构造过程可能在许多地方用到，进一步抽象为一个方法。

工厂方法

在 Tea 类中添加一个 factory 静态方法：

```
class Tea {
    // ...
public:
    static Tea *factory(string type) {
        if (type == "GreenTea")
            return new GreenTea;
        else if (type == "BlackTea")
            return new BlackTea;
        // ... 其他可能的茶叶类型
    }
};
```

这一方法被称为“工厂方法”。

单独的工厂类

当构造逻辑过于复杂，或者有必要进行分离时，可以把工厂方法放在单独的类中：

```
class TeaFactory {
public:
    void setMilk(int amount) { ... }
    void setSugar(int amount) { ... }
    Tea *createTea(string type) {
        Tea *tea = nullptr;
        if (type == "GreenTea")
            tea = new GreenTea;
        else if (type == "BlackTea")
            tea = new BlackTea;
        if (milkAmount > 0) tea->addMilk(...);
        if (sugarAmount > 0) tea->addSugar(...);
        // ...
    }
};
```

工厂方法的用途

1. 包装复杂的构造逻辑
2. 为重载的构造函数提供描述性名称
3. 对象构造需要用到当前函数体无法访问的信息
4. 需要集中管理被构造对象的生命周期

构造函数的描述性名称

一些类可能具有多个重载的构造函数，可以改写为工厂方法以使用描述性的名称：

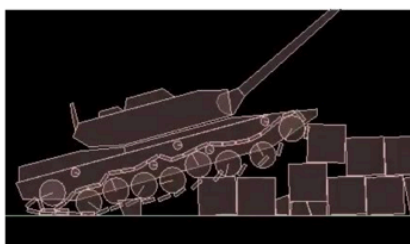
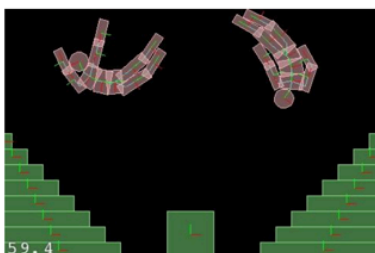
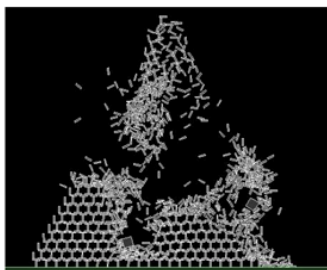
```
class Complex { // 复数类
    float real, imag; // 实部、虚部
    Complex(float real, float imag) { ... }
public:
    // 笛卡尔坐标
    static Complex fromCartesian(float x, float y) {
        return Complex(x, y);
    }
    // 极坐标
    static Complex fromPolar(float r, float a) {
        return Complex(r * cos(a), r * sin(a));
    }
};
```

```
}  
};
```

一个现实的例子：Box2D

一个现实的例子： Box2D

- **Box2D** 是一款高性能、工业级的 2D 物理引擎。它在处理物理对象（如刚体、碰撞体）的创建时，并没有直接让用户使用 **new** 操作符，而是采用了严格的工厂模式。



30

Box2D 是一款高性能、工业级的 2D 物理引擎。它在处理物理对象（如刚体、碰撞体）的创建时，并没有直接让用户使用 **new** 操作符，而是采用了严格的工厂模式。

Box2D 中的主要类型

	b2World（物理世界）	b2BodyDef（配置描述）	b2Body（刚体实例）
角色	管理所有刚体的“容器”；负责物理步进、碰撞检测等核心逻辑；持有内存分配器	纯数据对象，无业务逻辑，可复用，可序列化；作用：解耦配置和创建	构造函数为 private；只能通过 <code>b2World::createBody</code> 创建；内部持有 <code>b2World</code> 引用，用于内存管理

构造 `b2Body` 必须使用工厂方法：

```
b2World *world = new b2World(...);  
b2BodyDef bodyDef(...);  
b2Body *body = world->createBody(&bodyDef);
```

原因有二：

1. `b2Body` 在构造时需要用到 `b2World` 的私有成员
2. Box2D 需要频繁申请并释放小块内存，为高效处理，自己实现了小块内存分配器 `b2BlockAllocator` 并手动管理内存。通过工厂方法，可以避免用户误用系统的内存分配机制

优势：

1. 封装复杂创建逻辑（内存分配、世界注册、碰撞初始化等）
2. 保证对象状态一致性
3. 支持未来扩展（如添加新类型刚体，只需改工厂逻辑即可）

另外，使用 `b2BodyDef` 可以避免在函数调用中标明所有属性，也可以方便地构造多个属性相同或类似的 `b2Body` 对象。

三、抽象工厂（Abstract Factory）模式

工厂方法的局限性

工厂方法的目的是构造**单个类**的对象。如果我们要构造的是**多个类**的对象，而且有特定的组合方式呢？

一个很现实的例子：编译器前端——由代码构建统一格式的中间表示，需要三个步骤：语法分析、语义分析、中间代码生成。对于每种语言来说，每个步骤的实现不同，同一种语言应该使用同一种实现。不妨假设有两种语言：C++ 和 Java。

编译器前端设计

三个步骤的类：

1. 语法分析 Lexer: CppLexer / JavaLexer
2. 语义分析 Parser: CppParser / JavaParser
3. 中间代码生成 Generator: CppGenerator / JavaGenerator

框架的实现：

```
class Compiler {
    string type;
public:
    Compiler(string type) { this->type = type; }
    LexResult *lex(Code *input) {
        Lexer *lexer;
        if (type == "cpp")
            lexer = new CppLexer;
        else if (type == "java")
            lexer = new JavaLexer;
        return lexer->lex(input);
    }
    ParseResult *parse(LexResult *input) {
        Parser *parser;
        if (type == "cpp")
            parser = new CppParser;
        else if (type == "java")
            parser = new JavaParser;
        return parser->parse(input);
    }
    // ...
};
```

问题：当前实现有过多的代码重复。如果要添加新语言支持，需要在每个步骤中加一条 if 分支。

添加一层抽象

设计一个基类，抽象同一语言所需的所有步骤：

```
class AbstractFactory {
public:
    virtual Lexer *createLexer();
    virtual Parser *createParser();
    virtual Generator *createGenerator();
};

class CppFactory : public AbstractFactory {
public:
    Lexer *createLexer() { return new CppLexer; }
    Parser *createParser() { return new CppParser; }
    Generator *createGenerator() { return new CppGenerator; }
};
```

```
class JavaFactory : public AbstractFactory {
    // ... Java 的三个步骤实现
};
```

修改框架实现：

```
class Compiler {
    AbstractFactory *factory;
public:
    Compiler(AbstractFactory *factory) {
        this->factory = factory;
    }
    LexResult *lex(Code *input) {
        return factory->createLexer()->lex(input);
    }
    ParseResult *parse(LexResult *input) {
        return factory->createParser()->parse(input);
    }
    // ...
};
```

抽象工厂模式

将刚才的类对应到术语：

术语	对应类
抽象工厂	AbstractFactory
具体（Concrete）工厂	CppFactory、JavaFactory
抽象产品	Lexer、Parser、Generator
具体产品	CppLexer、JavaParser、.....

抽象工厂模式即，将同类的具体产品包装为一个具体工厂，以抽象工厂的形式呈现给上层代码；上层代码只需要关心抽象工厂和抽象产品，而不需要知道具体的工厂和产品是哪些。

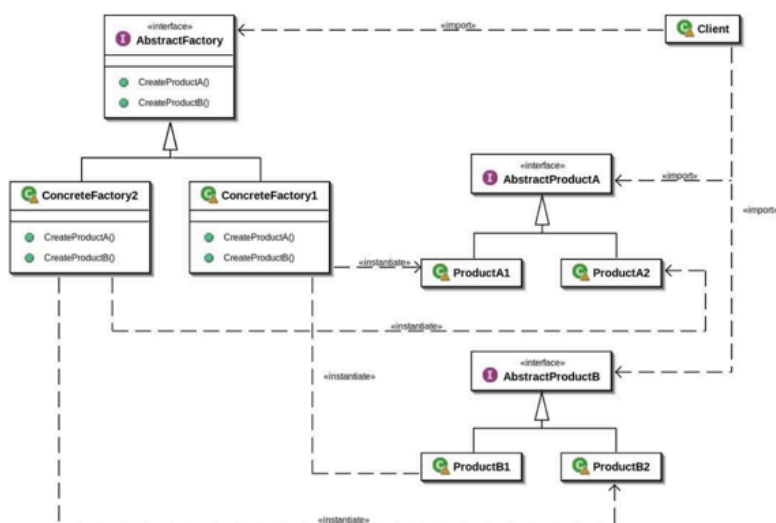
框架设计理念

编译器前端框架如此设计有其合理性：

1. **分阶段进行可以解耦合，方便测试**：语法分析、语义分析、代码生成各自独立，可以分别测试每个阶段
2. **延迟初始化可以缩短 Compiler 类的构造时间**：具体工厂在需要时才传入，Compiler 类构造时无需等待所有工厂就绪

抽象工厂模式

■ 抽象工厂模式的UML图如下：



40

四、设计模式总结

三大设计模式回顾

行为型模式 (Behavioral Patterns)

关注对象行为功能上的抽象，提升对象在行为功能上的可扩展性，能以最少的代码变动完成功能的增减。常用于描述对类和对象的交互与职责分配。

1. **模板方法模式**：定义算法骨架，将具体步骤的实现放到子类中实现。可以在不改变算法流程的情况下，自定义某些步骤
2. **策略模式**：定义一类算法，将每个算法分别封装，不同算法可以相互替换
3. **迭代器模式**：用于遍历数据集合（数组、链表、树、图等），解耦算法与数据访问

结构型模式 (Structural Patterns)

关注对象之间结构关系上的抽象，从而提升对象结构的可维护性、代码的健壮性，能在结构层面上尽可能解耦合。常用于处理类和对象的组合关系。

1. **适配器模式**：将不兼容的接口转换为可兼容的接口
2. **代理/委托模式**：在不改变原始类接口的条件下，为原始类定义一个代理类，增加控制访问
3. **装饰器模式**：用组合来替代继承，给原始类添加增强功能

创建型模式 (Creational Patterns)

将对象的创建与使用进行划分，从而规避复杂对象创建带来的资源消耗，能以简短的代码完成对象的高效创建。用于对象的创建。

1. **单例模式**：用来创建全局唯一的对象
2. **工厂方法模式**：用来创建不同但相关类型的对象，由给定的参数来决定创建哪种类型的对象
3. **抽象工厂模式**：提供一个创建一系列相关或相互依赖对象的接口，而无需指定它们具体的类

七大设计原则

原则	说明
开闭原则	一个软件实体（类、模块、函数）应该对扩展开放，对修改关闭。最基础的设计原则

单一职责原则	每个类应该只有一个职责，只有一个原因可以引起它的改变。例如：迭代器模式使得数据结构与算法分离；可视化程序设计中页面与逻辑分离
里氏代换原则	只要父类出现的地方子类就可以出现，即子类尽量不修改父类的数据与方法，实现基类代码的充分复用
依赖倒转原则	要依赖于抽象，不要依赖于具体。针对接口编程，而不是针对实现编程。具体而言就是上层模块不应该依赖底层模块。例如：策略模式/模板方法模式不依赖于具体的策略实现，只依赖于抽象
接口隔离原则	不要建立臃肿庞大的接口。接口尽量细化的同时接口中的方法尽量少。功能拆分粒度太小，将使得类、接口的数量过多；功能拆分粒度太大，将使得类之间耦合度高，程序不灵活
迪米特原则	最少知道原则，一个对象应该对其他对象有最少的了解，使得功能模块相对独立
合成复用原则	在新对象里通过关联关系（委派调用）来使用已有的对象，使之成为新对象的一部分；优先考虑使用组合而不是继承。如需要使用继承，则遵守里氏代换原则

注意：在程序设计中尽量遵循七大原则，但也需根据实际情况调整，切勿滥用设计模式使得代码过度冗余。